

C - C++

C jezik je izmislio i implementirao Dennis Ritchie na DEC PDP-11 stroju koristeći UNIX operativni sistem, kao rezultat razvoja počeo sa jednim od starijih programskih jezika, BCPL, koji je još ponegdje u upotrebi.

Iz BCPL-a se razvio jezik zvanim B, a za to je zaslužan Ken Thompson. Otuda i naziv C jeziku, kao logičan nastavak razvoja programskog jezika.

Dugi niz godina, defacto standard C jezika je bio onaj ponudjen sa UNIX verzijom 5 operativnog sistema. 1983 osnovan je komitet koji je kreirao ANSI standard koji je definisao C jezik jednom zauvijek.

Da bi smo mogli raditi na C jeziku, prije svega nam je potreban kompajler, a potom malo znanja kako taj kompajler upotrebiti. Informacije o tome možete naći u lekciji koju smo zgodno nazvali Kompiler.

Potom, mislim da bi bilo dobro pozabaviti se malo osnovnom strukturom C programa, da bi smo mogli početi raditi na složenijim programima.

I za kraj ovog djela našeg tutorijala, mislim da bi bilo dobro obratiti malo pažnju na osnovne greske pri kodiranju, jer najboljim programerima uspije nekako da zabrljaju, tako da ne vidim načina kako da mi to ovdje izbjegnemo.

Sve u svemu, za početak dovoljno.

Ne zaboravite, C jezik je strukturni jezik, i ovaj tutorijal se oslanja na baš tu strukturu, naprosto je nemoguće početi citati sa kraja. Morate proći lekciju po lekciju da bi vam bilo jasno o čemu govorimo. No to će vam postati jasnije tokom tutorijala.

Znači lekcije za danas su:

1: Kompiler

2: Struktura C programa

3: Osnovne greske pri kodiranju

Kada završite naše 3 današnje lekcije, krenut ćemo dalje, na date i varijable, tu je tek interesantno, vidjet ćete.

Naravno, pravilo znate: Za sve *kritike* i *pohvale*, kao i *pitanja* koje vjerovatno imate, ili ako

uocite neku gresku, javite se direktno autoru na behar@bhwebmasters.net kada je C tutorijal u pitanju, ili na emir@jusic.de za C++.

Ovim se ogradjujem od bilo koje odgovornosti u vidu bilo cega ovdje, a specificno mislim na C/C++ programski jezik, pored ostalih stvari koje vam moze pasti na pamet. Sa ovim tutorijalom vam predstavljam programiranje u C jeziku na onaj nacin na koji ja vidim shodno. Ne tvrdim da je tutorijal sveobuhvatan, niti da je 100% tacan, ali vam izrazito tvrdim da ste sposobni savladati C programski jezik ukoliko mozete pratiti ovaj tutorijal, i ukoliko je to slucaj, onda vas urgiram da se toga sto prije prihvatite. Nigdje ni na koji nacin ne polazem bilo koje pravo na posao osobe koja se bavi obrazovnim radom, kao ni pravo na bilo koji trademark ovdje naveden.

Data i varijabile

[prethodno](#) [iduce](#)

U C jeziku postoje 5 osnovnih vrsta (tipova) date (vrjednosti): karakter (character - **char** - 8 bita), cijeli broj (integer - **int** - 16 bita), dugi int (long integer - **long** - 16 bita, decimalni brojevi (floating point - **float** - 32 bita) i dupli decimali (double floating point - **double** - 64 bita velicine). int, short i long mogu imati predznak, znaci na predznak otpada jedan bit.

Char se koristi za male vrjednosti (0 do 255). Na primjer, ASCII codovi su uobicajno pothranjeni u char dati. Tako da nam char data tip moze koristiti za unosenje podataka posredstvom tastiture i scanf() funkcije.

int recimo koristimo u varijabili koja broji dane u godini, a long int za broj kilometara (120 000 000). Pitanje koje se postavlja je, koliko moze brojcano stati u pojedinoj vrsti varijabile?

Odgovor sljedi u sljedecoj [tabeli](#).

Nekoliko poenti:

Posto su short i int najvjerojatnije u istom rasponu na vasem stroju, short necemo bas mnogo koristiti ovdje.

Pitate se sta bi se desilo da u pojedinoj vrsti varijabile smestite broj koji prevazilazi vrjednosti koje su ovdje naznacene? Odlicno, bas sam o tome htio nesto reci.

Recimo da kao brojcanik u nekom nasem programu postavimo varijabilu tipa int koja broji slova ukucana na tastituri pri uniosenju texta. I recimo da je taj brojcanik dostigao vrjednost od 32 766. Pri iducem prisitku tastiture, vrjednost se povecava za jedan, odn. 32 767. Pri sljedecem pritisku tastiture vrjednost koja je pohranjena u nasoj varijabili ce bit -32 768 !!! Sta se desilo?

Pa posto smo dosli do gornje granice vrjednosti koja se moze pothraniti u varijabilu tipa int, sljedeca inkrementacija pocinje od njene najnize vrjednosti, odn. -32 768. Na to morate obratiti paznju pri kodiranju programa, tj. da varijabila po svom tipu moze "izdrzati" moguće vrjednosti. Ne moram govoriti kakve posljedice moze imati slican previd na vas program, ali dovoljno je reci da se tako nesto dogadja svima. Jedna od solucija je da za svaku varijabilu koristite jednu od vecih tipova varijabila, ali to je niti prakticno (zbog memorije) niti ispravno.

Kod decimala (float i double) bitno je naponenuti tacnost, odn. do koje decimale funkcionisu. float je tacan do 6. decimale, dok double nudi 12 decimala. Poslje tih vrjednosti cifre se zaokruzuju. Interesantno, zar ne?

Da bi ste koristili varijabilu u vasem programu, morate je deklarirati. Tek je onda mozete koristiti u programu.

Potrebno je da podjelimo riječ-dvije o imenovanju varijabila.

Prosto ko riza. Htjedoh reci ko pita, ali se tu moja zena ne slaze, za nju je pita umjetnost. I da ne bi kojim slucajem prekinuo lanac snabdjevanja najljepsih pita sa ove strane ekvatora, primoran sam se sloziti sa njom. A ja sada odoh na sirnicu, a mi se vidimo sutra, kada cemo govoriti o Stringovima, Konstantama i Funkcijama.

Samo da jos jednom naznacim danasnje lekcije:

1: Tabela Data Vrsta

2: Deklarisanje Varijabila

3: Imenovanje varijabila

[prethodno](#) [iduce](#)

Naravno, pravilo znate: Za sve *kritike* i *pohvale*, kao i *pitanja* koje vjerovatno imate, ili ako uocite neku gresku, javite se direktno autoru na behar@bhwebmasters.net kada je C tutorijal u pitanju, ili na emir@jusic.de za C++.

Ovim se ogradjujem od bilo koje odgovornosti u vidu bilo cega ovdje, a specifino mislim na C/C++ programski jezik, pored ostalih stvari koje vam moze pasti na pamet. Sa ovim tutorijalom vam predstavljam programiranje u C jeziku na onaj nacin na koji ja vidim shodno. Ne tvrdim da je tutorijal sveobuhvatan, niti da je 100% tacan, ali vam izrazito tvrdim da ste sposobni savladati C programski jezik ukoliko mozete pratiti ovaj tutorijal, i ukoliko je to slucaj, onda vas urgiram da se toga sto prije prihvatite. Nigdje ni na koji nacin ne polazem bilo koje pravo na posao osobe koja se bavi obrazovnim radom, kao ni pravo na bilo koji trademark ovdje naveden.

String i funkcije

[prethodno](#) [iduce](#)

Tastiture u ruke, i hajdemo, novi program:

```
#include <stdio.h>

/* Primjer koristenja char varijabila */

main()
{
    char ch;

    ch = 'A';

    printf("%c", ch);

    ch = 'B';

    printf("%c", ch);

    ch = 'C';

    getch();

}
```

Primjetit ce te novi "format code" za printf() funkciju, "%c", koja naredjuje printf() funkciji da na mjesto coda postavi char vrjednost.

Da bi smo dodjelili karakter varijabili, moramo je "zapakovati" izmedju znakova navodnika, kao u primjeru `ch = 'A';`

Rezultat ovog programa je sljedece:



ABC

Sasvim je moguće koristiti scanf() funkciju pri učitavanju sa tastiture, ali za jedno slovo je uobicajnije koristiti funkciju getche(). getche() funkcija ceka dok se tastitura ne pritisne, i vraca ascii code glavnoj funkciji. Da bi smo to objasnili, uradit cemo sljedeci program, koji ce isprintati poruku:



pritisli ste magicnu tipku

ukoliko pritisnete tipku H.

Primjetite molim vas kako u ovom programu koristimo dvije "library" iliti header datoteke, jer je `getche()` dio `conio.h` datoteke.

```
#include <stdio.h>

#include <conio.h>

/* Primjer koristenja char varijabila */

main()
{
    char ch;

    /* Sljedece cita jednu tipku sa tastiture */

    ch = getche();

    if(ch == 'H')
        printf("pritisli ste magicnu tipku\n");

    getch();
}
```

Sljedeća linija je značajna za ovaj program:

```
ch = getche();
```

U varijabli `ch` se smesta povratna vrijednost funkcije `getche()`, i ukoliko je ta vrijednost vrijednost velikog slova `h`, onda je jednacina:

```
ch == 'H'
```

tacna.

"==" je znak usporedbe, i ako su vrijednosti sa obje strane iste, jednacina vraća pozitivnu vrijednost, ukoliko nisu, vraća nulu.

U našem slučaju ako ubacimo bilo koje drugo slovo, znak ili broj, jednacina će biti netacna, i "if" komandi će biti vraćena nula, tako da se komande sadržane u "if"-u neće odvijati. Ali ako je

jednacija tacna, odn. ako pritisnemo veliko slovo 'h', onda je "if" tacan, i sljedeća linija coda će se izvršiti, odn. jedino tada ćemo u ovom programu dobiti ispisanu poruku na ekranu.

Vrlo je bitno da savladate string data tip, i ka tim ciljem smo njima posvetili citavu jednu stranicu.

A kada to savladate, vrijeme je precizirati na funkcije koje koristimo za obradu stringova.

Dalje sljede Koncepti stringa i Funkcija.

I da vas i formalno upoznam sa nekim od najznacajnijih C funkcija, ovaj put su to printf() i fprintf() funkcije.

Jesmo li dosta za danas?

I ja mislim.

Samo da navedemo lekcije koje smo danas presli:

- 1: string data tip
- 2: string i Funkcije
- 3: Koncepti stringa i Funkcija
- 4: printf() i fprintf() funkcije

Hajde onda, pa ćemo sutra malo razgovarati o Operatorima u C jeziku. Kaos!

[prethodno](#) [iduce](#)

Naravno, pravilo znate: Za sve *kritike* i *pohvale*, kao i *pitanja* koje vjerovatno imate, ili ako uocite neku gresku, javite se direktno autoru na behar@bhwebmasters.net kada je C tutorijal u pitanju, ili na emir@jusic.de za C++.

Ovim se ogradjujem od bilo koje odgovornosti u vidu bilo cega ovdje, a specificno mislim na C/C++ programski jezik, pored ostalih stvari koje vam može pasti na pamet. Sa ovim tutorijalom vam predstavljam programiranje u C jeziku na onaj nacin na koji ja vidim shodno. Ne tvrdim da je tutorijal sveobuhvatan, niti da je 100% tacan, ali vam izrazito tvrdim da ste sposobni savladati C programski jezik ukoliko mozete pratiti ovaj tutorijal, i ukoliko je to slucaj, onda vas urgiram da se toga sto prije prihvatite. Nigdje ni na koji nacin ne polazem bilo koje pravo na posao osobe koja se bavi obrazovnim radom, kao ni pravo na bilo koji trademark ovdje naveden.

Operator je simbol koji govori kompileru da izvrši određenu matematičku ili logičku operaciju.

Mislim da je ovim sve rečeno, i onima koji su uspjeli shvatiti bit teme mogu preci odmah na sljedeću lekciju.

Nama ostalima ostaje da se baccemo malo matematikom i logikom, 'ja veselja!

OK, ukratko, u C jeziku postoje 3 opste grupacije operatora: aritmeticke, relacionalne i logicke, i 'bitwise' operatori.

Posebni operatori postoje za pojedine zadace, no to je zasada van domena ovog tutorijala, i zadržat ćemo se ovdje vecinom na prve dvije grupacije operatora. Po potrebi ćemo možda spomenuti malo i ostale, no vidjet ćemo...

Assingment operator takodje zauzima posebno mjesto, vrjedit će vam da se pozabavite snjim....

Kada imamo na umu operatore, vrlo bitno je razumjeti redosljed po kojem se oni odvijaju u programu. Naime, svima je poznato da recimo množenje obavljamo prije sabiranja u aritmetici, i da je moguće poremetiti taj 'prirodan' odnos računskih radnji postavljanjem zagrada. E, pa isto tako nešto postoji u C jeziku, i o njima govorimo u lekciji Operatori i Presedani. Naime, sasvim je jedno znati upotrijebiti operator, ali ako uprskamo redosljed odvijanja radnji, konstantno ćemo dobijati pogresne rezultate, a da nismo svjesni gdje grijesimo. Ovako nemamo opravdanje. Ignorantnost nije opravdanje:-))

OK, toliko za ovaj put.

Dozvolite da samo jos jedno navedemo danasnje lekcije, pa da onda idemo dalje u nasim poduhvatima ka shvatanju odnosa onih malih keceva i nulica koje sacinjavaju osnovu svega kompjuterskog....

1: Aritmeticki Operatori u C Jeziku

2: Racionalni i Logicki Operatori

3: Assignment Operator

4: Operatori i Presedani

Naravno, pravilo znate: Za sve *kritike* i *pohvale*, kao i *pitanja* koje vjerovatno imate, ili ako

uocite neku gresku, javite se direktno autoru na behar@bhwebmasters.net kada je C tutorijal u pitanju, ili na emir@jusic.de za C++.

Ovim se ogradjujem od bilo koje odgovornosti u vidu bilo cega ovdje, a specifino mislim na C/C++ programski jezik, pored ostalih stvari koje vam moze pasti na pamet. Sa ovim tutorijalom vam predstavljam programiranje u C jeziku na onaj nacin na koji ja vidim shodno. Ne tvrdim da je tutorijal sveobuhvatan, niti da je 100% tacan, ali vam izrazito tvrdim da ste sposobni savladati C programski jezik ukoliko mozete pratiti ovaj tutorijal, i ukoliko je to slucaj, onda vas urgiram da se toga sto prije prihvatite. Nigdje ni na koji nacin ne polazem bilo koje pravo na posao osobe koja se bavi obrazovnim radom, kao ni pravo na bilo koji trademark ovdje naveden.

Expressions i statments

[prethodno](#)

[iduce](#)

Operatori, konstante i varijabile su djelovi Expressiona (izraza).

Expression je bilo koja ispravna kombinacija tih djelova. Ali, iz razloga sto vecina izraza sljede opsta pravila algebre, uzeta su zdravo - za - gotovo. Cinjenica je da postoji nekoliko aspekata koje su specifini za C jezik, i o njima cemo po koju ovdje podjeliti.

1: [Type Converzija u izrazima](#)

2: [Casts, Slobodna mjesta i Zagrade](#)

3: [Tvrdnje i Izrazi](#)

I sa ovim nekako završavamo prvi dio ovog kursa. Sljedi tok programa, i tu cemo se zavuci malo sa osnovnim uslovnim izrazima (for, if, while...). Kazem vam, ludnica ziva. Ali! Tek tu postaje pravo interesantno, tako da vas sviju ocekujem na iducim predavanjima... A do tada, imate slobodno....

[prethodno](#)

[iduce](#)

Naravno, pravilo znate: Za sve *kritike* i *pohvale*, kao i *pitanja* koje vjerovatno imate, ili ako uocite neku gresku, javite se direktno autoru na behar@bhwebmasters.net kada je C tutorijal u pitanju, ili na emir@jusic.de za C++.

Ovim se ogradjujem od bilo koje odgovornosti u vidu bilo cega ovdje, a specifino mislim na C/C++ programski jezik, pored ostalih stvari koje vam moze pasti na pamet. Sa ovim tutorijalom vam predstavljam programiranje u C jeziku na onaj nacin na koji ja vidim shodno. Ne tvrdim da je tutorijal sveobuhvatan, niti da je 100% tacan, ali vam izrazito tvrdim da ste sposobni savladati C programski jezik ukoliko mozete pratiti ovaj tutorijal, i ukoliko je to slucaj, onda vas urgiram da se toga sto prije prihvatite. Nigdje ni na koji nacin ne polazem bilo koje pravo na posao osobe koja se bavi obrazovnim radom, kao ni pravo na bilo koji trademark ovdje naveden.

Tok programa

[prethodno](#)

[iduce](#)

Ukratko, u ovom poglavlju cemo govoriti o osnovnim tvrdnjama i operatorima koje se koriste da bi se kontrolisao tok progama u C jeziku.

Pocet cemo sa 'if' kontrolom, i njemu blisko vezanom 'if-else' i 'else-if' kontrolom programa, kao i switch statmentu.

Kasnije cemo preci na 'while' i 'for' strukture koje kontrolisu loop (petlju).

Program-control naredbesu osnova svakog kompjuterskog jezika, jer uslovljavaju tok programskog izvršenja. Nacin na kojem su implementirani u pojedinim jezicima, odredjuje 'licnost' programskog jezika. Turbo C posjeduje program-control naredbe koje su u isto vrijeme snazne i bogate, i pomazu u shvatanju popularnosti jezika.

Kontrolne naredbe mozemo svrstati u 3 kategorije:

- Prva je uslovna instrukcija 'if' i 'switch'
- Druga je tvrdnja za kontrolisanje petlje, 'while', 'for' i 'do-while'
- Posljednja kategorija je bezuslovna instrukcija 'goto'.

[prethodno](#)

[iduce](#)

Naravno, pravilo znate: Za sve *kritike* i *pohvale*, kao i *pitanja* koje vjerovatno imate, ili ako uocite neku gresku, javite se direktno autoru na behar@bhwebmasters.net kada je C tutorijal u pitanju, ili na emir@jusic.de za C++.

Ovim se ogradjujem od bilo koje odgovornosti u vidu bilo cega ovdje, a specificno mislim na C/C++ programski jezik, pored ostalih stvari koje vam moze pasti na pamet. Sa ovim tutorijalom vam predstavljam programiranje u C jeziku na onaj nacin na koji ja vidim shodno. Ne tvrdim da je tutorijal sveobuhvatan, niti da je 100% tacan, ali vam izrazito tvrdim da ste sposobni savladati C programski jezik ukoliko mozete pratiti ovaj tutorijal, i ukoliko je to slucaj, onda vas urgiram da se toga sto prije prihvatite. Nigdje ni na koji nacin ne polazem bilo koje pravo na posao osobe koja se bavi obrazovnim radom, kao ni pravo na bilo koji trademark ovdje naveden.

Funkcije

[prethodno](#)

[iduce](#)

Kada govorimo o funkcijama, mislim da je vrlo bitno da porazgovaramo o strukturi prostije funkcije, i da ponudimo instrukcije kako definisati funkciju.

I u krajnjem slucaju, zbog cega nam funkcije i trebaju.

Vrijedi spomenuti i 'return' komandu, jer ce vam u funkcijama svakako trebati.

Opisat cemo kako pozvati funkciju iz druge funkcije, i kako saljemo vrjednosti izmedju funkcija, odn. o koristenju formalnih i actualnih argumenata. Tu cemo malo govoriti i o pravilima dometa funkcija (scope rules).

Upoznat cemo vas takodje i sa adresama i pointerima, tim najstrasnijim monstrumima C jezika, i obuzdati ih da bi smo iskoristili u slanju vrjednosti izmedju funkcija.

I sto se tice ove lekcije, mislim da sam vam dosta servirao, ako ne i previse. No vidjet cemo na kraju.

A do tada, zelim vam cisto codiranje...

[prethodno](#)

[iduce](#)

Naravno, pravilo znate: Za sve *kritike* i *pohvale*, kao i *pitanja* koje vjerovatno imate, ili ako uocite neku gresku, javite se direktno autoru na behar@bhwebmasters.net kada je C tutorijal u pitanju, ili na emir@jusic.de za C++.

Ovim se ogradjujem od bilo koje odgovornosti u vidu bilo cega ovdje, a specifino mislim na C/C++ programski jezik, pored ostalih stvari koje vam moze pasti na pamet. Sa ovim tutorijalom vam predstavljam programiranje u C jeziku na onaj nacin na koji ja vidim shodno. Ne tvrdim da je tutorijal sveobuhvatan, niti da je 100% tacan, ali vam izrazito tvrdim da ste sposobni savladati C programski jezik ukoliko mozete pratiti ovaj tutorijal, i ukoliko je to slucaj, onda vas urgiram da se toga sto prije prihvatite. Nigdje ni na koji nacin ne polazem bilo koje pravo na posao osobe koja se bavi obrazovnim radom, kao ni pravo na bilo koji trademark ovdje naveden.

Prosta C funkcija

[prethodno](#)

[iduce](#)

Tipicnu C Funkciju sacinja:

- [Vrsta funkcije](#) (Function Type)
- [Ime Funkcije](#)
- [Viticasta zagrada](#) (koja koja otvara 'tjelo' funkcije)
- [Tijelo funkcija](#)
- [Viticasta zagrada](#)(koja zatvara 'tijelo' funkcija)

Viticaste zagrade se ustvari smatraju dijelom tjela funkcije, niposto posebni elementi funkcije. Ovdje ih isticemo na ovaj nacin da bi smo naznacili kako se koriste u definiciji funkcije.

Vecinu C programa sacinja vise funkcija.

Programi pocinju sa **main()** funkcijom, koja poziva druge funkcije u programu. Ove funkcije, sa svoje strane, mogu pozivati druge funkcije.

Da bi ste pozvali funkcije, specificirate **njeno ime**, zajedno sa **listom argumenata**.

Kada program dodje do komande tog tipa, pozvat ce navedenu funkciju, i kontrola programa prelazi na pozvatu funkciju.

Sekvenca gdje kontrola programa prelazi iz funkcije u funkciju zovemo '**tok kontrole**' (*control flow*) programa.

Funkcija je samostalna jedinka coda koja ostvaruje specifcnu radnju.

Funkcija u C jeziku ostvaruje ulogu koju cine funkcije, sub-rutine i procedure u drugim programskim jezicima, i pored mogucnosti razlike u detaljima.

Pojedine funkcije koje cine specifine radnje su vec predefinisani, npr **printf()** se koristi da bi se ispisao tekst na ekran.

Funkcije koje cine slicne ceste radnje, kao npr. **printf()**, **scanf()**, **putchar()**, **getchar()**,... su pothranjene u *C Library* i pozivaju se po potrebi.

Cesto je potrebno definisati funkciju da obavlja radnju specficnu potrebama programa na kojem radite. Kada ta potreba iskrsne, na vama je da napravite C funkciju koja ce ispuniti taj zadatak.

Zasto su nam funkcije potrebne, cujem da pitate?

Znam, moguće je sav code potreban jednom C programu svrstati pod `main()` funkciju, i mirna Bosna

... ups! Nije bas tako.

U iducoj lekciji objasnjavamo zasto...

Naravno, pravilo znate: Za sve *kritike* i *pohvale*, kao i *pitanja* koje vjerovatno imate, ili ako uocite neku gresku, javite se direktno autoru na behar@bhwebmasters.net kada je C tutorijal u pitanju, ili na emir@jusic.de za C++.

Ovim se ogradjujem od bilo koje odgovornosti u vidu bilo cega ovdje, a specficno mislim na C/C++ programski jezik, pored ostalih stvari koje vam moze pasti na pamet. Sa ovim tutorijalom vam predstavljam programiranje u C jeziku na onaj nacin na koji ja vidim shodno. Ne tvrdim da je tutorijal sveobuhvatan, niti da je 100% tacan, ali vam izrazito tvrdim da ste sposobni savladati C programski jezik ukoliko mozete pratiti ovaj tutorijal, i ukoliko je to slucaj, onda vas urgiram da se toga sto prije prihvatite. Nigdje ni na koji nacin ne polazem bilo koje pravo na posao osobe koja se bavi obrazovnim radom, kao ni pravo na bilo koji trademark ovdje naveden.

Kompajler

OK. Napokon.

Naime, ono sto vi ne znate, je da sam se namucio dok sam uspio pronaci kompajler za C jezik koji ce odgovarati uslovima koje sam nekako sam sebi postavio za ovaj kurs.

Da vas upoznam sa uslovima:

Prvenstveno sam zahtijevao da je kompajler besplatan. Naime, veliki sam pobornik interneta i mogucnosti koje nam predstavlja, a komercijalizacija mi dize kosu na glavi, i u duhu ovog mog rada, koji je besplatan, bila bi glupost da vas sada tjeram da kupujete alatke sa kojim ce te raditi besplatan kurs. Taj uslov je najlakse ispuniti.

Drugo, trazio sam nesto sto se da lahko savladati, sto je fakticki nemoguće u C jeziku, ali eto, tu sam djelimicno uspio. Naime, na trzistu su razni kompajleri koji rade sve-i-sve, pa je sasvim moguće da kod sebe imate posljednje izdanje Borland Buildera, Cinjenica je da nam za ovaj kurs treba nesto jednostavno, ne zelim da se hvatam u kostac sa OS specificnim programiranjem, nas code treba da je prenosiv i OS nespecifican. U jednom od kasnijij tutorijala pozabavit cemo se builderom, i ako savladate ovaj tutorijal, garantujem vam da cete savladati bez problem i taj.

Druga tacka je nekako zahtijevala da za ovaj kurs pronadjemo kompajler sa IDE, ond. Integrated Development Enviromentom, sto ce reci da su sve alatke koje su potrebne za kompilaciju, linkovanje, debugging i slicno ugradjene u jednu aplikaciju. Sa tim izbjegavamo poteskoce koje se javljaju kod pocetnika sa kommand line kompajlerima.

Trece, da je prisutna standardna biblioteka C filova.

I sta smo pronasli?

Turbo C v.2.01, izdat 11 maja 1987.

Odlicno.

Ljepota ovog kompajlera je da ga Borland djeli besplatno, i da sasvim ljepo radi ono sto nama treba.

Dos aplikacija, tako da ce se pojedini osjecati kao kod kuce, dok ce drugima da se dize kosa na glavi, ali od ovoga smo svi poceli nekako, pa zato nije lose da se tim istim korjenima nekad i vratimo, zar ne?

Link je ovdje: [,](#)

i brza navigacija stranice ce vas dovesti do:

[,](#) i tu cete naci 1.05 .zip file, koji je ustvari nas kompiler. Ukoliko je cijela ta navigacija suvise teska, file mozete naci na nasem serveru [,](#) ali nek su vam te dvije gornje stranice u favorit folderu, ukoliko naletite na neki problem pri radu.

Update!: **Borland C++ Compiler version 5.5 Free Download Is Here !!!**

Borland dijeli sa svima nama svoju premijernu alatku, C++ Compiler V 5.5 besplatno, za personalnu upotrebu!!!

Potrebno je i dovoljno da se registrujete, i ukoliko vam vasa net konekcija to dozvoljava, skinite ovaj super compiler!!!

Link je ovdje:

Sretno:-)

U smislu instalacije ove aplikacije, potrebno je prvo nekim Zip programom unzipovati datoteku, sa tim da sacuvate direktorijsku strukturu fila. Za to vam moze poslužiti skoro svaka aplikacija koja se bavi arhivima, od PKZip do posljednjeg izdanja WinZip-a, ili bilo sto izmedju ta dva.

Ja licno koristim WinZip, a njega mozete naci besplatno .

Posto je ovo prastara aplikacija, i posto se u prastarim vremenima sve instaliralo sa floppy disketa, tako i ova instalacija zahtjeva da sadrzaj 3 direktorija koje stvori kopirate na floppy diskete, i da sljedite instalaciju sa prve diskete tako sto cete "INSTALL" .exe datoteku, sljedite upute, i u slucaju ukoliko ne zelite prihvatiti ponudjene opcije, izvorsite vas izbor u smislu instalacijskih direktorija.

Instalacija ce uobicajno da zavrsi u C:\TC direktoriji, ukoliko niste drugacije naznacili, tako da cete pokrenuti kompajler tako sto otkucate u dos promptu path i ime aplikacije. Ovako.

Recimo da radite u jednom od verzija Windows-a, idete "Start", "Programs", "Accessories" i "Command Prompt", i otvorit ce vam se nas stari znalac, dos prompt prozorac.

Uglavnom se otvara u C:/Windows direktorij, i da bi smo izašli iz toga, i usli u nas "Root", odn. C:\, otkucamo "cd ..", sada bi trebalo da pise C:\>

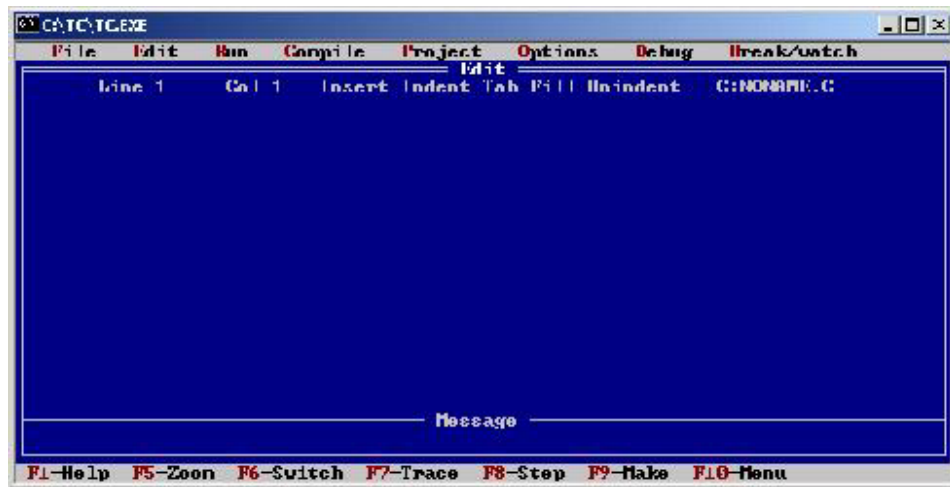
Potom, "cd tc", i nalazite se u Turbo C direktoriji vasesg compjuteru. Tada je dovoljno otkucati "tc", i turbo C je se pokrenuti. Dobrodosli:-))

Alternativno, moguće je navigati kroz windows do c:\tc direktorija i dvaput kliknuti na tc.exe datoteku, rezultat je isti. Prizor koji ce vas docekati je sljedeci:



OK, imamo makinju, cesta je prometna, vrijeme je da mu malo produvamo dizne, sta kazete da nesto pokusamo codirati? Moze!

Dok drzite "Alt" tipku, pritisnite "F", potom pustite te dvije i pritisnite "N", o dobit cete sljedece:



Sada cemo tu ukucati nesto coda, i potrebno je da ovo ukucate verbatim, odn. tacno kako ovdje pise. Idemo:

```
#include <stdio.h>

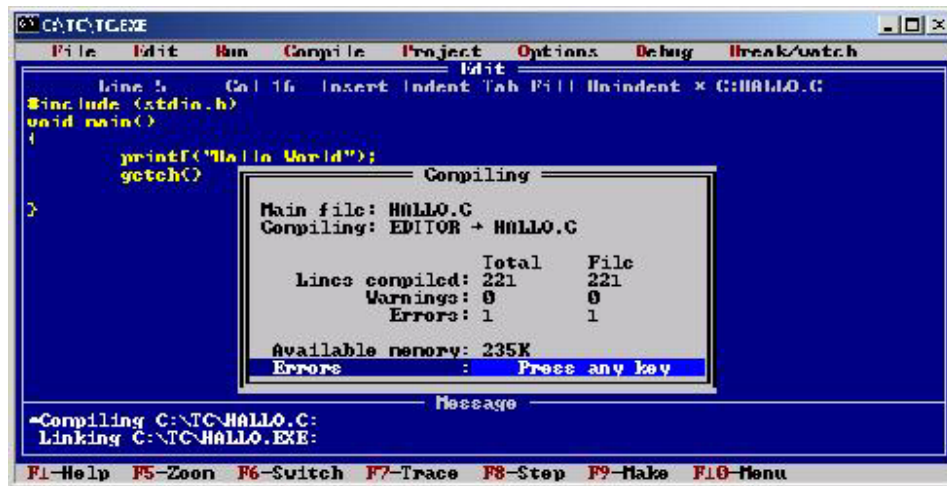
void main()

{
    printf("Hello World");

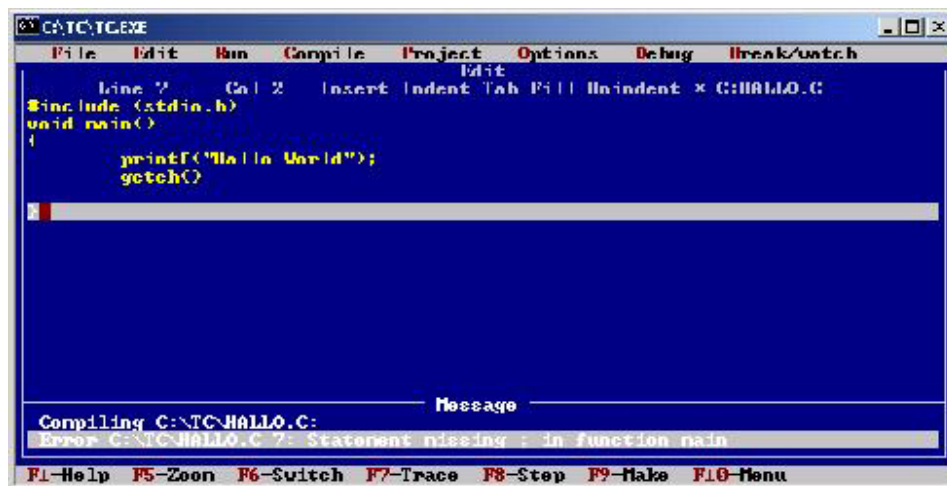
    getch();
}
```

Kada ste to ukucali, pritisnite F2, da bi ste pothranili vas ".c" file, nadjeni te mu neko ime, u ovom slucaju recimo "hello.c", i [ENTER].

Sada kada ste ga sacuvali, moguće je sa jim nesto uraditi, pritisnite "F9" tipku, i sa tim ste nacinili svoj prvi .exe file, u ovom slucaju hello.exe, koji se nalazi u C:\tc\ direktoriji. Ukoliko ste nacinili neku gresku, prikazat ce vam se ovo:



Primjetit cete kako pise "Error: Press any key" To cemo i uraditi:



Vidite u donjem redu kako pise objasnjenje greske? Jednostavo, u ovom slucaju sam izostavio znak ";" poslje rijeci "getch()", i ako to ispravimo, pritisnemo "F2", potom "F9", pothranit cemo ispravnu verziju i potom naciniti datoteku koja radi i koja ce prikazati na ekran sljedece:



Cestitam, ostvarili ste svoju prvu komunikaciju izmedju vas i kopjutera, posredstvom tastiture i kompajlera, i nacinili mali korak za covjeka, a jos manji korak za covjecanstvo, ili tako nekako. Vrijeme ce pokazati.

U svakom slucaju, zapoceli ste jedno predivno putovanje punu otkrica i znanja, i na tome vam cestitam od srca, i zelim vam da istrajete, jer su nagrade mnogobrojne i ceste, i ne govorim samo o personalnoj satisfakciji ovdje.

OK, codecuttersi, za danas je dosta, sutra cemo precu na nas prvi program i da malo razjasnimo sta to sve znaci i kako fercera, tako da cemo sutra precu dosta gradiva.

Vidimo se na idućoj stranici:-)

Naravno, pravilo znate: Za sve *kritike* i *pohvale*, kao i *pitanja* koje vjerovatno imate, ili ako uočite neku grešku, javite se direktno autoru na [Facebook](#) kada je C tutorijal u pitanju, ili na [Twitter](#) za C++.

Nas prvi program

Vazno je napomenuti da je C jezik "case-sensitive", odnosno da je osjetljiv na razliku između velikih i malih slova. Tako da varijable: suma, Suma i SUMA su tri različite varijable, a pozivati se na nepostojeću varijablu će u kompajleru izazvati error. Tako da morate obratiti pažnju na vještinu tipkanja pri kodiranju.

Primjer C programa

Za početak, unesite sljedeći kratki program:

```
#include <stdio.h>

/* Programski primjer broj 1 */

int main()
{
    int mjera;
    mjera = 180;
    printf("Ja sam visok %d centimetara\n", mjera);

    getch();

    return (0);
}
```

Kompajlirajte program, i kompajler će automatski izvršiti Linkovanje sa potrebnim Library funkcijama, i stvoriti .exe file. Kada pokrenete vaš .exe program, output programa će biti:



```
Ja sam visok 180 centimetara
```

Da pogledamo ovaj naš prvi program malo bliže.

Prva linija,

```
#include <stdio.h>
```

Naredjuje kompajleru da ukljuci stdio.h datoteku u kompilaciju.

Zasto je potrebno ukljuciti u kompilaciju stdio.h da bi program funkcionisao, e' o tome cemo malo kasnije, za sada je dovoljno znati da je ta datoteka potrebna, i da je jedna od standardnih "header" datoteka ukljucenih u standardnoj biblioteci tih datoteka koji osiguravaju uspjesan rad nasih programa, u ovom slucaju odredjuje I/O funkcije naseg programa. Zapamtite da je ponekad potrebno imati i vise od jednog .h file da bi program funkcionisao, ali o tom-po tom.

Druga linija:

```
/* Programski primjer broj 1 */
```

je komentar. I kao takav je ignorisan od strane kompajlera pri kompilaciji programa. Pocinje sa "/" i zavrшава sa "*". Bilo sto uneseno unutar tih znakova ne ulazi u program. Bitna poenta za zapamtiti je pri radu na programu da ce te poneka htjeti izbaciti pojedine djelove programa van kompilacije, da bi ste recimo pokusali pronaci "bug" u codu, i vjerovatno ce te to uraditi tako sto ce te postaviti dio coda pod komentaron (postoji i praviljniji nacin da bi ste to uradili, ali o tome kasnije), i ako se desi da imate kometar unutar komentara, prva instanca znaka */ na koji kompajler naidje ce biti uzeta kao zavrsetak komentara, tako da cete imati pomalo smeca u programu, i zasigurno nere raditi kako treba. Za sada to jos nije bitno, ali eto, da na to obratite paznju.

Primjetit ce te da u codu postoje prazne linije, na kojem nema nista.

Ukoliko ste se upitali dali je to potrebno, odgovor je da jeste, ali ne kompajleru, vec nama, ljudima. Naime, u C jeziku, osim u nekim posebnim instancama, tzv. "bijeli prostor", odn. prazna mjesta i linije ne cini nista, i ne ulazi u kompilaciju programa. Tako da bi ovaj nas program mogao da se napise na sljedeci nacin:

```
#include<stdio.h> /*Programski primjer broj 1 */int main(){int mjera;mjera=180;printf("Ja sam visok %d centimetara\n",mjera);getch();}
```

i jedino sto bi smo postigli je to da ne razumijemo sto smo napisali, a kamo li da neko poslje nas uspije to protumaciti i razraditi. Kompajleru je totalno svejedno, program koji ste napravili je isti kao u prijasnjem slucaju, niti je veci niti manji, niti je brzi niti sporiji.

Konvencija koja se bavi programiranjem nalaze odredjene stvari, a jedna od tih je i kako korisiti "bijeli prostor" da bi code bio citak i razumljiv, i toga cemo se pridrzavati ovdje.

Trecu liniju:

```
int main()
```

specificira ime funkcije.

Svi C programi pocinju izvršenje pozivajući se na "main()" funkciju. O funkcijama cemo kasnije, a za sada samo toliko da int ispred funkcije oznacava da funkcija vraca vrijednost po izvršenju, i to integral. Prazne zagrade oznacavaju da funkcija ne uzima vrijednost.

Na idućoj liniji je velika zagrada,

```
{
```

i ona odredjuje pocetak "main()" funkcije.

Logicno je takodjer da zadnja velika zagrada na kraju programa "}" odredjuje kraj main() funkcije, i samim tim kraj programa, ali to ste mogli i sami skuziti, jelde? Odlicno, derago mi je da razmisljate.

Prva linije coda main() funkcije je:

```
int mjera;
```

Ova linije deklarise varijabilu, koju smo nazvali "mjera", i govori kompajleru da je ta varijabila tipa integral. U C jeziku, sve varijabile moraju biti deklarise prije upotrebe. Sto znaci da ih morate imenovati kao i da im odredite tip. U ovom slucaju je varijabila "mjera", tipa int, sto znaci integral, sto ponovo, na nasem, znaci cijeli broj u rasponu od -32,768 do +32,767.

Primjecujete takodje da je deklaracija zavrshena znakom tacka-zarez ";". Sve deklaracije i komande zavrshavaju na taj nacin.

Na iducoj liniji je sljedece:

```
    mjera=180;
```

"assignment statement", odn. komanda kojom varijabili dajemo vrjednost integrala 180. Od sada, pa do kraja programa, ili dok ne promjenimo vrjednost varijabile 'mjera'tako da joj dodamo ili oduzmemo neku vrjednost, ona ce sadrzavati integralnu vrjednost od 180.

Iduca linija ,koja salje informacije na ekran je:

```
    printf("Ja sam visok %d centimetara\n",mjera);
```

Ova linija je vazna iz dva razloga. Prvo, sto je to primjer poziva funkcije, a drugi, sto ilustrira upotrebu standardne "output" funkcije "printf()".

Ovu liniju coda sacinjevaju dva dijela: ime funkcije [printf()] i dva argumenta funkcije ["JA sam visok %d centimetara\n"] i varijabila mjera.

C nema ugrađenih I/O (input/output) rutine, tako da to postizete na taj način što koristite funkcije iz standardne biblioteke funkcija C jezika.

U svakom slučaju, da bi ste pozvali funkciju (bila ona iz standardne biblioteke, ili neku koju ste i sami kodirali), dovoljno je joj napisati ime i da joj dostavite argumente koje su joj potrebne (argumenti su vrijednosti koje šaljete funkciji kada je pozvana i smjestaju se u zagradu funkcije).

`printf()` funkcija funkcionise na sljedeći način (funkcija-funkcionise, hehehe):

prvi argument koji joj dostavite je `STRING` koji može sadržavati obični znakovi (slova, brojevi i simboli) ili kodovi za formatiranje (format codes), koji počinju sa znakom `"%"`. Obični znakovi se šalju na ekran u onom redoslijedu u kojem su napisani u kodu programa, odn. u `printf()` funkciji, a format code informise `printf()` da prikaže varijabilu. U ovom slučaju `%d` informise `printf()` da prikaže integral u decimalnom formatu. Vrijednost koja će se prikazati je data u drugom argumentu funkcije, a to je u ovom slučaju naša varijabla `"mjera"`. Code `"\n"` je specijalna sekvenca koja informise `printf()` da izda komandu "nova linija" (odn. carriage-return-linefeed ili `newline`).

Da bi ste razumjeli odnos običnih znakova i format coda, promijenite pomenutu funkciju na sljedeći način:

```
printf("Ja sam %d centimetara visok\n",mjera);
```

i pokrenite program. Poruku koju ćete dobiti je:



```
Ja sam 180 centimetara visok
```

Cool, zar ne?

Linija:

```
getch();
```

je za sada malo izvan obima gradiva koje smo do sada prosli, ali je potreban za pojedine operativne sisteme, i volio bi kada bi ste malo experimentisali sa tim, odn. kako bi bilo bez toga. Da vam kazem odmah da na mom stroju, dos otvori prozor i zatvori ga u tren, tako da nemam vremena vidjeti sta pise. E, za takve slucajeve smo postavili jednu standarnu C funkciju u code, i "getch()" funkcija ceka da "pokupi" sljedecu tipku koju pritisnete na tastituri. Naravno, sa tom vrjednoscu necemo uraditi u ovom slucaju nista, ali nama dobro sluzi da bi program cekao dok smo spremi da zavrismo sa njim prije nego sto dozvoli OS-u da zatvori dos prozor.

Iduca linija

```
return (0);
```

nalaze programu da vrati vrjednost OS-u po izvršenju. Vrlo cest cemo se susretati u programu da nam funkcija vraca vrjednost glavnom programu poslje neke kalkulacije, i ovo je jedan od nacina da bi se to ostvarilo.

Zadnju liniju programa smo vec objasnili, a nju sacinjava sljedece:

```
}
```

i to oznacava kraj main() funkcije, sto opet znaci da je izvršenje programa završeno.

Uh, za danas smo presli podosta materijala, tako da cemo ovdje otpoceti malo, i nastaviti sutra sa nekim osnovnim greskama i nacin na kojem ih otkrivamo, to jeste ukoliko niste vec uspjeli unijeti greske u codu, i sami ih uspjeli otkloniti, a u tom slucaju vas trebam prvo ukoriti na nedostatak paznje, a potom pohvaliti na istrazivanju greski i nacin rada kompajlera. U svako slucaju navratite sutra, da vidimo dali smo dosli do istih rjesenja.

Kolege programeri (kako to fino zvuci, jelde), do Kodiranja...

Naravno, pravilo znate: Za sve *kritike* i *pohvale*, kao i *pitanja* koje vjerovatno imate, ili ako uocite neku gresku, javite se direktno autoru na [forum](#) kada je C tutorijal u pitanju, ili na [Facebook](#) za C++.

Ovim se ogradjujem od bilo koje odgovornosti u vidu bilo cega ovdje, a specifcno mislim na C/C++ programski jezik, pored ostalih stvari koje vam moze pasti na pamet. Sa ovim tutorijalom vam predstavljam programiranje u C jeziku na onaj nacin na koji ja vidim shodno. Ne tvrdim da je tutorijal sveobuhvatan, niti da je 100% tacan, ali vam izrazito tvrdim da ste sposobni savladati C programski jezik ukoliko mozete pratiti ovaj tutorijal, i ukoliko je to slucaj, onda vas urgiram da se toga sto prije prihvatite. Nigdje ni na koji nacin ne polazem bilo koje pravo na posao osobe koja se bavi obrazovnim radom, kao ni pravo na bilo koji trademark ovdje naveden.

Osnovne greske pri kodiranju

Nije vam bilo dosta mucenja juce, nego ste se vratili po jos?

OK, ova moja mala mazohistika grupa studenata pokazuje znakove upornosti, cestitam, trebat ce vam:-)

Da se malo vratimo code koji smo jucer napravili:

```
#include <stdio.h>

/* Programski primjer broj 1 */

int main()
{
    int mjera;

    mjera = 180;

    printf("Ja sam visok %d centimetara\n", mjera);

    getch();

    return (0);
}
```

Danas cemo malo sa njom da experimentisemo.

U sljedecem kodu cemo ubaciti sve ono sto je zeleno, a ono sto je crveno cemo izbaciti, plavom bojom cemo oznaciti promjene. Ovo radim je je preglednije na ovaj nacin:

```
#include <stdi.o.h>
```

```

/* Programski primjer sa greskama */

int main()
{
    int mjera;

    int tezina;

    mjera = 180.25;

    printf("Ja sam visok %d centimetara\n", mjera);

    Getch();

    return (0);
}

```

... i kao rezultat cemo dobiti sljedeci code:

```

#include <stdi.h>

/* Programski primjer sa greskama */

int main()
{
    int mjera

    int tezina;

    mjera = 180.25;

    printf("Ja sam visok %d centimetara\n", mjera);

    Getch();

    return (0);
}

```

..., i htio bi da sada prepisete ovaj code u kompajler, i da vidimo sta cemo dobiti kada pokusamo da kompajlujemo nas falicni program. Dakle, idemo prvo sa ukucanim programom:

```

TC
File Edit Run Compile Project Options Debug Break/watch
line 14 Col 2 Indent Tab Fill Unindent x C:\PBI\NKO.C
#include <stdi.h>

/* Programski primjer sa greskama */

int main()
{
    int mjera
    int tezina;
    mjera = 180.25;

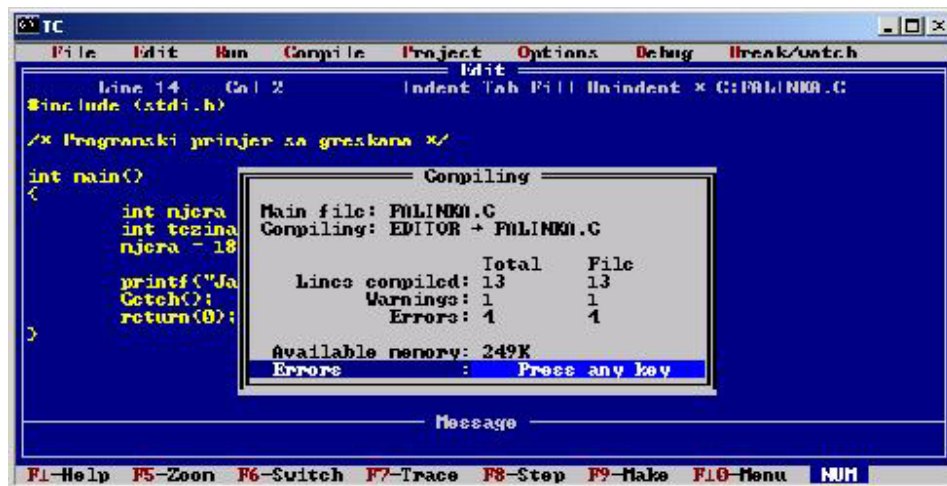
    printf("Ja sam visok %d centimetara\n", mjera);
    Getch();
    return(0);
}

Message

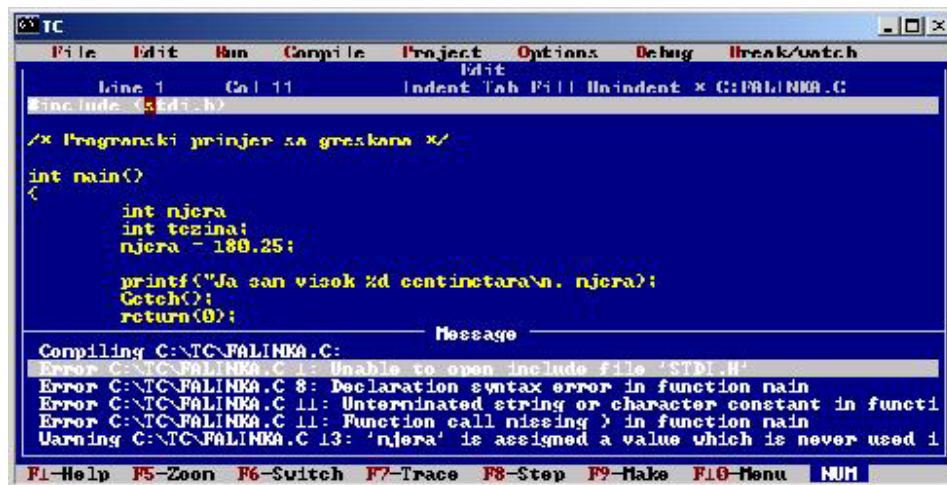
F1-Help F5-Zoom F6-Switch F7-Trace F8-Step F9-Make F10-Menu NUM

```

Kada pokusamo kopilovati datoteku, dobijemo sljedeću poruku:



Bitno je ovdje zapaziti poruku "Error". Pritisnimo neku tipku, da vidimo gdje smo to pogriješili:



Pod naslovom Poruke (Message) imamo sljedeće redove:

- Compiling C:\TC\FALINKA.C
- * Error C:\TC\FALINKA.C 1: Unable to open include file 'STD.LH'
- Error C:\TC\FALINKA.C 8: Declaration syntax error in function main
- Error C:\TC\FALINKA.C 11: Unterminated string or character constant in printf
- Error C:\TC\FALINKA.C 11: Function call missing) in function main
- Warning C:\TC\FALINKA.C 13: 'mjera' is assigned a value which is never used in function main

Bitno je zapaziti kako su naznacene linije u kodu i u listi gresaka koje je kompajler pokusao razvrstati, i one su povezane, dakle, naznacena greska se odnosi na naznaceni red, ili barem da je u tom redu compiler skontao da nesto nije uredi.

Medju greskama cete zapaziti da postoje greske (error) i opomene (warning). Razlika u njima je ta sto ce kompajler izvršiti kompilaciju programa sa opomenama, dok to nece uciniti sa greskama.

Opcenito, opomene tretirajte kao greske, i odstranit cete mnogo toga nepotrebnog ili opasnog iz code.

Idemo redom da popravljamo greske.

U prvoj poruci pise da je greska u liniji coda broj 1, ne moze otvoriti datoteku 'STDI.H', i stvarno, u codu je naznacena prva linija, sa tim da je crvena kockica na prvo slovo naziva doticne datoteke, koja stvarno ne postoji standardnoj biblioteci header datoteka. To cemo naravno ispraviti, idemo dalje, pritisnite tastituru strelice koja pokazuje na dole.

Aha! Sta je ovo? Pise da je greska na liniji 8, a mi znamo da smo nacinili gresku na liniji 7?

Razlog je taj sto je kompiler tek na sljedecoj liniji skontao da nesto nije uredi, naime poslje deklaracije integrala, pokusao je skontati ime koje smo dodjelili toj varijabili, mjera, nesto, int,... aha, to ga je zbunilo, posto iza deklaracije integrala ocekjuje jednu od sljedecih karaktera: tacka-zarez, znak jednakosti ili zarez, a nasao je prazno mjesto, na tom mjestu je proglasio da je greska. OK, ispravit cemo nasu deklaraciju varijabile mjera. Idemo dalje.

Linija 11: string ili karakterna konstanta bez terminatora. Kao sto znamo unutar printf() funkcije, stringove zapocinjemo i završavamo sa znacima navodnika. OK, i to cemo ispraviti, ali sta ovo pise na liniji gresaka ispod, da u istom redu imamo jos jednu gresku?

Funkciji nedostaje ')', a jasno vidimo da ga ima, sta je ovo?

Jednostavno, posto nas string nema zavrsetka, kompajler je pretpostavio da je character ')' dio tog stringa, i nije mogao da ga uzme kao zgradu kojom zaokružuje funkciju printf().

Znaci ispravljanjem jedne greske rjesit cemo se dva problema, fino:-)

Iduce nije greska, vec upozorenje, da smo deklatisali varijabilu koju nigdje ne koristimo, i posto zelimo pisati dobar code, rjesit cemo se i toga, dakle brisi!

Da vidimo sta smo postigli sa ovim izmjenama, nacinite ih, i ponovo kompajlujte program.

Opla!, sta je ovo? Rjesili smo sve sto je kompajler javio, ali opet imamo gresaka?

Jednostavno, jedna greska zna prikriti drugu, isto kao sto se od jedne greske moglo pojaviti dvije inastance. Kada smo rjesili gresku na liniji 11, kompajleru je postalo jasno da je Getch() odvojena funkcija, a ne dio stringa namjenjenoj printf() funkciji.

Sjecate se na pocetku kada sam rekao da C jezik razaznaje izmedju velika i mala slova? E pa Getch() nije standarna funkcija, getch() jeste.

OK, i to cemo popraviti. F9, i Uspjeh!

Hajdemo sada da pokrenemo nas program, da vidimo sta ce pokazati.

Hmmmmmm, opet nesto nije uredi. naime, varijabili mjera smo dodjelili vrjednost od 180.25, a ovdje pokazuje samo 180. Sta je ovo?

Hajde da promjenimo formate code u printf() funkciji, pa da, varijabila je formatirana kao intergal, a potreban nam je float, posto je decimalni broj... , umjesto %d stavit cemo %f.....

Opa, sta uradih, hoce kompilovati, ali na mom stroju nece da se prikaze output uopste.....

Ovim sam samo htio da oslikam kako je lahko zaglaviti se u osnovnim problemima u C jeziku. Naime, varijabilu mjera smo deklarirali kao int, da bi nas program funkcionirao, moramo ga prikazati kao int, ili ga deklarirati kao float, pa hajmo uraditi jedno od to dvoje...

I kada mjera deklariramo kao int, i pored toga sto mu pokusavamo dodjeliti decimalni broj, decimale se odbacuju u deklaraciji, tako da snjima vise nemamo problema, ali posto smo dobri programeri, odstranit cemo i to nepotrebno i zbunjujuce u codu.

I poslje svih tih popravaka naseg coda, na kraju smo dobili ono sto je i za ocekivati, na ekranu pise:



Ja sam visok 180 centimetara

: -)

Nije lahko, jelde?

Ne mozete reci da vas nisam opominjao.

Ovo je bilo samo radi ilustracije da se greske mogu naciniti na vise nivoa: u codu, greske pri sintaxi, logicne greske (koje je mnogo teze otkriti), tipkarske greske, greske u formatiranje, itd., itd...

Ponekad ce vam greska biti toliko ocigledna da je necete uopste moci vidjeti, dok vam zena ne uhoda u kancelariju, i ne zapita: "...dragi, sta ce ti ovo ovdje?..."

Sto samo po sebi ne bi bilo lose, da niste upoznati sa cinjenicom da mi zena ne razaznaje programski code od karburatora za ford....

Veselje.

OK, dragi moji i drage moje, dosta mi je za danas, navratite opet i sutra, uiobicalili smo se ovako sastati, jelde?

Naravno, pravilo znate: Za sve *kritike* i *pohvale*, kao i *pitanja* koje vjerovatno imate, ili ako uocite neku gresku, javite se direktno autoru na [ovaj link](#) kada je C tutorijal u pitanju, ili na [ovaj link](#) za C++.

Ovim se ogradjujem od bilo koje odgovornosti u vidu bilo cega ovdje, a specifcno mislim na C/C++ programski jezik, pored ostalih stvari koje vam moze pasti na pamet. Sa ovim tutorijalom vam predstavljam programiranje u C jeziku na onaj nacin na koji ja vidim shodno. Ne tvrdim da je tutorijal sveobuhvatan, niti da je 100% tacan, ali vam izrazito tvrdim da ste sposobni savladati C programski jezik ukoliko mozete pratiti ovaj tutorijal, i ukoliko

je to slučaj, onda vas urgiram da se toga sto prije prihvatite. Nigdje ni na koji način ne polazem bilo koje pravo na posao osobe koja se bavi obrazovnim radom, kao ni pravo na bilo koji trademark ovdje naveden.

O deklaraciji varijabila

Sve varijabile se deklarisu na isti način, jednostavno naznacite vrstu varijabile koju želite stvoriti, zatim navedite imena varijabila, i to sve završite ";".

```
char tipka;  
int duzina, sirina, površina;  
long razdaljina;  
float prosjek;  
double masa;
```

Kao što vidite, moguće je deklarirati više varijabila iste vrste u jednoj liniji, dovoljno ih je samo razdvojiti zarezima.

Inicijalizacija varijabile nastupa onda kada joj date neku vrijednost. A to možemo učiniti na dva načina:

```
int duzina;  
  
duzina = 33;
```

ili na sljedeći način:

```
int duzina = 33;
```

Do vas je koju konvenciju kada upotrebljavate, samo budite oprezni da vrsta varijabile moze ponjeti vrjednost.

Naravno, pravilo znate: Za sve *kritike* i *pohvale*, kao i *pitanja* koje vjerovatno imate, ili ako uocite neku gresku, javite se direktno autoru na [ovaj link](#) kada je C tutorijal u pitanju, ili na [ovaj link](#) za C++.

Ovim se ogradjujem od bilo koje odgovornosti u vidu bilo cega ovdje, a specificno mislim na C/C++ programski jezik, pored ostalih stvari koje vam moze pasti na pamet. Sa ovim tutorijalom vam predstavljam programiranje u C jeziku na onaj nacin na koji ja vidim shodno. Ne tvrdim da je tutorijal sveobuhvatan, niti da je 100% tacan, ali vam izrazito tvrdim da ste sposobni savladati C programski jezik ukoliko mozete pratiti ovaj tutorijal, i ukoliko je to slucaj, onda vas urgiram da se toga sto prije prihvatite. Nigdje ni na koji nacin ne polazem bilo koje pravo na posao osobe koja se bavi obrazovnim radom, kao ni pravo na bilo koji trademark ovdje naveden.

String

String je, doslovno prevedeno, niz. I nista bolje ne objasnjava string do niza karaktera, tj. slova i brojki i specijalnih karaktera.

Objasnjenje: string je char data tip. Sto ce reci, ako u string pothranite recimo: 1, 3 i 7, u stringu se nalazi niz 1,3,7.

Dakle u spomenutom stringu nisu niti vrjednost 137, niti vrjednost 11, niti bilo koja druga vrjednost. Jednostavno su tamo ascii karakteri koji predstavljaju 1, 3 i 7. Na spomenutom stringu ne mozete vrsiti neke racunske radnje (ukoliko ih prije toga niste pretvorili u varijabilu druge vrste), osim usporedbe, ili ukoliko te radnje nisu na nivou ascii codova, ali tada vise ne govorimo o vrjednostima koje brojevi 1, 3 i 7 nama imaju. Ali, o ascii codovima kasnije.

Ali, pazite na najgoru stvar u svemu ovome, Turbo C nema string data tip!

Ostali kompajelri mozda imaju, ali Turbo C nema.

I sta sada?

Na scenu stupaju matrice (kasnije pointeri, ali...), i sa njima imamo defacto string tip varijabile.

Znate sta su matrice? Sljedi graficki prikaz:

	0	1	2	3
Red1				
Red2				
Red3				
Red4				

U bjelom je prikazana dvodimenzionalna matrica sa 4 reda i 4 kolone. Array (matrica) u C jeziku je lista varijabila istog tipa. Array može imati više dimenzija, ali ako predjemo preko 3-4 dimenzije, postaje jako komplikovano i intenzivno što se tiče memorije koju program koristi. Za ovaj čas govorit ćemo o jednodimenzionalnim matricama, koje ćemo od sada pa do kraja zvati pravim imenom, Array.

Stvorit ćete jednodimenzionalni array tako što navedete vrstu varijabile, recimo char, zatim ime koju dajemo stringu, recimo "ime", potom u srednjim zagradama odredimo veličinu arraya. Ovako:

```
char ime[80];
```

i sa ovim smo deklarirali array od 80 elemenata tipa char koju smo nazvali ime.

Da bi smo prozvali specifičan element iz niza, dovoljno je postaviti odgovarajući index elementa u srednju zagradu (imajući na umu da su arrays indexirani od nule, ne od jedan) poslije ime arraya.

U našem arrayu ime[0] je prvi element, ime[1] drugi, i ime[79] je 80ti i zadnji element u nizu.

Najbitnija stvar koju morate zapamtiti u C jeziku što se tiče arrays, je da nema provjeravanja granica arraya. Što faktički znači da je moguće da predjete granicu arraya u svom programu i da opozivate memorijsku adresu u kojem je pothranjeno nešto drugo od onoga što očekujete. Najobičnija greška u C jeziku je recimo da napisete

```
ch = ime[80]
```

i kako je naš array deklarisan malo iznad ovoga, u char varijabili bi smjestili ko zna šta, jer naš array ide samo do ime[79], u idućem bajtu koda je možda neka instrukcija, ili početni dio neke druge varijabile, u svakom slučaju nešto neznano i nepoželjno.

Svi nizovi (strings) završavaju sa NULL karakterom '\0'. Jedino na taj način možete kao programer da provjerite kada ste dosli do kraja stringa, ali samim tim to znači da vaš string mora biti dovoljno velik da primi sve što mislite u njega staviti, kao i NULL terminator. Primjerice, želite pothraniti string "zdravo". Graficki prikaz sljede:

z	d	r	a	v	o	\0
---	---	---	---	---	---	----

Mada nam samo treba 6 slova, moramo naćiniti array od 7 mjesta da bi naša poruka stala komotno.

Znam, želite da kodirate, dosta vam teorije. Evo prilike.

```

#include <stdio.h>

#include <string.h>

/* Primjer stringa */

main()
{
    char ime[80];

    printf("Unesite vase ime: ");

    gets(ime);

    printf("\nPozdrav %s, sta ima", ime);

    getch();
}

```

Primjetit ce te %s komandu, koja naredjuje printf() funkciji da na mjesto komande formatira string varijabilu. Kako polako napredujemo kroz tutor C jezika, upoznat cete jos tih i takvih komandi.

Strings, i rad sa njima zauzima veliki dio C jezika, i postoji veliki broj standardnih funkcija koje se koriste pri obradi i manipulaciji stringeva. A dva koja se najcesce koriste su:

```
strlen();
```

(skraceno za "string length" - duzina stringa), koja vraca, kao sto ime kaze, duzinu stringa, i

```
strcpy();
```

(skraceno za "string copy" - ne moram vam prevoditi), koja preslikava sadrzaj stringa iz jednog u drugi string.

Sintaksa za navedene funkcije sljedi:

```
strlen(string);

strcpy(odrediste, izvor);
```

Postoji jedan vrlo dobar razlog zbog cega se ove dvije funkcije najvise koriste, i zasto ce te ih najcesce naci da se zajedno upotrebljavaju. Naime, ukoliko preslikavate string u varijabilu koja svojom velicinom ne moze primiti cjeli string, dobice te nepredvidljive rezultate. O tome cemo opsirnije kada budemo pokrivali if-else uslove, jer nam je i to potrebno za tacno kopiranje stringova.

U svakom slucaju, sada cemo demonstrirati upotrebu navedenih funkcija. Kada pokrecete ovaj program, budite sigurni da unesete ime sa manje od 10 slova.

```
#include <stdio.h>

#include <string.h>

/* Primjer stringa II */

main()
{
    char str[10];

    char novistr[12];

    printf("Unesite vase ime: ");

    gets(str);

    printf("\nPozdrav %s.\n", ime);

    printf("U vasem imenu su %d slova.\n", strlen(str));

    strcpy( novistr, str);

    printf("novistr je sada: ", novistr);

    getch();
}
```

Ako se pitate dali je moguće inicializirati string varijabile na sljedeći način:

```
str = "hello"
```

odgovor je ne. Za sada morate koristiti strcpy() funkciju, a kasnije cete nauciti kako je to moguće izvesti. Znam, ali sam vam rekao na pocetku da morate biti strpljivi.

Jesmo li dosta za danas?

I ja mislim.

Hajde onda, pa cemo sutra malo razgovarati o konceptima stringa i konceptima funkcija. Drama!

Naravno, pravilo znate: Za sve *kritike* i *pohvale*, kao i *pitanja* koje vjerovatno imate, ili ako uocite neku gresku, javite se direktno autoru na [GitHub](#) kada je C tutorijal u pitanju, ili na [Facebook](#) za C++.

Ovim se ogradjujem od bilo koje odgovornosti u vidu bilo cega ovdje, a specificno mislim na C/C++ programski jezik, pored ostalih stvari koje vam moze pasti na pamet. Sa ovim tutorijalom vam predstavljam programiranje u C jeziku na onaj nacin na koji ja vidim shodno. Ne tvrdim da je tutorijal sveobuhvatan, niti da je 100% tacan, ali vam izrazito tvrdim da ste sposobni savladati C programski jezik ukoliko mozete pratiti ovaj tutorijal, i ukoliko je to slucaj, onda vas urgiram da se toga sto prije prihvatite. Nigdje ni na koji nacin ne polazem bilo koje pravo na posao osobe koja se bavi obrazovnim radom, kao ni pravo na bilo koji trademark ovdje naveden.

Funkcije za obradu stringova

Strings, i rad sa njima zauzima veliki dio C jezika, i postoji veliki broj standardnih funkcija koje se koriste pri obradi i manipulaciji stringeva. A dva koja se najcesce koriste su:

```
strlen();
```

(skraceno za "string length" - duzina stringa), koja vraca, kao sto ime kaze, duzinu stringa, i

```
strcpy();
```

(skraceno za "string copy" - ne moram vam prevoditi), koja preslikava sadrzaj stringa iz jednog u drugi string.

Sintaksa za navedene funkcije sljedi:

```
strlen(string);

strcpy(odrediste, izvor);
```

Postoji jedan vrlo dobar razlog zbog cega se ove dvije funkcije najvise koriste, i zasto ce te ih najcesce naci da se zajedno upotrebljavaju. Naime, ukoliko preslikavate string u varijabilu koja svojom velicinom ne moze primiti cjeli string, dobice te nepredvidljive rezultate. O tome cemo opsirnije kada budemo pokrivali if-else uslove, jer nam je i to potrebno za tacno kopiranje stringova.

U svakom slucaju, sada cemo demonstrirati upotrebu navedenih funkcija. Kada pokrecete ovaj program, budite sigurni da unesete ime sa manje od 10 slova.

```
#include <stdio.h>

#include <string.h>

/* Primjer stringa II */

main()
{
    char str[10];

    char novistr[12];

    printf("Unesite vase ime: ");

    gets(str);

    printf("\nPozdrav %s.\n", ime);

    printf("U vasem imenu su %d slova.\n", strlen(str));

    strcpy( novistr, str);

    printf("novistr je sada: ", novistr);

    getch();
}
```

Ako se pitate dali je moguće inicializirati string varijabile na sljedeći način:

```
str = "hello"
```

odgovor je ne. Za sada morate koristiti strcpy() funkciju, a kasnije cete nauciti kako je to moguće izvesti. Znam, ali sam vam rekao na pocetku da morate biti strpljivi.

Naravno, pravilo znate: Za sve *kritike* i *pohvale*, kao i *pitanja* koje vjerovatno imate, ili ako uocite neku gresku, javite se direktno autoru na [GitHub](#) kada je C tutorijal u pitanju, ili na [Facebook](#) za C++.

Ovim se ogradjujem od bilo koje odgovornosti u vidu bilo cega ovdje, a specifino mislim na C/C++ programski jezik, pored ostalih stvari koje vam moze pasti na pamet. Sa ovim tutorijalom vam predstavljam programiranje u C jeziku na onaj nacin na koji ja vidim shodno. Ne tvrdim da je tutorijal sveobuhvatan, niti da je 100% tacan, ali vam izrazito tvrdim da ste sposobni savladati C programski jezik ukoliko mozete pratiti ovaj tutorijal, i ukoliko je to slucaj, onda vas urgiram da se toga sto prije prihvatite. Nigdje ni na koji nacin ne polazem bilo koje pravo na posao osobe koja se bavi obrazovnim radom, kao ni pravo na bilo koji trademark ovdje naveden.

Koncepti stringa i funkcija

C jezik dozvoljava upotrebu stringa kao konstantu kao i varijabilu. Kada se upotrebljavaju kao konstante, mogu se unijeti direktno u coda programa kao dio printf() funkcije, naprimjer:

```
printf("Danas je stvarno predivan dan");
```

Ukoliko string koristite na vise mjesta kroz program, moguće ga je definisati kao konstantu, simbolicno, i opozvat kroz koristenje imena koje mu dodjelimo.

Ali, kada su stringovi karaktera varijabile, onda se sa njima ophodimo drugacije.

Da bi smo string kao varijabilu, potrebno je koristiti array (matricu).

Array je, ako sto znamo, sljed podataka istog tipa.

U slucaju stringa karaktera, array je skup karaktera u nizu.

Da bi smo deklarisali string karaktera kao varijabilu, koristili bi format arraya, kao na primjer:

```
char ime[x];
```

gdje je 'char' vrsta varijabile koju koristimo (character).

Ime je ovdje obican naziv koji smo dodjelili varijabili, a mogli smo tu staviti bilo koji naziv koji se slaze sa convencijom imenovanja varijabila i funkcija.

Kockaste zagrade odredjuju da je nasa varijabila array, i brojka unutar tih zagrada odredjuje koliko elemenata nas array ima.

Na primjer, da smo prije proglasili da je constanta $x = 25$, nasa varijabila bi mogla u ovom slucaju u sebi pothraniti 25 karaktera, sa tim da bi zadnji karater morao biti specijalni NULL karakter, odn. '\0'.

Broj elemenata koje specificiramo u nasem arrayu mora biti vecoj ili jednakoj totalnom broju karaktera koje namjeravamo pothraniti u to arrayu, plus 1.

Ako znaci namjeravamo u nekoj varijabili pothraniti imena gradova, i znamo da necemo unositi imena gradova koja sadrze preko 8 slova u svom imenu, pravilo u gornjem paragrafu ukazuje da deklariseмо array karaktera sa najmanje 9 elemenata, jer na zadnjem mjestu u arrayu moramo pothraniti NULL karakter, odnosno '\0'. Uradimo to.

Graficki:

Index arraya	0	1	2	3	4	5	6	7	8
ime	S	A	R	A	J	E	V	O	\0

Niste zaboravili da array indexi pocinju od nule? Odlicno, samo provjeravam:-))

Kada bi ovaj nas array stavili kroz strlen() funkciju o kojoj smo govorili prije nekoliko lekcija, zapazili bi da ona ustvari vraca 8 bajta, ali Deni, do sada si nam stalno govorio da nas array mora imati jedno vise mjesto od toga, tako da bi trebala vratiti vrjednost od 9 bajta?

Ali, strlen() funkcija broji samo stvarne karaktere u arrayu, odn. sadrzaj arraya, NULL karakter je kao granicnik, i u tom smislu nije uracunat kroz koristenje strlen() funkcije. Znaci toj vrjednosti morate sami dodati 1 da bi ste dobili stvarnu duzinu arraya. Vrlo bitno za kasnije koristenje malloc() i calloc() funkcija, ali o tom - po tom.

OK, toliko za danas. Sutra govorimo o fprintf() funkciji, znaci da cemo biti zauzeti.

Do tada.....

Naravno, pravilo znate: Za sve *kritike* i *pohvale*, kao i *pitanja* koje vjerovatno imate, ili ako uocite neku gresku, javite se direktno autoru na [ovaj link](#) kada je C tutorijal u pitanju, ili na [ovaj link](#) za C++.

fprintf() funkcija

Printf() funkcija je jedna od I/O (input/output) funkcija koja vam omogucava da saljete informacije ka perefernim aparatima prikopcanim na kompjuter (obicno ekran).

Printf() funkcija se sastoji iz 4 djela:

1: Ime funkcije, u ovom slucaju 'printf'.

2: Kontrol String, navodni znaci i sve sto je okruzeno njima. U kontrol stringu mozemo ubaciti literarne karaktere koje ce se prikazati na onaj nacin na koji ih stavimo u string, kao i specijalni codovi koji ukazuju gdje i kako da se prikazu konstante i varijabile koje smo odredili na drugim mjestima u nasem codu.

3: Lista varijabila i konstanti, odn. imena vrjednosti koje zelimo poslati printf() funkciji, i za koje smo vec postavili format codove u kontrol stringu. Vrjednosti su odvojeni od control stringa i od jednih drugih zarazima, i moraju da se povinuju sljedecim specifikacijama:

-Ukoliko je vrjednost varijabila, mora biti deklarisana prije upotrebe negdje u programu. Kada koristite varijabilu kao vrjednost, jednostavno navedite ime te varijabile

-Ukoliko je vrjednost konstanta, mogucnosti su da je ta konstanta brojka, karakter, string karaktera ili vec definisana simbolicka konstanta. MOrate se pridrzavati sljedecih formata kada ukljucujete konstante:

* Ukoliko je konstanta broj, moguće je jednostavno unijeti taj broj.

* Ukoliko je konstanta karakter, unesite je izmedju navodne znake, ovako -> 'X'

* Ukoliko je konstanta string, unesi te je izmedju duplih navodnih znakova, kao npr.: ->"Vase ime"

* Ukoliko je konstanta simbolicka konstanta, jednostavno navedite ime konstante kao sto ste je definisali na pocetku coda sa #define direktivom (o tome detaljnije kasnije).

4: Zatvorite zagradu i tacka-zarez, i to završava komandu.

Djelovi printf() komande:

printf("Od izvora %d putica %s\n", x, "vode");			
1	2	3	4
printf(	"Od izvora %d putica %s\n",	x, "vode"	);

i njegov output:

```
Od izvora 2 putica vode
```

podrazumjevajuci da smo prije toga varijabili pripisali vrjednost od dva.

U ovom slucaju smo koristili dva coda, %d i %s. %d ukazuje da ce mjesto tog coda biti prikazan integral, a %s karakter string ("vode").

Rjedosljed codova uvijek mora pratiti redosljed varijabila, tako da se prvi code uvijek odnosi na prvu varijabilu, drugi na drugi, itd.....

Budite oprezni da uvijek imate isti broj codova koliko i varijabila koje prikazujete!

Format codovi su takodje poznati pod nazivom "Specifikacije za konverziju", jer oni naredjuju kako da se data prikaze. Primjer:

```
printf("%d %o %x", 16, 16, 16);
```

Naredba u ovom slucaju trazi da brojku 16 prikaze na 3 razlicita nacina: u decimalnoj notaciji, kao octal, i kao hexsdecimala.

Kao sto znamo, decimalni brojevi imaju za bazu 10, octalni brojevi imaju bazu od 8, a hexadecimalni imaju bazu 16 (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F) .

Kao rezultat nasih "Specifikacija za konverziju" nase date (u ovom slucaju broja 16), na ekran cemo dobiti sljedece:

```
16 20 10
```

jer je broj 16 u decimalnom sistemu 16,

broj 16 je u octalnom sistemu 20,

i broj 16 u hexadecimalnom sistemu je 10.

fprintf() je jedna od standardnih funkcija C jezika, i radi skoro identicno na nacina na koji printf() radi, ali umjesto na ekran, output moze slati na naznaceni aparat ili file, otud i naziv fprintf().

Jedna od tih aparata na koje mozete slati output je i stampac.

C jezik raspolaze sa standardnim nazivom za stampac: stdprn.

Do sada ste vjerovatno usli u naviku da u svim vasim programima koji koriste I/O funkcije kao sto su printf(), scanf() i fprintf() uvrstite #include direktivom i stdio.h header file:

```
#include <stdio.h>
```

u suprotnom necete biti u mogucnosti da usmjerite vas output ka stampacu.

Jedina razlika u sintaksi izmedju fprintf() i printf() funkcija je ta sto fprintf() koristi naknadni parametar: odrediste outputa!

Svi primjeri kojim cemo se mi sluziti koristiti ce se stdprn (stampacem) kao odredistem.

Sa fprintf() funkcijom, kada je pozovemo, prvi parametar koji joj moramo dati je odrediste. Primjer:

```
fprintf(stdprn, "Stampac je on-line \n");
```

Jedini je problem sto nam to nece funkcionisati u Turbo C pod Windows 95/98...

Sta sada?

OK, prije nego sto se pocnemo spoticati sa ovom materijom, samo da kazem da je teskoca C jezika dijelimicno u tome sto je neke stvari nemoguće objasniti ukoliko ih vec ne znate. Znam da zvuci malo nelogicno, ali je to tako. A posto ne znate o pointerima jos nista, ili o FILE pointerima (to ce tek doci), onda morate uzeti zdravo za gotovo ono sto cu vam reci, dok ne dodjemo do toga da vam to i objasnim.

Ukoliko zelite da vježbate output, i sa tim u vezi da ne trrosite papir, nas output cemo usmjeriti ka file (datoteci), i na tom primjeru vidjeti kako to sve funkcionise. Kada dodjemo do stampaca vec, onda je jednostavno promjenuti pointer ka stampacu, no otom-potom.

U nasem codu cemo ubaciti sljedece redove:

```
FILE *fp;
```

```
fp = fopen("c:\\test1.doc", "w");
```

```
fprintf(fp, "Stampac je on-line \n");
```

Ukratko objasnjenje: prvi red stvara pointer vrste FILE, drugi red usmjeruje taj pointer da pokazuje na pocetak datoteke c:\\test1.doc, i komanda 'w' naredjuje da je otvori u modu za pisanje. Ukoliko ne postoji pomenuta datoteka, ova komanda ce je naciniti. Ovako nacinjenu datoteku mozete otvoriti u MS Wordu, i ustvari vidjeti fomatiranje stranica na djelu.

Pazite na ovo:

```
#include <stdio.h>

void main()

{
    FILE *fp;

    fp = fopen("c:\\test1.doc", "w");

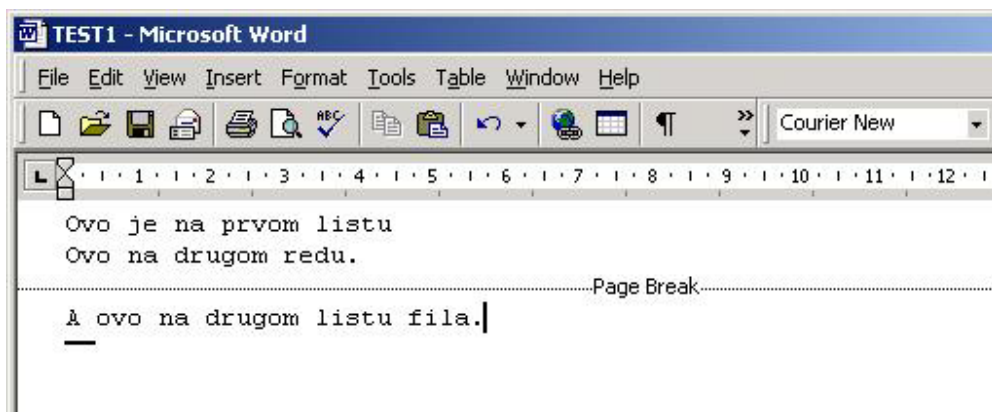
    fprintf(fp, "Ovo je na prvom listu.\n");

    fprintf(fp, "Ovo na drugom redu.\f");

    fprintf(fp, "A ovo na drugom listu fila.");
}
```

Ovdje nismo ubacili getch() funkciju, jer nam program nista nece ispisati na ekran, zapamtite, output smo usmjerili ka datoteci test1.doc u c:\\ direktoriji.

Otvorite test1.doc u MS Word programu, i sta cete dobiti:



Cool!

Dvapat cool:-)

Vec ste upoznati sa '\n' control karakterom, koji izdaje newline controlu, ovdje ste jos imali priliku vidjeti na djelu form-feed kontrolu, odn. '\f', koja forsira nastavak outputa na vrh sljedece stranice papira. Recimo da se nalazite u situaciji da ste razvili aplikaciju koja pravi izvjestaj, i u codu imate brojac linija koje stampate na listu papira. Kada taj brojac dostigne odredjenu vrjednost, htjet cete isprinatati footer te stranice, potom stampanje nastaviti na sljedecem listu. A to cete postici bas tom control karakterom.

Jos jedna interesantna stvar. Posto govorimo o stringovima, moguće je recimo, da string razbijemo na više komada, ako nadjemo da je suviše dugacak da bi stao u prozor kompilera. Dozvolite da prikazem primjerom.

```
fprintf(stdprn, "Ovo je primjer dugackog stringa,"  
" koji ne moze stati u jednom redu");
```

A output naseg primjera ce biti kao da je sve napisano pod istim navodnim znacima.,

```
Ovo je primjer dugackog stringa, koji ne moze stati u jednom redu
```

Iduci primjer formatiranja funkcionise takodje jednako za printf() i fprintf() funkcije, tako da cemo mi koristiti printf(). Ukucajte sljedeci programcic, izvrsite kompilaciju, u pogrenite dobijeni .exe file.

```
/* printtest.c */  
  
/* koristenje odrednica za sirinu prikazivanja podataka */  
  
#include <stdio.h>  
  
void main()  
{  
    float a = 5.443;  
  
    long b = 43759;  
  
    double c = 23000.035;  
  
    printf("\n123456789 123456789 123456789\n");  
  
    printf("%5.2f * %07ld %-4s %4.2lf\n", a, b, "$$", c);  
}
```

ukoliko ste sve uradili kako treba, output bi trebao biti identican ovom:

```
1 2 3 4 5 6 7 8 9 | 1 2 3 4 5 6 7 8 9 | 1 2 3 4 5 6 7 8 9 |  
5 . 4 4 | * | 0 0 4 3 7 5 9 | $ $ | | 2 3 0 0 0 . 0 3 |
```

Sljedi analiza;

Podaci se obicno prikazuju poredani na desnoj strani, i sirina polja koje zauzimaju, ukoliko nije naznacena, automatski raste da bi dozvolila mjesta onome sto prikazujemo.

Recimo, kada bi smo preuredili prethodni program, tako da iz zadnjeg reda izbacimo formate za sirinu polja, program bi pokusao uciniti sljedece:

1	2	3	4	5	6	7	8	9		1	2	3	4	5	6	7	8	9		1	2	3	4	5	6	7	8	9
1	2	3	4	5																								

U crveno polje, koje smo obiljezili sa 1, postavio bi vrjednost varijabile 'a', potom u nasem stringu sljedi slobodno mjest, znak '*' u polje 2, jos jedno mjesto, potom u polje 3 smjesta varijabilu 'b', opet mjesto, potom string "\$\$" u oplje 4, mjesto, zatim u polje 5 nasu zadnju varijabilu, 'c'. Znaci svakom karakteru i mjestu u nasem kontrol stringu pripada po jedno output mjesto.

A kako smo vec rekli da printf() funkcija automatski dodjeljuje onoliku sirinu kolika je potrebna tim vrjednostima, u stvarnosti cemo dobiti sljedeci output:

1	2	3	4	5	6	7	8	9		1	2	3	4	5	6	7	8	9		1	2	3	4	5	6	7	8	9
5	.	4	4	3	*		4	3	7	5	9		\$	\$		2	3	0	0	0	.	0	3	5				

Crvenim poljima smo obiljezili mjesto odakle program postavlja vrjednosti, poredani na desnoj strani.

Cesto se javlja potreba da nas output formatiramo, i tu na scenu nastupaju oni nasi projevi iz naseg gornjeg primjera.

```
printf("%5.2f * %07ld %-4s %4.2lf\n", a, b, "$$", c);
```

Sirnu polja odredjuje prva cifra koja sljedi znak '%', i za nasu prvu varijabilu ona je 5, tako da ce u nasem outputu ta vrjednost zauzimati 5 mjest, a kako radimo sa cetverocifrenim brojem, posto su vrjednosti poredani na desno, slobodno mjesto ostaje sa ljeve strane. Decimala odreduje koliko decimalnih mjesta ce se prikazati, i u ovom slucaju je 2 prisutno, tako da output pokazuje broj sa dvije decimale, i pored toga sto je nasa varijabila imala 3 decimale.

Ukoliko je prvi broj nula (0), kao u nasem drugom primjeru formatiranja vrjednosti sa varijabilom 'b', onda slobodna mjesta program ispisuje nulama, znaci umjesto slobodnog mjesta, postaviti ce nulu.

Za string "\$\$" smo odredili u našem primjeru 4 mjesta, ali znak minusa (-) ispred broja označava da će ta vrijednost biti poredana na lijevu stranu, tako da se dva slobodna mjesta javljaju sada sa desne strane.

Pri formatiranju varijable 'c', ostavili smo cijelom broju samo 4 mjesta, sa tim da smo za decimalu odvojili 2, kako to da se prikazala u cijelom obliku?

Kao što smo rekli, ukoliko je polje suviše usko, printf() će automatski proširiti polje da bi vrijednost stala, pridržavajući se naše želje za dvije decimale. ALI! Ukoliko imate polja iza te vrijednosti koje ste mukotrpno formatirali ispod nekog naslova, onda će oni biti izbaceni iz svog mjesta jer je naša varijabla 'c' zauzela suviše prostora.

Primjetili ste upotrebu novog format codea '%ld' za long int i '%lf' za double float?

Vama nista ne može promaci:-)))

OK. Toliko za danas, sutra nastavljamo sa Operatorima u C jeziku, spremite se.

Naravno, pravilo znate: Za sve *kritike* i *pohvale*, kao i *pitanja* koje vjerovatno imate, ili ako uočite neku grešku, javite se direktno autoru na [GitHub](#) kada je C tutorijal u pitanju, ili na [Facebook](#) za C++.

Aritmetički operatori

prikazuje aritmetičke operatore u C jeziku.

+, **-**, ***** i **/** funkcionišu manje više kao i u većini ostalih programskih jezika i u C jeziku se mogu primjeniti na većini vrsta vrijednosti ugrađenih u C jezik.

U djeljivosti int i char data vrsta, decimali se odbacuju, tako da će 10/3 uvijek biti jednako 3. U slučaju da vam je potrebna tačnost na visini decimala, u naznačenom djeljivanju ćete morati koristiti float data vrstu.

Modulus Operator '%' daje ostatak pri djeljivanju integrala, i kao takav se ne može koristiti na float ili double data tipovima.

Da pokazemo primjerom:

```
/* Primjer djeljivosti integralima */
```

```
#include <stdio.h>
```

```
void main()
```

```

{
    int x, y;

    printf("Unesite djelitelja i djelioca: ");

    scanf("%d%d", &x, &y);

    printf("\nRezultat je : %d", x/y);

    printf(" i ostatak je %d", x%y);

    getch();
}

```

i u zavisnosti od toga koje brojeve unosite, rezultati ce pokazivati prvo rezultat djelidbe integralima, a potom ostatak takvog djeljenja.

Ilustracije radi, ukoliko unesete 15 i 4, output ce biti:

```
Rezultat je: 4 i ostatak je 1
```

takodje ako unesete 20 i 4:

```
Rezultat je: 5 i ostatak je 0
```

Incrementacija i decrementacija je nesto uglavnom svojstveno C/C++ jeziku.

```

x++;      =      x = x + 1;
x--;      =      x = x - 1;

```

znaci ako je $x = 10$; poslje incrementacije $x++$, vrjednost x ce biti 11.

Incerementacija i decrementacija se mogu obaviti na dva nacina:

```

x++;      =      ++x;
x--;      =      --x

```

sa jednom vrlo bitnom razlikom: kada je increment operator ispred varijabile, onda C jezik izvodi incrementaciju prije koristenja vrjednosti. Primjer:

```
x = 5;
```

```
y = ++x;  
  
printf("y = %d, x = %d", y, x);
```

rezultat ce biti:

```
y = 6, x = 6
```

dok je slucaj obrnut kada je increment ili decrement operator iza varijabile, u tom slucaju ce prvo koristiti vrjednos, a potom izvorsiti incrementaciju ili decrementaciju. Primjer:

```
x = 5;  
  
y = x++;  
  
printf("y = %d, x = %d", y, x);
```

```
y = 5, x = 6
```

posto je prvo pripisao vrjednost x varijabile y varijabili, a tek potom izvrsio incrementaciju.

Nadam se da vam je ovo jasno, ukoliko nije, morat cu vas zamoliti da jos jednom predjete dok ne ukapirate, zbilja ne znam objasniti nimalo prostije.

A ako ste to vec savladali, onda je vrijeme da predjemo na Racionalne i Logicke operatore. Zurka!

Racionalni i logicki operatori

prikazuje Racionalne operatore u C jeziku, a Logicke operatore mozete vidjeti u

U smislu Relacionalnih i Logickih operatora, Relacionalno se odnosi na relaciju koje vrijednosti mogu imati jedno spram drugih, i Logicki se odnosi na nacin na koje su te relacije mogu spojiti

Kljucno mjesto u ovim operatorima je ideje TRUE i FALSE, odn. Istinito i Pogresno.

U C jeziku, Istinito je sve sto je razlicito od nule. Pogresno je nula.

Izrazi koji se koriste Relacionalnim i Logickim operatorima ce vratiti vrijednost nule za pogresno, i 1 za istinito.

Relacionalni operatori se koriste u odredjivanju odnosa jedne vrijednosti naspram druge. Uvijek ce vratiti 1 ili 0, u zavisnosti od rezultata testa.

Iduci program ilustrira rezultat spomenutih operatora i prikazuje rezultat kao 1 ili 0:

```
/* Primjer Relacionalnih Operatora*/

#include <stdio.h>

void main()
{
    int x, y;

    printf("Unesite dva broja: ");

    scanf("%d%d", &x, &y);

    printf("\n%d == %d je %d", x, y, x == y);

    printf("\n%d != %d je %d", x, y, x != y);

    printf("\n%d <= %d je %d", x, y, x <= y);

    printf("\n%d >= %d je %d", x, y, x >= y);

    printf("\n%d < %d je %d", x, y, x < y);

    printf("\n%d > %d je %d", x, y, x > y);

    getch();
}
```

Ovaj program unesite i kompilirajte, i experimentisite malo unoseci razne vrijednosti da bi ste vidjeli razliku u rezultatima.

Logicki operatori se koriste kao podrška osnovnim logickim operatorima AND, OR i NOT prema ovoj .

Tabela koristi naravno 1 za TRUE i 0 za FALSE.

Sljedeci program ilustrira Logicke Operatore:

```
/* Primjer Logickih Operatora*/
```

```

#include <stdio.h>

void main()
{
    int x, y;

    printf("Unesite dva broja ( 1 ili 0): ");

    scanf("%d%d", &x, &y);

    printf("\n%d AND %d je %d", x, y, x && y);

    printf("\n%d OR %d je %d", x, y, x || y);

    printf("\nNOT %d je %d", x, !x);

    getch();
}

```

Unesite program, i vježbajte malo, dok ne steknete rutinu kada su logicki operatori u pitanju.

Moguće je spojiti više operatora u jedan izraz, kao u sljedećem primjeru:

10 > 5 && !(10 < 9) || 3 <= 4

...izraz je naravno.....znate li?

Provjerite .

Kako ste prošli? Aaaaaaa.... Hocete da vas još malo resetam?

Vazi. Hajde, reci te mi koji je output sljedećeg programa, bas da vidimo koliko ste pratili ovo predavanje:

```

#include <stdio.h>

void main()
{
    int x;

    x = 100;

    printf("%d", x > 10 );

    getch();
}

```

Mislite da znate? Mislite da ste pogodili?

Provjerite .

Relacionalni i Logicki operatori se koriste u program-control naredbama, kao u svim loop i if naredbama. O tome malo kasnije, ali da bi vam aktivirao apetit o temi, kusnite malo .

I toliko sto se toga tice za sada.

Prelazimo na Assignment Operatora, i znam da ce te to naci toliko interesantnim, da vjerujem da jedva cekate...

Pa, ako je tako, onda sta cekamo.....

Malo o 'if' i 'for' naredbama

Samo da malo okusate prste na program-control naredbama.

U ovom primjeru, program koristi 'if' naredbu da bi ispisao parne brojeve izmedju 1 i 100:

```
/* Parni brojevi izmedju 1 i 100 */  
  
#include <stdio.h>  
  
void main()  
{  
    int i;  
  
    for( i = 1; i <= 100; i++ )  
    {  
        if ( ! (i % 2 ) )  
            printf ( "%d ", i );  
    }  
  
    getch()  
}
```

Output bi isao nesto ovako:

```
2 4 6 8 10 12 14 16 18 20 22 24 26 .....itd.
```

Sa malim modifikacijama bi ste mogli naciniti da program pita korisnika u kojim granicama zeli da ispise parne brojeve, ili cak i neparne, znate li koja modifikacija je potreban za to?

Odgovor .

OK, malo smo preduhitрили ovdje sa ovim, ali uglavnom mislim da ce te moci razumjeti tok programa i naredbe kojim se sluzimo.

Proglasili smo integral 'i', i zapazit ce te da mu nismo dodjelili vrjednost.

Ali bez brige, to ce mu uraditi 'for' naredba.

A ona uglavnom ide ovako:

Varijabili je postavljena pocetna vrjednost, u ovom slucaju 1.

Sljedece polje trazi uslov pod kojim 'for' funkcionise, u nasem slucaju sve dok je 'i' manje ili jednako 100.

Zadnje polje u 'for' naredbi mjenja varijabilu, ovdje vrsimo incrementaciju.

Ukratko, 'for' naredba u svakom svom loop-u prolazi kroz sve brojeve od 1 do 100.

Tu ga saceka 'if' naredba, i ukoliko je uslov tacan (pozitivan broj), izvrsava printf() funkciju. U suprotnom se controla programa vraca 'for' naredbi.

Da bi 'if' uslov bio ispunjen, postavili smo sljedecu spletku:

```
(!(i % 2))
```

i tu odredjujemo sta uzimamo a sta odbacujemo.

Recimo da je program tek poceo svoj posao, i 'for' je dodjelio varijabili 'i' vrjednost od 1.

Rezultat modulus djeljenja, odn. ostatak djeljenja $1/2 = 1$,

tako da je rezultat operatora unutar untrasnjih zagrada jednak 1.

Ali nama trebaju parni brojevi, znaci moramo rezultat 'izvrnuti' , u ovom slucaju negirati, sto ce reci da ce biti tacno za parne, a netacno za neparne brojeve, sasvim suprotno od onoga sto dobijamo modulusom.

Ali funkcionise, i to sasvim elegantno. Bas ono sto nam treba.

I, sto kazete?

Interesantno ++

Assignment operator

Assignment Operator je grubo prevedeno operator dodjele vrijednosti. Sastoji se od jednog znaka jednakosti '='.

Razlika između opšteprihacenog značenja znaka jednakosti i onoga što taj znak označava u C jeziku je sljedeći: u stvarnom životu znak jednakosti predstavlja izrazaj vrijednosti, odn. jednakost vrijednosti sa lijeve i desne strane tog znaka. U C jeziku, znak jednakosti pripisuje vrijednost sa desne strane znaka varijabli na lijevoj strani znaka.

Zamislite sljedeći isjecak coda iz nekog zamisljenog programa:

```
x = 10;

printf("\n%d", x);

x = 15;

printf("\n%d", x);

x = 20;

printf("\n%d", x);

x = x + 20;

printf("\n%d", x);
```

Ovo što smo ovdje napisali je sasvim uredu, dok bi, što se aritmetike tice, prva jednačina isključivala tačnost ostalih, zar ne? Ta ako je $x = 10$, ne može biti niti 15 niti 20, zar ne? A kamoli ova zadnja jednačina, ta kako neki broj može biti jednak tom istom broju plus 20???

Output ovog coda bi bio:



```
10
15
20
40
```

jer znak jednakosti u C jeziku pripisuje varijabli 'x' vrijednost sa desne strane znaka. Jasno?

Znaci, u prvom redu se varijabili pripisuje vrijednost od 10, i to potvrđujemo tako sto to ispisujemo na ekranu, tako da tacno znamo kolika je vrijednost pothranjena u varijabili 'x' poslje prvog znaka jednakosti.

Potom, u tu istu varijabilu smjestamo vrijednost 15, pa zatim 20.

Dakle, kada dodjemo do prezadnjeg reda, imamo varijabilu u kojoj je smjestena vrijednost 20, i u zadnjoj radnji cinimo sljedece: varijabili 'x' dodajemo vrijednost od 20, i potom tu novu vrijednost smjestamo u istu varijabilu 'x'. Otud u zadnjem redu outputa broj 40. Jednostavno u C jeziku, racunske radnje se odvijaju sa desna na lijevo, ali o tome u sljedecoj lekciji.

Turbo C dozvoljava da se Assignment Operator koristi u izrazima koje koriste druge operatore.

Primjer:

```
#include <stdio.h>

void main(void)
{
    int x, y, produkt;

    printf( "Unesite dva broja: ");

    scanf("%d%d", &x, &y);

    if ( (product = x * y) < 0)
        printf(" "\nJedan broj je negativan...");
    else
        printf("\nPozitivan produkt je: %d", produkt);
}
```

Zapazite izraz u zagradama if operatora. Prvenstveno je produktu dodjeljena vrijednost $x * y$. Potom je izraz u unatrasnjim zagradama usporedjena naspram nule. Ovaj code je sasvim validan, ustvari, na slican code ce te naci u mnogim profesionalno uradjenim programima. Da pogledamo kako ustvari funkcionise:

U Turbo C, Assingment Operator cini dvije stvari: prvo pridodaje vrijednost sa desne strane varijabili na lijevoj. Ali, kada je dio veceg izraza, Aassignment Operator proizvodi vrijednost desne strane izraza.

Dakle,($product = x * y$) dio izraza 'productu' vrijednost izraza ' $x * y$ ' kao i sto vraca tu vrijednost. I ta je vrijednost u ovom slucaju usporedjena sa nulom. Zagrade su potrebne jer je Assignment Operator manji po vrijednosti od Relationalnog Operatora.

A o operatorima, i njihovim vrijednostima u sljedecoj lekciji.

Operatori i presedani

Sljedeća tabela pokazuje operatore poredanih po nivoima vaznosti. Oni iz gornjih nivoa se odvijaju prije onih iz donjih, osim ako na redosljed nismo utjecali postavljanjem zagrada, posto se prvenstveno operatori rješavaju unutar zagrada, potom van njih.

Najveci

()	[]	->						
!	~	++	--	-(type)	*	&	sizeof	
*	/	%						
+	-							
<<	>>							
>	>=	<	<=					
==	!=							
&								
^								
&&								
?:								

Najmanji

=	+=	-=	*=	/=
,				

Ukoliko radimo sa operatorima iste vaznosti, one se u C jeziku rješavaju sa lijeva na desno.

Primjer:

```
x = 12 % 5 * 2;
```

U varijabli 'x' ce biti smjestena vrijednost od 4 ($12 \% 5 = 2$, $2 * 2 = 4$).

Ukoliko zelimo promjenuti redosljed izvršenja operatora, naravno, koristimo zagrade. Zagrade je moguće ugraditi unutar samih zagrada (Nesting), i u tim slučajevima, prvenstveno se rješavaju operatori iz unutrašnjih zagrada, da bi se napredovalo rješanju operatora van njih.

Primjer:

```
x = 25 - ( 2 * ( 10 + ( 8 / 2 ) ) );
```

Rješava se iz trećih zagrada, '8 / 2', dakle imamo:

$x = 25 - (2 * (10 + 4))$; Potom:

$x = 25 - (2 * 14)$; Zatim:

$x = 25 - 28$; I na kraju:

$x = - 3$;

Zagrade se koriste i u slucajevima kada nisu potrebni, iz razloga jasnoce coda. Sasvim cesto ce te naici na izraz za koji znate da ce evaluirati praviljno bez zagrada, ali da su koristene zagrade. To nije pogresno, cak stavise.

Za neke operatore sa gornje liste mozda jos ne znate, ali ce vam lista biti potrebna kada do njih vec dodjete. I polako, sta ste pozurili, vi bi da naucite sve odjednom:-))

Eto, toliko za danas. Ostatak dana imate slobodno.

Jesam dobar profesor, jelde?

Type konverzija

Kada u izrazima mjesamo konstante i varijabile raznih tipova, pretvorene su automatski u jedan tip. Vrlo bitno.

C kompiler ce "podici" sve vrjednosti na onu vrstu koju je najveca konstanta ili varijabila.

Ova radnja se odvija u parovima, kada se susretnu u izrazu dvije vrjednosti, ona koja je manje vrste, podize se na vecu. Pravila kojim se pokrivaju pretvaranje vrste vrjednosti su opisana u sljedecim tackama:

1: Svi char i short su pretvoreni u int, svi float su pretvorenu u double

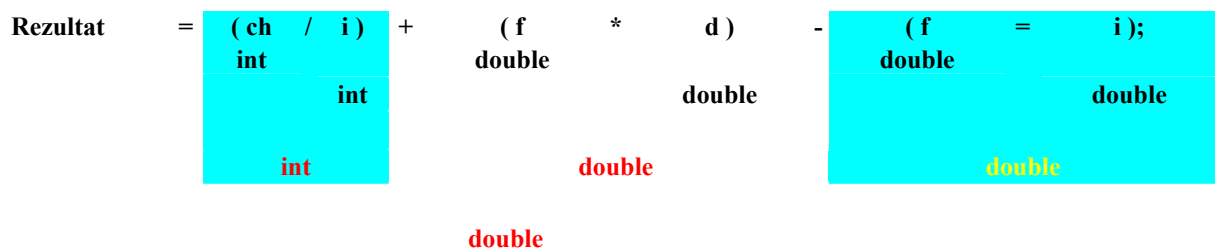
2: U parovima vrjednosti, ukoliko je jedna vrjednost long double, druga je pretvorena takodje. Ukoliko je jedna vrjednost double, druga je pretvorena takodje. Potom sljedi long, i na kraju, ukoliko je jedna vrjednost unsigned, druga je pretvorena takodje.

char ch;

int i;

float f;

double d;



double

Prvo, karakter `ch` je pretvoren u `int`, `i` float `f` je pretvoren u `double`. Onda je rezultat '`ch / i`' pretvoren u `double` jer '`f * d`' je `double`. I napokon, na kraju, rezultat je `double`, jer su obje vrijednosti sada već `double`.

Jednostavno, zar ne?

Type Cast, here we come...

Type cast, slobodni prostor i zagrade

Moguće je naterati izraz da postane određene vrste koristeći ono što zovemo "cast". Oblik cast-a je:

(vrsta) izraz;

gdje je 'vrsta' jedna od standardnih C vrsta vrijednosti.

Na engleskom to ide:

(type) expression;

I zbog toga koristim izraz Type Cast

Primjer:

Recimo da je `int x`; i vi želite da osigurate da će izraz '`x / 2`' u rješavanju postati `float`, da bi ste osigurali decimalna mjesta, mogli bi ste napisati:

`( float ) x / 2`

Ovdje je cast asocirana sa `int x`, koja proizvodi to da je cjela vrijednost `x` 'podignuta' na nivo floata, odn. decimalne vrijednosti, tako da će sada rješavanje biti decimalna vrijednost (`float`).

Ali, budite na oprezu. Ako to napisete na sljedeći način:

`( float ) ( x / 2 )`

dobit ćete rješavanje bez decimalnih vrijednosti. Zna li zasto?

Haj bas da vidimo. Kada mislite da ste dovoljno razmisljali, rjesenje cete naci .

Nista lakse.

Kada je u pitanju "bjeli prostor" izmedju izraza, komandi, date i slicnog, u C jeziku vam je dato na nahodjenje kako cete to upotrijebiti. Bitno je samo, iz prakticnih razloga, da slobodni prostor cini code citkijim. Inace, slobodni prostor nema utjecaja na sami program.

Sljedeca dva primjera su identicna:

```
x=643/(ulazni_broj)-y*(2234/saldo);
```

```
x = 643 / ( ulazni_broj ) - y * ( 2234 / saldo );
```

...mada je vise no ocigledno koji je izraz laksi za shvatiti i razumjeti. Znaci, ne moram vam vise ponavljati o tome? Odlicno.

Kada su zagrade u pitanju, koristenje nepotrebnih ili suvisnih zagrada ne izaziva greske u codu, niti ce usporavati izvršenje naredbe, tako da se od vas fakticki i ocekuje da ih koristite, da bi ste svoj code ucinili citkim, kako drugima, koji ce naknadno doci da odrzavaju vase programe, tako i sebi samima (jako je nezgodno kada niste u stanju opisati tok izvodjenja vaseg programa poslje 6 mjeseci, a vjerujte mi, desit ce vam se, ukoliko ne pazite na ove sitnice, kao i na komentare koje MORATE unositi u code).

U sljedecem primjeru, recite koji je isjecak coda jasniji iz pika?

```
x=y/3 - 34*temp - 127;
```

```
x = ( y / 3 ) - ( 34 * temp ) - 127;
```

Jasno da jeste, o tome nema zбора.

Nego da vas nesto pitam. Ovo je recimo problem, da vidim koliko ste ukopcani kada su ovakve stvari u pitanju:

Na stolu imate 3 kutije, i u kutijama su klikeri. Klikeri su obojeni plavom ili crvenom bojom. Kutije su obilježene pogresno, i na njima pise:

1. Kutija: 100 Plavih Klikera
2. Kutija: 100 Crvenih Klikera
3. Kutija: 50 Plavih i 50 Crvenih Klikera

Potrebno je iz jedne kutije, na "nevidjeno", izvuci jedan kliker, i da otkrijete sta se u kojoj kutiji nalazi.

Mislite da znate?

Odgovor u sljedecoj lekciji.

Idemo dalje....

Tvrdnje i izrazi

U C jeziku, Izraz (Expression) je kombinacija operatora i operanda. Kada Expression izvorsava akciju, onda je Tvrdnja (Statement) u programu. Statementi su osnovni dijelovi C programa.

Statement Tip:

Deklaracija (Declaration)

Dodjela (Assignment)

Poziv Funkcije (Function Call)

Kontrola (Control)

Združeni (Compound)

NULL (NULL)

Primjer:

```
int broj;
```

```
broj = 31
```

```
printf("%d", broj);
```

```
while(broj > 10)
```

```
    broj = broj + 3;
```

```
{
```

```
    broj = temp + 5;
```

```
    printf("%d", broj);
```

```
}
```

```
;
```

U C jeziku, Statement uvijek završava sa ':' (tacka-zarez).

Ukoliko se ne završava sa tacka-zarezom, bit ce uzeta u obzir kao izraz koji je dio veceg izraza ili tvrdnje, i zbog toga je ':' neophodno.

Statemente je moguće kombinirati unutar Compound Statementa, ili Združenih Tvrdnji. One su okružene velikim zagradama '{ }', da bi osigurali da su sve potredne komande izvršene pod istim uslovima.

Oh! Da ne zaboravim. Ukoliko tražite odgovor na pitanje koje sam vam postavio u prošloj lekciji, on se nalazi [ovdje](#). Bas bi volio znati kako ste prošli.

I to bi bilo to što se tiče ovog dijela našeg kursa. Dalje sljede Programski Tok, i znam da će vas to interesovati, jer je samo od sebe interesantno. Tako da vas neću više ovdje zadržavati....

Uslovna instrukcija 'if'

if (ako, ukoliko) u svom najosnovnijem obliku je:

```
if (izraz)
    tvrdnja;
```

Ukoliko je izraz tačan (tj. bilo što osim nule), onda tok programa prelazi na tvrdnju ili tvrdnje koje sljede 'if'. Naravno, ukoliko se radi o skupu tvrdnji (compound statements), onda će oni biti uokvireni velikim zagradama, i na taj način će kompajler znati da je potrebno izvršiti više od jedne tvrdnje.

Uzmimo situaciju gdje koristimo prostu 'if' instrukciju da bi smo dodavali pojene za tačne odgovore i ispisali poruku na ekran da bi smo obavjestili korisniku da odgovor ustvari jeste tačan.

U idućem primjeru, ukoliko igrač pogodi 'magicnu' brojku, rezultat tačnih odgovora mu se povećava za jedan, i na ekranu se ispise poruka u kojem se obavjestava o uspjehu. U suprotnom, rezultat mu nije povećan, i nikakva poruka se ne ispisuje.

```
#include <stdio.h>

void main()
{
    /* pocetak main funkcije */

    int izbor = 0;

    int rezultat = 0;

    printf("Koliko je 4 puta 7 ?");
```

```

scanf("%d", &izbor);

if ( izbor == 28 )
{
    /* pocetak 'if' uslova */
    rezultat++;

    printf("Izvršno! Dobijate jedan pojen!");

    printf("\nVas rezultat je: %d", rezultat);
}
/* kraj 'if' uslova */
/* Kraj main() funkcije

```

Koristimo dva para velikih zagrada, jedan par je naravno za main() funkciju, a drugi, kao granicnik skupu komandi koje se ostvaruju ako je uslov ispunjen, i koji naravno sljedi 'if'.

Zanimljivo je primjetiti sta bi se desilo kada bi smo izostaviti velike zagrade pri skupu komandi iz 'if'.

Mislim da bi smo to mogli uraditi kao vjezbu, ako ste spremni?

Ubacimo sljedeći code u kompiler:

```

#include <stdio.h>

void main()

{
    /* pocetak main funkcije */

    int izbor = 0;

    int rezultat = 0;

    printf("Koliko je 4 puta 7 ?");

    scanf("%d", &izbor);

    if ( izbor == 28 )
        rezultat++;

    printf("\nIzvršno! Dobijate jedan pojen!");

    printf("\nVas rezultat je: %d", rezultat);
}
/* Kraj main() funkcije

```

Pokrenimo program, i pogledajmo koje rezultate dobijamo.

Recimo da smo ubacili tačan odgovor, output programa bi bio:



Izvršno! Dobijate jedan pojen!

Vas rezultat je 1

OK, sve je uredu.

Hajmo sada ispitati program nekim drugim vrjednostima. Recimo da ubacimo 12 kao odgovor.

Pokrenimo program, i dobit cemo sljedeci rezultat:

Izvršno! Dobijate jedan pojen!

Vas rezultat je 0

Opa! Nesto ovdje ne stima!!! Ta program nam jednostavo laze!!!

Ili smo dobili pojen, i imamo 1, ili nismo dobili nista, znaci ne bi trebao da nam cestita!

Da vidimo sta je ovdje posredi:

kada tok programa dodje do ispisa prve poruke, i zatrazi unosenje vrjednosti u varijabilu 'izbor', iduci korak je da usporedi tu varijabilu sa nasom tacnom vrijednoscu, odn. 28.

```
if ( izbor == 28 )
```

i ukoliko je ta usporedba tacna, onda se odvija komanda (ili komande u slucaju skopa komandi, sto u nasem probnom primjeru nije slucaj) koja sljedi 'if' usporedbu.

Znaci, ubacili smo netacan odgovor, bilo koju vrjednost osim 28, i iduca komanda se nije ostvarila:

```
rezultat++;
```

I kada je program udari u tu 'tacku-zarez' komandu (ukoliko nije zaokruzena velikim zagradama), sto se programa tice, zavrsla je sa uslovnom 'if' komandom, i ponovo se vraca u redovan tok programa.

Iduce naredbe bi znaci bile:

```
printf("\nIzvršno! Dobijate jedan pojen!");  
  
printf("\nVaš rezultat je: %d", rezultat);
```

sto u našem slučaju nije tačno. E, iz tog razloga koristimo te zagrade, i grupisali smo više komandi iza 'if' usporedbe, i osigurali da se izvršavaju jedino ukoliko je 'if' tačan.

Znači, ukoliko vratimo naše zagrade na svoje mjesto, ukoliko 'if' nije tačan, kao što i nije, kada program preskoci 'if', iduća naredba koju dobije je

```
}
```

što će reći kraj programa. I tako i treba biti, jer nismo kodirali za slučaj ispisa poruke u slučaju netačnog odgovora.

Ustvari, to ćemo ostaviti za 'if-else' usporedbu, odlično, sada imamo dobar primjer za idući čas:-)

OK onda, nemojte brisati ovaj program iz kompilera, razradit ćemo je malo i dodati joj funkcionalnost.

Što bi se reklo, izradjujemo verziju 2 našeg programa.

Uslovna instrukcija 'if - else'

Struktura ove usporedbe vam dozvoljava da specificirate naredbe koje će se odnošiti u zavisnosti ishoda usporedbe.

Osnovni format strukture je:

```
if ( izraz )  
    tvrdnja  
  
else  
    tvrdnja
```

Naravno, ukoliko koristimo više od jedne komande pri izvršenju, potrebno ih je zaokviriti u velikim zagradama.

Ukoliko je usporedba tacna, komande koje sljede 'if' su izvršene, i one koje sljede 'else' se preskacu. Takodje i obrnuto, ukoliko je 'if' netacan, preskace se, i tok programa se nastavlja kod 'else', i komandama koje su navedene iza nje.

Da nadogradimo nas posljednji program, code koji cemo ubaciti je crvene boje:

```
#include <stdio.h>

void main()

{
    /* pocetak main funkcije */
    int izbor = 0;

    int rezultat = 0;

    printf("Koliko je 4 puta 7 ?");

    scanf("%d", &izbor);

    if ( izbor == 28 )
    {
        /* pocetak 'if' uslova */
        rezultat++;

        printf("Izvršno! Dobijate jedan pojen!");

        printf("\nVas rezultat je: %d", rezultat);
    }
    /* kraj 'if' uslova */
    else
    {
        /* pocetak 'else' uslova */
        rezultat--;

        printf("\nNetacno! Odgovor je 28");

        printf("\nVas rezultat je %d", rezultat);
    }
    /* kraj 'else' uslova */
    printf("\nDa oprobamo jos koji problem...");
}
/* Kraj main() funkcije */
```

U ovoj verziji naseg programa, 'rezultat' je povecan za jedan ukoliko je nas odgovor tacan. U suprotnom, kontrola programa prelazi na 'else', U tom slucaju je rezultat umanjen za jedan, odn. postaje -1 u nasem slucaju, i na ekranu je poslata poruka o nasem neuspjehu, tacan rezultat, i nas rezultat (mizernih -1).

Poslje toga smo dodali jos jednu poruku, koja se ispisuje u svakom slucaju, posto nije dio skupa komandi koje potpadaju 'if' ili 'else' usporedbama.

Jednostavno, zar ne?

Ali!

Nije to sve,nastavak sljedi, 'else - if' struktura.....

Uslovna instrukcija 'else - if'

Treća struktura 'if' kontrole, 'else - if' dozvoljava neograničen broj alternativnih tvrdnji koje možete izabrati. Format ove strukture je:

```
if ( izraz1)
    tvrdnja;
else if( izraz2)
    tvrdnja;
else if( izraz3)
    tvrdnja;
    :
    :
else
    tvrdnja;
```

Program prvo uspoređuje 'if' komandu, i ukoliko je tačna, izvršava tvrdnju koju je sljede, da bi potom tok programa nastavio tek iza posljednjeg 'else' usporedbe i njenih krajnjih komandi.

Ukoliko je 'if' netačna, program ide redom po 'else if' usporedbama, dok ne nađje na tačnu, da bi potom izvršio navedene komande. Po izvršenju preskace preostale 'else if' i 'else' usporedbu da bi nastavio tok programa poslije njih.

Ukoliko ne nađje na tačnu 'else if' usporedbu, posljednja 'else' usporedba (ukoliko je prisutan, jer nije obavezna) je izvršena.

Verzija 3 naseg programa sljedi, ukucajte sljedeći code u kompiler (novine su u raznim bojama):

```
#include <stdio.h>

void main()

{
    int izbor = 0;

    int rezultat = 0;

    printf("Koliko je 8 puta 4?");

    scanf("%d", &izbor);

    if ( izbor == 32)
    {
```

```

        rezultat++;

        printf("Izvršno! Dobijate jedan pojen!");

        printf("\nVas rezultat je: %d", rezultat);
    }
    else if ( izbor == 12 )
    {
        rezultat--;

        printf("\nNetacno! Mora da ste zbrajali");

        printf("\nVas rezultat je %d", rezultat);
    }
    else if ( izbor == 4 )
    {
        rezultat--;

        printf("\nNetacno! Mora da ste oduzimali");

        printf("\nVas rezultat je %d", rezultat);
    }
    else if ( izbor == 2 )
    {
        rezultat--;

        printf("\nNetacno! Mora da ste djelili");

        printf("\nVas rezultat je %d", rezultat);
    }
    else
    {
        rezultat--;

        printf("\nNetacno! Odgovor je 28");

        printf("\nVas rezultat je %d", rezultat);
    }
    printf("\nDa oprobamo jos koji problem...");
}

```

Milim da je malo objasnjiva. Code se lako prati, i objasnjenja koja sam dao su savim jasno prikazana. Ovdje smo dali korisniku malo bolja objasnjanje u smislu njegovih netacnih odgovora, i sve dok kontrola programa ne naide na tacnu usporedbu, ignorise komande koje potpadaju datoj usporedbi. Nista jednostavnije.

Ali, ovo je malo neelegantno.

Zelite elegantnije rjesenje?

switch statment!

U napad.....

'switch' statment

Iako 'if-else-if' kombinacija uslova moze izvesti test koji se bavi izborom jednog od vise mogucih ponudjenih rjesenja, ni u kojem slucaju mu ne mozemo pridodati atribut 'elegantong' rjesenja.

Code postaje tezak, nejasan, zadaje problema cak i samim autorima poslje odredenog vremena, tako da je nemoguće sa tačnošću znati šta smo to imali na umu kada smo codirali odredjenu sekvencu.

Iz tih razloga C ima ugradjenu komandu koja se bavi tim stvarima, a zove se 'switch' (u slobodnom prevodu sklopka, prekidač, zamjena...).

U switch komandi, varijabila je usporedjivana sa listom cijelih vrijednosti ili char konstanti, i kada je nacinjena usporedba, komande i tvrdnje koje su dodjeljene tom izboru su izvršene.

Opsta struktura switch komande je:

```
switch (izraz)
{
    case konstanta1:
    {
        tvrdnje;
        break;
    }
    case konstanta 2:
    {
        tvrdnje;
        break;
    }
    :
    :
    :
}
case konstanta n:
{
```

```

        tvrdnje;

        break;
    }

    default:

    {
        tvrdnje;
    }
}

```

... gdje se 'default' izvršava ukoliko nijedna usporedba nije uspješna (imajući na umu da 'default' nije obavezan u strukturi komande, i ukoliko nije prisutan, komande se ne izvršavaju ukoliko su sve usporedbe neuspješne).

Kada je usporedba uspješna, komande i tvrdnje se izvršavaju dok program ne naiđe na 'break' komandu, ili u slučaju 'default' ili posljednje 'case' usporedbe, na kraj 'switch' komande, koja je označena zadnjim velikim zagradama.

Tri su bitne stvari koje morate zapamtiti kada je 'switch' u pitanju, a o njima saznajte

Cesto, koristimo 'switch' da bi smo odredili tok programa poslije izbora kojeg je načinio korisnik programa u menu. To ćemo pokazati u sljedećem primjeru. Program kojeg ćemo sada ispisati pretvara brojeve iz jedne baze u drugu. Npr. broj decimalne baze (10) u hexadecimalnu (16) ili octalnu (8).

```

#include <stdio.h>

void main()
{
    int izbor, vrijednost;

    printf("Pretvori:\n");

    printf(" 1: decimalu u hexadecimalu\n");

    printf(" 2: hexadecimalu u decimalu\n");

    printf(" 3: decimalu u octal\n");

    printf(" 4: octal u decimalu\n");

    printf("Unesite vas izbor: ");

    scanf("%d", &izbor);

    switch (izbor)
    {
        case 1:

```

```

        printf("Unesite decimalnu vrijednost: ");

        scanf("%d", &vrjednost);

        printf("%d kao hexadecimala je: %x", vrjednost, vrjednost);

        break;
    case 2:
        printf("Unesite hexadecimalnu vrijednost: ");

        scanf("%x", &vrjednost);

        printf("%x kao decimala je: %d", vrjednost, vrjednost);

        break
    case 3:
        printf("Unesite decimalnu vrijednost: ");

        scanf("%d", &vrjednost);

        printf("%d kao octala je: %o", vrjednost, vrjednost);

        break
    case 4:
        printf("Unesite opctalnu vrijednost: ");

        scanf("%o", &vrjednost);

        printf("%o kao hexadecimala je: %d", vrjednost, vrjednost);

        break;
    default:
        printf("Unijeli ste pogresan izbor, pokusajte ponovo!");
}
}

```

Kao sto vidite, 'default' je pogodan da bi u ovom slucaju obavjestio korisnika da je unio nepostojecu vrijednost iz menua, i na taj nacin smo ljepo i elegantno zaokruzili sve moguće vrijednosti koje neko može unijeti.

'break' komanda, striktno govoreći, nije neophodna. 'switch' može funkcionirati i bez njega. 'break' koristimo da bi smo prekinuli izvršenje skupa komandi koje smo asocijirali sa pojedinim konstantama pri izboru. Ukoliko program ne naiđe na 'break' komandu, ona će nastaviti izvršenje programa sa komandama koje se nalaze u sljedećem 'case'. Pomalo čudno, jelde? Da ilustriramo....

```
#include <stdio.h>
```

```
void main()
```

```

{
    int t;

    for(t=0; t<10; t++)
        switch (t)
        {
            case 1:
                printf("Snijeg");

                break;
            case 2:
                printf(" pade ") ;
            case 3:
                printf(" na behar, na voce\n");
            case 5:
            case 6:
                printf( " tralala ");

                break;
            case 7:
            case 8:
            case 9:
                printf(" .");
        }
}

```

I kada pokrenemo nas melodican program, dobijamo:

```

Snijeg pade na behar, na voce
na behar, na voce
tralala tralala ...

```

Ovaj program ilustrira cinjenicu da mozemo imati prazne 'case' tvrednje, i ovo je od pomoci kada nekoliko razlicitih uslova zahtjeva isovjetne komande. Kao sto mozete zakljuciti, dozvoljava da se kutarisemo nepotrebnog ponavljanja coda pri takvim slucajevima.

Ova lekcija mozda zahtjeva da je ponovite kojiput dok ovo ne savladate. Pokusajte pratiti u zadnjem primjeru kako smo dobili bas ono sto i jesmo kao rezultat programa, da bi vam postalo jasnije u vezi 'switch' strukture, i odnos 'switch' i 'case', kao i 'break' komande.

U iducoj lekciji pocinjemo da se bavimo petljama, prijateljem svih programera.

Jesam li vam rekao da je ovo interesantno?

E pa da znate da jeste.....

Loops (petlje)

'for' petlja

Loops (petlje), dozvoljavaju da se skupovi instrukcija ponavljaju dok se odredjeni uslovi ne ispune.

Turbo C podrzava sve petlje koje se javljaju u modernim strukturnim programima, a to su: `for`, `while`, `do-while`.

U svom opstem obliku, 'for' (za vrijeme trajanja, dok traje...) petlja zauzima sledeci oblik:

```
for ( inicijalizacija; uslov; inkrementacija) instrukcija;
```

U najprostijoj formi, inicijalizacija je assignment statment koja se koristi da bi se postavila kontrolna varijabila petlje.

Uslov je relacionalni izraz koji odredjuje kada ce se petlja završiti stimo provjerava kontrolnu varijabilu nasprav neke vrjednosti.

Inkrementacija odredjuje kako ce se promjenuti kontrolna varijabila petlje pri svakom prolazku kroz samu petlju.

Ova tri djela 'for' petlje su odvojena 'tacka-zarezima' (;).

'for' petlja ce nastaviti da se izvrsava dok got je uslov tacan.

Onog momenta kada uslov postane netacan, izvršenje programa prelazi na sljedecu tvrdnju koja sljedi petlju.

Sljedeci kratki program ispisuje brojeve od 1 do 100:

```
#include <stdio.h>

void main()
{
    int i;

    for ( i = 1 ; i <= 100 ; i++)
        printf ( "%d ", i );
}
```

```
}
```

U ovom primjeru, int i je data vrijednost od 1, i posto je $1 < 100$, procedura 'printf()' je pozvata, koja u nasem slucaju ispisuje vrijednost varijabile na ekran. Potom, kontrola je ponovo u petlji, inkrementacija nastupa, i vrijednost je ponovo usporedjena u uslovnom djelu petlje. Ovaj proces se ponavlja do momenat kada varijabila 'i' postane veca od 100, tada uslov vise nije tacan, i petlja ispusta kontrolu programa na sljedeci red, koji je u nasem slucaju kraj samog programa.

U ovom slucaju 'i' je kontrolna varijabila petlje, koja je inkrementovana pri svakom ponavljanju petlje.

Petlja ne mora samo inkrementovati varijabili, moze je i dekrementovati.

Sljedeci primjer ispisuje brojeve od 100 do 1 na ekran:

```
#include <stdio.h>

void main()

{
    int i;

    for ( i = 100 ; i > 0 ; i--)
        printf ( "%d ", i );
}
```

Primjecujeteli razlike izmedju ova dva programa?

I niste samo ograniceni na inkremenataciju i dekrementacija, ustvari, umjesto inkrementacije mozete koristiti bilo koju assignment statment.

Npr., sljedeci program ispisuje svaki peti broj od 1 do 100:

```
#include <stdio.h>

void main()

{
    int i;

    for ( i = 0; i <= 100 ; i = i + 5 )
        printf ( "%d ", i );
}
```

Koristeci velike zagrade, mozemo, naravno, uciniti da se izvrsava blok coda umjesto jedne instrukcije, ali to vec znamo, jelde???

```
#include <stdio.h>

void main()

{
    int i;

    for ( i = 1 ; i <= 100 ; i++)

    {
        printf ( "ovo je i: %d ", i );

        printf ( " i ovo je i na kvadrat %d\n", i * i )
    }
}
```

Bitna stvar koju morate zapamtiti kod 'for' petlji je sljedece: provjera uslova petlje se izvrsava prije ulaska u petlju, tako da se moze desiti da se kroz petlju ne prodje uopste ukoliko uslov nije ispunjen kada se zapocinje petlja.

Uzmimo sljedeci primjer:

```
y = 20;

for ( z = 20 ; z != y; ++z ) printf ( " %d", z );

printf ( " %d", z );
```

U spomenutom primjeru, petlja se nikada nece izvorsiti, jer su 'y' i 'z' ustvari jednaki pri ulasku u petlju. Zbog toga petlja nikada nije izvorsena, i inkrementacija varijable nije ucinjena. Vrlo bitno.

'for' petlja ima i svoje varijacije, do sada smo gledali samo na njegov osnovni oblik. A te varijacije povecavaju njegovu snagu, fleksibilnost i pogodnost pri pojedinim programskim situacijama.

Jedan od cescih varijacija je korištenje

Umjesto standarnih uslova petlje, koje smo do sada upotrebljavali, moguće je kao uslov postaviti

Bitno je spomenuti i (koju trebamo izbjeći ukoliko je ne želimo, ali koja može i koristiti u programu)

(odn. petlja koja ne izvršava instrukcije).

I na kraju, kako

I što se tiče 'for' petlje, to bi za sada bilo to:-)

Iduća petlja je 'while', pa vidimo se tamo....

'for' petlja sa više kontrolnih varijabla

Jedna od čestih varijacija na temu osnovnih 'for' petlji je ta da kontrolu petlje vrši ne jedan, već više kontrolnih varijabla.

Sljedeći primjer prikazuje programčić sa petljom, gdje varijable 'x' i 'y' kontrolisu izvođenje petlje:

```
#include <stdio.h>

void main()

{
    int x, y;

    for ( x = 0, y = 0; x + y < 100; ++x, y++)
        printf ( " %d ", x + y );
}
```

Program ispisuje svaki drugi broj između 0 i 98.

Primjetujete li kako su korišteni zarezi? Jeli vam sada jasnije zašto se operatori petlje rastavljaju 'tacka-zarezom'? Naravno, zarez nam ostaje da bi smo mogli imati veći broj varijabla, i da bi smo ih uspjeli logično rastaviti. Zgodno, zar ne?

Zarez je ustvari C operator, koji doslovno prevedeno znači: 'uradi ovo i ovo'.

Toliko za sada o ovoj varijaciji 'for' petlje.

Idemo dalje, njih ima još....

Uslovni izraz kao kontrola petlje

Kontrolni izraz petlje ne mora provjeravati kontrolnu varijabilu petlje naspram neke predodredjene vrijednosti. Ustvari, bilo koji validni C izraz može poslužiti. Tako da možete provjeravati za nekoliko uslova koje će određivati izlazak iz petlje.

Da bi to ilustrirali, sljedeći program pomaže djeci u učenju zbrajanja.

Ukoliko korisnik nadje da je umoran, i želi stati sa programom, on ili ona ukucava 'N' kada je program upita dali želi nastaviti.

Obratite posebnu pažnju na uslov u 'for' petlji. Uslovni izraz će ponoviti petlju 25 puta, ili dok korisnik ne odgovori negativno na pitanje dali želi nastaviti.

```
#include <stdio.h>

#include <conio.h>

void main()
{
    int i, j, odgovor;

    char gotovo = ' ';

    for (i = 0; i < 25 && gotovo != 'N'; i++)
    {
        for (j = 1; j < 10; j++)
        {
            printf ("Koliko je %d +%d? ", i, j);

            scanf (" %d", &odgovor);

            if ( odgovor != i + j)
                printf (" Pogresno!\n");
            else
                printf ("Tacno!\n");
        }
    }
}
```

Jos jedna interesantna stvar kod 'for' petlji, je da djelovi definicije petlje nisu neophodni da bi se petlja mogla ostvariti.

Primjer:

```
for ( x = 0 ; x != 10 ; )  
    scanf("%d", &x);
```

Primjetit ce te da u ovom primjeru inkrementacijski dio deklaracije petlje uopste ne postoji, a da petlja i dalje funkcionise...

Uzgred, spomenuti primjer ce se zadržati u petlji dok se ne unese broj 10.

O ovome cemo malo kasnije, ustvari, govorit cemo o ovome u sljedecoj lekciji, kada spominjemo beskonacne petlje....

Beskonacna 'for' petlja

Da se vratimo malo na nas prosli primjer, iz prosle lekcije:

```
for ( x = 0 ; x != 10 ; )  
    scanf("%d", &x);
```

Kao sto smo rekli, u ovom primjeru inkrementacijski dio deklaracije petlje uopste ne postoji, a to znaci da kontrolna varijabila petlje nije promjenjena pri toku programa kroz petlju.

Jedini nacin da se izađe iz te petlje, je da se unese broj 10.

Ako razvijemo tu logiku do kraja, dolazimo do interesantne pojave beskonacne 'for' petlje.

Sada vidim da se pitate, zasto ce nam beskonacna petlja, zar nije to nesto sto trebamo izbjegavata?

Dakako, treba to izbjegavati kada to ne zelimo, ali postoje prilike kada nam upravo treba beskonacna petlja... da objasnim:

```
for ( ; ; )  
    printf ("Ova je beskonacna petlja \n");
```

U ovom primjeru, program ce se uvaliti u beskonacnu petlju iz koje nece moci izaci.

Naime, za nenaznacenu vrjednost, koju ne usporedjujemo, i ne inkremetiramo, petlja je tacna, i u beskonacnosti ce izvorsavati datu naredbu, a to je da ispise taks na ekran.

Glupo, jelda?

Jeste u ovoj formi, ali zamislite da su naredbe u petlji menu programa, koje opet pozivaju svoje specificne funkcije, i sve dok ne zadovoljimo jednu stavku menua, koja u sebi sadrzi komandu za izbijanja iz petlje, onda beskonacna petlja ima malo smisla....

Prosto, jede?

O toj komandi cemo u iducoj lekciji, ali je samo vrjedno spomenuti da beskonacnu petlju mozete napraviti i nehotice, tj. da je ne zelite napraviti, i ne moram vam reci da ce vam uprskati program, a mozda i krahirati kompjuter, zato pazite sta radite sa petljama.

A sada o tome kako komandom izaci iz petlje, bila ona beskonacna ili ne....

Izbijanje iz petlje ('break' komanda)

U primjeru beskonacne petlje:

```
for ( ;; )
```

mora postojati nacina da bi se elegantno izaslo iz petlje, u suprotnom, program bi bio beskoristan.

To ostvarujemo 'break' naredbom.

Ukratko, u tjelu petlje, kada program naidje na 'break' komandu, ona izaziva trenutnu terminaciju petlje.

Program kontrol nastavlja na sljedecoj liniji iza petlje.

```
for ( ;; )
```

```
{
```

```
    ch = getch();

    if ( ch == 'A' )
        break;
}

printf ("Pritisli ste tipku A");
```

Petlja ce se odvijati sve dok ne pritisnemo veliko slovo 'a'

Jedna napomena, ove lekcije izgledaju malo kratke, ali zbog jasnoce gradiva, mislim da je ovako najlakse.

I sljedeca lekcija je kratka, tek par redaka, mozete je savladati za minutu-dvije..

Petlja bez tijela

Statment, po sintaksi C jezika, moze biti prazan.

To znaci da tjelo 'for' petlje (a i bilo koje druge petlje) moze biti prazno.

Ta cinjenica nam moze pomoci da bi smo poboljsali ucinkovitost programa u pojedinim slucajevima kao i u stvaranju 'time-delay' petlji.

Primjer:

```
for ( x = 0 ; x < NEKA_VRJEDNOST ; x++ );
```

Kao sto vidite, petlja je zavrшена sa tackom-zarez, sto do sada nismo vidjeli. Naime, uglavnom tacka-zarez dolazi na izrazima koje se izvrsavaju unutar petlje....

Ali u ovom slucaju cinjenica da je tacka-zarez tu gdje jeste ukazuje kompileru da nasa petlja ne izvrsava komande, sem inkrementacije kontrolne varijabile.

Sasvim dozvoljeno u C jeziku.

I sto se tice 'for' petlje, to bi bilo to:-)

'while' petlja

Druga dostupna petlja u C jeziku je 'while'.

Osnovni oblik petlje je:

```
while (uslov) tvrdnja;
```

Gdje **tvrdnja** moze biti:

- prazno, kao sto smo prije naveli
- jedna tvrdnja (komanda)
- blok komandi i upustva

Uslov moze biti bilo koji prihvatljiv izraz.

Petlja se ponavlja sve dok je **uslov** tacan.

Kada **uslov** postane netacan, kontrola programa prelazi na sljedecu liniju coda koja sljedi petlju.

Sljedeci primjer pokazuje ruitinu koja jednostavno ponavlja petlju dok se ne pritisne veliko slovo 'a' ('A') :

```
sacekaj_char()
{
    char ch;

    ch = '\0'; /* Inicijalizacija ch varijabile */

    while (ch != 'A')
        ch = getch();
}
```

Prvo, **ch** je inicializirana na **NULL**. Kao lokalna varijabila, njegova vrjednost nije poznata kada **sacekaj_char()** zapocinje izvršenje.

While petlja zapocinje stim sto provjerava dali je '**ch**' jednaka '**A**'

Posto smo prije toga inicijalizirali '**ch**' sa vrjednos **NULL**, provjera je izvršena, i petlja zapocinje.

Svaki put kada se pritisne neka tipka na tastituri, provjera je ponovo izvršena.

Jednom kada se pritisne '**A**', uslov postaje netacan, jer '**ch**' == '**A**', i petlja završava, odn. prebacuje kontrolu programa na sljedeću liniju coda.

Kao i sa '**for**' petljama, '**while**' provjerava uslov na pocetku petlje, sto znaci da postoje prilike kada se petlja uopšte neće izvršiti. Zbog tog razloga smo morali inicijalizirati nasu varijabilu prije petlje, da se kojim slucajem njenom deklaracijom u njoj ne nadje '**A**' i bez naseg znanja.

Zato sto je uslov provjeravan na pocetku petlje, '**while**' petlja pogoduje situacijama kada i ne želite da vam se petlja u svim uslovima izvršava, bez potrebe da uslovljavate posebnim uslovima pristup petlji.

Primjer cemo pokazati u sljedećem programu.

Program '**centriraj()**' koristi '**while**' petlju da bi ispisao tacan broj praznih mjesta da bi poredao liniju teksta na sred ekrana rezolucije 80 karaktera sirine.

Ukoliko je '**duzina**' jednaka nuli, kao u slucajevima kada je linija teksta koja se treba centrirati duga svih tih 80 karaktera, petlja se neće izvršavati.

Ovaj program koristi posebnu library funkciju '**strlen()**', koja vraca duzinu stringa koju primi kao argument. Pomenuta funkcija koristi **STRING.H** heder file.

```
/* Program koji centrirá tekst na ekranu */

#include <stdio.h>

#include <string.h>

void centriraj(int);

void main(void)
{
    char str[255];

    printf("Ukućaj string: ");

    gets(str);

    centriraj(str);
}

/* Izracunaj i izbací potreban broj praznih mjesta prije stringa da bi bio centriran na ekranu */

void centriraj(int duzina)
{
    duzina = (80 - duzina) / 2;

    while (duzina > 0 )
```

```
    {  
        printf(" ");  
        duzina--;  
    }  
}
```

Slučaj 'while' petlje bez tjela? Sljedi:

```
while (( ch = getch()) != 'A' );
```

Kao što vidite, **tacka-zarez** sljedi zagradu koja sadržaje uslov petlje, znači sa tim je petlja završena, ne izvršaje više komandi, osim što provjerava za ukucanu tastituru '**A**', i ponavljaće petlju sve dok je ne nadje...

Iduća lekcija je do/while petlje.... vidimo se tamo...

'do - while' petlje

Za razliku od 'for' i 'while' petlje, gdje se uslov provjerava prije ulaska u petlju, do/while petlja provjerava uslov na kraju petlje.

To znači da će se do/while petlja uvijek izvršiti barem jednom.

Osnovni oblik do/while petlje je:

```
do  
{  
  
    komande;
```

```
} while (uslov);
```

Viticae zagrade nisi potrebne ukoliko je samo jedna komanda prisutna, ali se obicno koriste zbog citkosti coda i jasnoce namjera programera.

Sljedeci program koristi do/while petlju da bi citao ukucate brojeve, sve dok ne naidje na broj manji od 100:

```
#include <stdio.h>

void main (void)

{
    int broj;

    do
    {
        scanf("%d", &broj);

    } while (broj > 100);

}
```

Vjerovatno najcesca primjena do/while petlje je u rutini izbora menu-a.

Posto cete htijeti da se izbor menu-a izvrsi barem jedanput, do/while petlja je ocigledan izbor.

Testirajuci validan izbor na dnu petlje, mozete ponovno zattraziti unosenje izbora ukoliko dati izbor nije validan, sve dok korisnik ne unese jedan od odobrenih izbora.

Sljedeci isjecak coda pokazuje kako naciniti menu sa do/while petljom:

```
/* Osigurajte da korisnik unese validnu opciju */

do
{
    printf ("Pretvori: \n");

    printf ("\t1: decimalni u hexadecimalni\n");

    printf ("\t2: hexadecimalni u decimalni\n");

    printf ("\t3: decimalni u octalni\n");

    printf ("\toctalni u decimalni\n");
```

```
printf ("unesite vas izbor: ");

scanf("%d", &izbor);
} while (izbor < 1 || izbor > 4);
```

Poslje prikaza opcija, program ce uci u petlju sve dok se ne pritisne neka od validnih opcija.

OK, sada prelazimo na funkcije, sve sto ste uvijek htijeli da znate, a niste imali koga da pitate u vezi stvaranja i koristenja funkcija.

C Tutorijal

Autor: Miroslav Vasiljević **Email:** miroslav@prijedor.com

Napomena: U ovom tutorijalu, osim što će biti objašnjen programski jezik C, biće prikazane osnove rada u BORLAND C-u 3.x.

Uputstvo za korišćenje tutorijala:

Čitajte pažljivo, i što je najvažnije sa razumijevanjem, zato jer programiranje se ne može samo nabubati, mora se razumijeti. Prije nego što pročitate objašnjenja za pojedine primjere, pokušajte sami razumijeti zašto je nešto u tom programu napisano baš tako kako je, a ne drugačije. Zadatke koji se nalaze u ovom tutorijalu svakako probajte sami riješiti prije nego što pogledate rješenje. Kako gradivo odmiče neće svaki put biti objašnjen svaki pojedini korak u primjerima programa.

Na kraju kada proučite cijeli tutorijal svakako sami sebi stvarajte zadatke, i po mogućnosti riješite ih. Preporučujem da se samostalno potrudite pozabaviti sa programiranjem nekih grafičkih efekata, kao npr. kretanje neke loptice preko ekrana i sl., jer često se programiranje najbolje nauči tek kada se krenu raditi grafički programi.

Ako radite pod Win OS-om, tad ćete source kodove primjera ili rješenja najlakše prebaciti u Borland C tako da pokrenete BC u prozoru (ne u full-screenu), te označite željeni kod, i pritisnete dugme *paste* u dos-prozoru. Windowsi će tada oponašajući kucanje prenijeti taj tekst u BC, pri tome morate paziti da aktivni prozor u BC-u bude source window.

1. Osnovni tipovi podataka.

Prvo što ću reći je zašto je bitno razlikovati tipove podataka: U C-u svaku varijablu prije nego je počnete koristiti, morate je deklarirati, to znači da određujete kog je varijabla tipa podatka. Da bi uspješno napisali efikasan program koji zauzima tačno onoliko memorije koliko je potrebno, da neka vrijednost u programu slučajno nebi izašla iz opsega itd, morate znati osnovne razlike između osnovnih tipova podataka. Razlika može biti u veličini podatka (8bit, 16bit, ...), druga razlika može biti u predznaku, tj. dali je sa nekim tipom podatka moguće označiti negativnu vrijednosti ili nije. Sada ću navesti par osnovnih tipova podataka i osnovne karakteristike tih tipova:

int	16 bit, cjelobrojan, ima predznak, opseg: -32768 do +32767
-----	--

unsigned int	16 bit, cjelobrojan, nema predznak, opseg: 0 do +65535
char	8 bit, cjelobrojan, odgovara jednom znaku, ima predznak, opseg -128 do +127
unsigned char	8 bit, cjelobrojan, nema predznak, opseg: 0 do +255
long	32 bit, cjelobrojan, ima predznak, opseg: -2 147 483 648 do + 2 147 483 647
float	32 bit, broj sa pomičnim zarezom (sa ovim tipom moguće je pamti decimalni broj), sa predznakom, opseg: $-3.4 \cdot (10 \exp -38)$ do $+3.4 \cdot (10 \exp +38)$

1.1 Kako deklarirati varijablu:

Varijable se deklariraju tako da se navede oznaka tipa podatka, te iza te oznake navedemo sva imena varijabli odvojena zarezom za koje želimo da budu tog tipa. Na kraju, red mora završiti sa tačkom -zarez, na primjer:

int a,b; - deklariraju se varijable a i b kao tipovi podatka int
float neki_broj; - varijable *neki_broj* deklariraju se kao tip podatka float

2. Deklaracija funkcija:

Dobar stil programiranja je "razbiti" program na više različitih funkcija, gdje svaka funkcija obavlja neki specifični zadatak. Opšti oblik funkcije je da prima određene parametre kao ulaz, a vraća kao rezultat jednu jedinu veličinu. Pod pojmom deklaracije funkcije podrazumijeva se određivanje koje tipove podataka funkcija prima kao parametar, te koji tip podatka funkcija vraća kao rezultat. Opšti oblik deklaracije funkcije je da se upiše prvo oznaka tipa podatka koji funkcija vraća, zatim se upiše naziv funkcije, a potom u zagradama slijede parametri, na primjer:

int mnozi(char a, char b); - Deklariramo funkciju *mnozi* da prima dva tipa podatka char, koje smo označili sa a i b, te funkcija vraća kao rezultat podatak tipa int.

2.1 Operand "return"

Operator **return** služi za vraćanje rezultata. Nema potrebe o njemu puno govoriti, biće sve jasno iz primjera.

2.2 Tip podatka "void"

Postoje funkcije kojima kao ulaz nije potreban nikakav parametar, odnosno isto tako postoje funkcije koje ne vraćaju nikakav rezultat (iako to tada više nije funkcija, ali mi ćemo je zbog jednostavnosti i dalje zvati tako). Da bi se takve funkcije mogle deklarirati postoji tip podatka **void**, primjeri deklaracije upotrebom tipa podatka **void**:

void cekaj_tipku(void); - funkcija *cekaj_tipku* je deklarirana tako da, niti prima nikakav parametar, niti bilo šta vraća kao rezultat.

char koja_tipka(void); - funkcija *koja_tipka* vraća podatak tipa **char** u kojem je pohranjena ASCII vrijednost pritisnute tipke na tastaturi, međutim ova funkcija ništa ne prima kao parametar.

2.3 Funkcija main

Pri pokretanju programa, OS mora znati koju će funkciju prvu pokrenuti, za tu svrhu služi funkcija **main**, ona se počinje izvršavati odmah poslije pokretanja programa, drugim riječima svaki C program mora imati funkciju pod nazivom **main**. Za početak će biti dovoljno dobro da funkcija **main** bude deklarirana kao **int main(void)**, tj. da funkcija **main** ništa ne prima, a vraća **integer** kao rezultat, taj rezultat trebao bi govoriti o uspješnosti izvršavanja programa, ako je program uspješno izvršio zadatak, tada, kao rezultat, valja vratiti 0,

u svim ostalim slučajevima, npr. zbog nedostatka memorije i sl. program treba vratiti, kao rezultat, vrijednost različitu od 0, npr. 1.

2.4 Vidljivost varijabli

Varijable mogu biti: **globalne** ili **lokalne**. Globalne varijable se deklariraju izvan tijela neke funkcije i kao takve one su vidljive u svim funkcijama ispod deklaracije, dok se lokalne varijable deklariraju na početku tijela neke funkcije, te su one vidljive samo unutar te funkcije. Dobar stil programiranja kaže da globalne varijable treba izbjegavati kad god je to moguće. Može se dogoditi da u nekoj funkciji deklariramo neku lokalnu varijablu sa istim imenom kao i neku globalnu varijablu, tada će se unutar te funkcije uvijek upotrebljavati samo lokalna varijabla, te globalnoj varijabli ne možemo pristupiti. Dalje, treba napomenuti da se funkcije uvijek deklariraju izvan tijela neke funkcije, te su one uvijek vidljive ispod deklaracije.

3. Jednostavan program

U ovom ću odjeljku prikazati kako napisati jednostavan program u C-u koji će ispisati poruku na ekranu. Program izgleda ovako:

```
#include <stdio.h>
#include <conio.h>

int main(void)
{
    printf("\nMoj prvi c program!!!! ");
    getch();

    return 0;
}
```

Ovaj kod možete direktno kopirati u Borland C, te izabrati naredbu RUN i time pokrenuti program.

Objašnjenje koda:

Prva dva reda **#include <stdio.h>** i **#include <conio.h>** neka vas ne zbunjuju previše. Dovoljno je znati da ako želite upotrebljavati funkciju **printf** morate imati red **#include <stdio.h>** a za funkciju **getch()** morate imati **#include <conio.h>**. Kako znati koji include morate imati za koju funkciju? Jednostavno, odete kursorom ispod željene funkcije i pritisnete CTRL+F1. Otvoriće se help u kom će biti između ostalog navedeno koji include je potreban za upotrebu te funkcije. Sa kombinacijom SHIFT+F1, otvara vam se prozor u kojem su izlistane sve funkcije koje standardno dolaze sa BORLAND C-om..

Red **void main(void)** predstavlja deklaraciju funkcije main, tačka zarez poslije deklaracije se ne postavlja zato jer smo sa otvorenom vijugavom zagradom označili početak tijela funkcije. Tijelo funkcije je kod koji se treba izvršavati kada se ta funkcija pozove. Zatvorenom vijugavom zagradom označavamo kraj tijela funkcije.

Red **printf("\nMoj prvi c program!!!!");** ispisuje poruku na ekranu, poruka se mora nalaziti unutar navodnika, znak **\n** unutar navodnika je specijalni znak koji označava novi red, postoje i drugi specijalni znakovi: **** - backslash, **\b** - back space, **\0** - null znak...

Red **getch();** čeka dok se ne pritisne tipka, skraćenica od (**get_char**), da smo izostavili ovu naredbu tada bi program pri pokretanju odmah izašao iz DOS - prozora, te bi se kontrola vratila na Borland C, pa korisnik nebi stigao vidjeti poruku pri izvršavanju ovog programa iz Borland C-a. Takve situacije se rješavaju sa **getch()** naredbom koja će zaustaviti program prije samog završetka, ili možete pritisnuti ALT+F5, da bi pogledali DOS-prozor.

Vjerovatno ste već i sami uočili da svaki red unutar tijela funkcije **main** završava sa tačka-zarez. To je sintaksa jezika C, znači upamtite da svaki poziv funkcije mora završiti sa točka zarez.

4. Aritmetičke i logičke operacije

Postoje četiri osnovne aritmetičke operacije: množenje, dijeljenje, sabiranje, oduzimanje. Svaka od tih operacija ima svoj simbol: $*$ / $+$ / $-$. Kod logičkih operacija moramo razlikovati **bitwise** logičke operacije i **obične** logičke operacije. Kod običnih logičkih operacija rezultat je uvijek 1 ili 0, tj. true ili false, dok **bitwise** operatori uspoređuju zadane varijable bit po bit. Mi se za sada nećemo baviti **bitwise** operatorima.

Simbol za logički I (AND): **&&**

Simbol za logički ILI (OR): **||**

Najbolje ćete naučiti ako proučite sledeći primjer:

4.1 Jednostavan program koji sabira, dijeli, oduzima i množi

```
#include <stdio.h>
#include <conio.h>

int main(void)
{
    int b1,b2,rez;
    b1 = 100;
    b2 = 10;

    rez = b1+b2;
    printf("Sabiranje: %d + %d = %d\n",b1,b2,rez);

    rez = b1/b2;
    printf("Dijeljenje: %d / %d = %d\n",b1,b2,rez);

    rez = b1-b2;
    printf("Oduzimanje: %d - %d = %d\n",b1,b2,rez);

    rez = b1*b2;
    printf("Mnozenje: %d * %d = %d\n",b1,b2,rez);

    getch();

    return 0;
}
```

Objašnjenje programa:

Ovaj program možete direktno kopirati u Borland C, i pokrenuti ga. Kako i sami vidite za osnovne matematičke operacije nema neke mudrosti, prvo inicijaliziramo početne vrijednosti, **b1 = 100**; dodijeljuje varijabli **b1** vrijednost **100**. U slučaju da nismo varijabli **b1** dodijelili vrijednost **100**, tada bi vrijednosti varijable **b1** bila nepredvidiva, nebi bila 0 kao što možda BASIC programeri pretpostavljaju.

Dalje možete uočiti da se matematika svodi na to da jednostavno napišete matematički izraz i pridružite taj izraz znakom dodjeljivanja (=) nekoj varijabli, npr: **rez = b1+b2**; znači da u varijablu **rez** unesemo zbir od varijable **b1** i **b2**, pošto varijabla **b1** sadrži vrijednost **100**, a varijabla **b2** sadrži vrijednost **10**, vrijednost izraza koja će se dodijeliti varijabli **rez** je **100 + 10 = 110**.

Rad koji vas možda zbunjuje je **printf("Zbrajanje: %d + %d = %d\n",b1,b2,rez);**, ovim redom na ekranu se printa poruka **Sabiranje: 100 + 10 = 110**, proučite prvi parametar funkcije **printf**: **"Sabiranje: %d + %d = %d\n"**, prethodno smo spomenuli da je prvi parametar funkcije **printf** string koji se treba isprintati na ekran,

u slučaju da želimo vrijednosti varijabli printati na ekran moramo se koristiti specijalnim znakovima unutar tog stringa, označavajući na taj način da želimo printati vrijednost varijable, tako npr. specijalni znak **%d** znači da na to mjesto želimo ispisati broj, poslije prvog parametra onda moramo navoditi varijable čije vrijednosti želimo ispisati na mjestima specijalnih znakova.

Svakako vam preporučujem da se sa gornjim programom malo poigrate, tj. mjenjajte inicijalne vrijednosti, radite složenije izraze, mijenjajte string unutar **printf** funkcije itd. Za slaganje složenijih izraza možete se služiti i običnim zagradama, npr:

```
rez = (b1 + b2) * b1;
```

Osim osnovnih aritmetičkih operacija postoji još nekoliko zanimljivih aritmetičkih operatora:

- ++** - operator uvećavanja za jedan, npr **k++**; uvećava varijablu **k** za 1, ekvivalentno tome je **k = k + 1**;
- - operator umanjivanja za jedan, npr **t--**; smanjuje varijablu **t** za 1
- +=** - operator dodavanja, npr **k+=12**; **k** će se uvećati za 12, ekvivalentno tome je **k = k + 12**;
- =** - operator oduzimanja, npr **k-=16**; **k** će se umanjiti za 16, ekvivalentno tome je **k = k - 16**;
- %** - operator ostatka cjelobrojnog dijeljenja (modul), npr: **x = 7%3**; **x** je sada 1;

Ostale operatore potražite u HELP-u od Borlanda pod ključnom riječi **operators**.

4.2 If operator, ispitivanje uslova

Ideja operatora **if** je da provjeri vrijednost nekog izraza, i onda na osnovu toga odluči da li će izvršiti neke naredbe ili ne. Opšti oblik **if** operatora:

if (izraz) naredba;

Ako je vrijednosti izraza 0 (false) tada se naredba neće izvršiti, a ako je izraz bilo koji drugi broj, tada će se naredba izvršiti. Postoje specijalni operatori za provjeru:

- ==** - operator jednakosti
- !=** - operator nejednakosti
- >** - operator veće od
- <** - operator manje od
- >=** - operator veće jednako
- <=** - operator manje jednako
- !** - operator **true** ili **false**

Svi ovi operatori u slučaju **true** vraćaju vrijednost 1, a u slučaju **false** vraćaju vrijednost 0, npr: izraz **10 == 2** vraća vrijednosti 0, zato jer je **false** da je 10 jednako 2. Naravno, umjesto konkretnih brojeva možete staviti i varijable, evo primjera programa u kojem je sve to implementirano:

```
#include <stdio.h>
#include <conio.h>

int main(void)
{
    int b1, b2;

    printf("\n Unesi dva broja(b1 b2):    ");
    scanf("%d %d", &b1, &b2);

    if( b1 > b2) printf("b1 je veci od b2\n");
```

```

if(b1 < b2) printf("b2 je veci od b1\n");

if(b1 == b2)
{
    printf("b1 jednak b2  ");
    b1 = b1 + 10;
    printf("...medjutim sada b1 postaje za 10 veci i iznosi: %d\n",b1);
}

getch();

return 0;

}

```

Objašnjenje programa:

Prvo program ispisuje poruku na ekranu da treba unijeti dva broja. Funkcija koja omogućuje unos varijabli sa tastature je **scanf**. Prvi parametar funkcije **scanf** je string koji sadrži specijalne znakove koji označavaju koji tip podatka želimo unijeti; Gornji primjer unosi dva podatka tipa **int**, a specijalni znak za **int** je **%d**. U gornjem primjeru unosimo dva podatka tipa **int**, pa je zbog toga unutar stringa **scanf** funkcije dva puta naveden specijalni znak za **int**.

Parametri koji slijede iza stringa kod **scanf** funkcije su **pokazivači** na mjesta u memoriji gdje želimo pohraniti vrijednosti unesene sa tastature. Do pokazivača na željenu varijablu dolazimo sa znakom **&** ispred naziva varijable. Nemojte se sada puno zamarati sa pokazivačima, o njima ću kasnije više govoriti.

Svakako se sa gornjim primjerom poigrajte mijenjajući uslove unutar **if** operatora. Uočite da kada želite da **if** operator izvrši samo jednu naredbu tada nisu potrebne vijugave zagrade iza njega, međutim, ako **if** operator treba izvršiti više naredbi, tada obavezno morate sve te naredbe staviti unutar vijugavih zagrada.

4.3 Upotreba logičkih operatora

Kada želite staviti više uslova unutar jednog **if** operatora, tada možete koristiti **logičke operatore**. Npr.: imamo deklarisanu varijablu X, i želimo je pomnožiti sa dva ako ona iznosi 2 ili 4 ili 5. Taj uslov bi napravili ovako:

```
if(X == 2 || X == 4 || X == 5) X *= 2;
```

Uočite unutar zagrada **if** operatora logički operator **ILI (||)**. Gornji izraz znači:

Ako je X jednak 2 ili je X jednak 4 ili je X jednak 5, tada pomnoži x sa 2.

U sledećem primjeru pretpostavimo da imamo dvije varijable X i Y. Varijabli X želimo dati vrijednost varijable Y samo ako je X > 10 a Y <= 10, ovaj uslov izveli bi sa logičkim operatorom **I (&&)**:

```
if(X > 10 && Y <= 10) X = X + Y;
```

Kad bi gornji izraz prevodili na ljudski jezik tada bi prevod glasio: **Ako je X veći od 10, i ako je Y manji ili jednak 10, tada dodaj X-u Y;**

Za kraj treba ponoviti dva osnovna logička operatora: **I (&&), ILI (||)**

5. Petlje

Ideja petlje je da omogućiti izvršavanje tijela petlje sve dok se neki uslov ne ispuni potreban za izlazak iz petlje. Postoje 2 osnovne vrste petlji: **for** i **do...while**.

5.1 "For" petlja

Opšti oblik **for** petlje:

```
for(inicijalni dio; uslov; korak)
```

```
{
    naredbe;
}
```

Primjer programa sa **for** petljom:

```
#include <stdio.h>
#include <conio.h>

int main(void)
{
    int i;
    for(i=0; i<5; i++)
    {
        printf("\nTrenutna vrijednost: %d", i);
    }
    getch();

    return 0;
}
```

U gornjem programu tijelo petlje će se izvršiti 5 puta, sa vrijednostima varijable ***i* = 0,1,2,3,4**. Tijelo petlje je kod koji se nalazi unutar vijugavih zagrada **for** petlje, u gornjem slučaju to je jedna **printf** funkcija koja ispisuje trenutnu vrijednost varijable ***i***.

Šta se tačno događa kod petlje? Kada program uđe u gornju petlju prvo će postaviti varijablu ***i*** na vrijednost 0, zatim će se provjeriti da li je uslov ***i*<5** istinit, ako je uslov istinit izvršiće se tijelo petlje, odn. na ekranu će biti ispisana poruka: *Trenutna vrijednost: 0*, a ako uslov nije istinit tada se prekida izvršavanje petlje, pa se program nastavlja izvršavati od zatvorene vijugave zagrade. Poslje izvršenja tijela petlje, izvršiće se izraz ***i++***. Taj izraz povećava varijablu ***i*** za 1, nakon toga izvršavanje se nastavlja od provjere istinitosti uslova ***i*<5**, ako je uslov istinit, onda će se u sledećem izvršavanju tijela petlje na ekranu ispisati: *Trenutna vrijednost: 1*. itd...

Rezime: Svrha **for** petlje je omogućavanje da se neki dio koda višestruko izvrši.

5.2 "do...while" petlja

do...while se upotrebljava kada se treba neki kod izvršavati sve dok uslov unutar zagrada ključne riječi **while()** ne postane istinit, npr. program treba ispitivati koja je tipka pritisnuta sve dok pritisnuta tipka ne bude tipka 'q'. Opšti oblik **do...while** petlje:

```
do
{
    naredbe;
}while(uslov);
```

Kratko objašnjenje **do...while** petlje: Tijelo petlje će se izvršavati sve dok uslov unutar zagrada ključne riječi **while()** ne postane istinit.

Primjer programa sa **do...while** petljom:

```
#include <stdio.h>
#include <conio.h>

int main(void)
{
```

```

char c;
do
{
printf("\nCekam tipku...");
c = getch();

}while(c != 'q');

printf("\nIzasao sam");
getch();

return 0;
}

```

Objašnjenje gornjeg programa:

Prvo deklariramo varijablu **c** kao tip **char**. Onda program ulazi u **do while** petlju i ispisuje poruku "*Cekam tipku...*", poslije toga program ulazi u funkciju **getch()**, za koju od maloprije znamo da čeka pritisak tipke, međutim, ta funkcija osim što čeka pritisak tipke, ona, takođe, vraća kod pritisnute tipke, (pogledajte help), taj kod se dodjeljuje varijabli **c**, pa se onda prelazi na ispitivanje uslova unutar **while** operatora, uslov unutar **while** operatora kaže: *da li je c različit od slova 'q'*, ako je uslov istinit, tada će se izvršavanje programa nastaviti od ključne riječi **do**, tj, ponovno će se ispisati poruka "*Cekam tipku...*", itd. sve dok se ne pritisne tipka **q**, tada će uslov unutar zagrada **while** postati neistinit (**false**), pa će se prekinuti izvršavanje petlje.

Uočite da se slovo **q** nalazi unutar apostrofa ('**q**'). Naime, operatori apostrofa vraćaju ASCII vrijednost znaka unutar apostrofa, da smo kojim slučajem izostavili apostrofe, tad bi compiler **q** smatrao kao neku varijablu, te bi prijavio grešku da varijabla **q** nije definisana..

5.3 Break i continue

Ove naredbe djeluju samo unutar **for**, **do...while()** i **while()** petlje. Naredba **break** prekida izvršavanje petlje, a naredba **continue**, nastavlja izvršavanje petlje sa sledećom veličinom, npr:

```

do{
znak = getch();
if (znak == 'q') break;          /*ako je pritisnuta tipka q tada izađi iz petlje*/
if (znak < '0' || znak > '9') continue; /*ako pritisnuta tipka nije broj tada
uzmi sledeći znak (nastavak petlje) */

rez += (znak - '0'); /*povećaj rez za vrijednost unešenog broja.*/

}while(1); /*beskonačna petlja*/

```

Sa završetkom ovog poglavlja, smatram da ste dobili dovoljno znanja za rješavanje nekih jednostavnih zadataka, osim što će se kod svakog zadatka definisati problem (ono što program mora raditi), navešću i ukratko kako bi se taj program mogao riješiti. Takođe vam predlažem, da prije nego što pročitate kratka uputstva, da se pokušate sami sjetiti kako riješiti zadatak. Naravno, vaše rješenje zadatka možda traži veći broj varijabli ili ne koristi funkcije koje samo naveo u uputstvu, itd..., glavno da program radi ono što traži zadatak.

Predlažem vam da zadatak zaista samostalno pokušate riješiti, a za slučaj da ne uspijete stavio sam na kraju tutorijala link ka stranici gdje se nalaze rješenja svih zadataka iz tutorijala.

Zadatak 1: Napravite program koji će unijeti 10 brojeva sa tastature, pa će na kraju ispisati prosječnu vrijednost unesenih brojeva.

Za rješenje ovog zadatka moraćete upotrijebiti **for** petlju, **scanf** funkciju za unos brojeva, dvije pomoćne varijable, varijablu u kojoj ćete čuvati ukupni zbir i **printf** funkciju za ispis rezultata.

Zadatak 2: Napravite program koji će brojati koliko je korisnik puta pritisnuo tipku na tastaturi, sve dok ne klikne na tipku **q**, te na kraju ispišite broj pritisaka.

Za rješenje ovog zadatka treba vam: **do...while** petlja, **getch()** funkcija, dvije varijable (jedna pomoćna, jedna u kojoj ćete brojati, te vam treba **printf** funkcija za ispis rezultata).

6. Nizovi

Razmislimo kako bi mogli napraviti program koji bi pamtio unešene brojeve, te na kraju te brojeve ispisao obrnutim redoslijedom. Ovakav problem rješavao bi se pomoću niza. Pod pojmom niza podrazumijeva se deklaracija neke varijable tako da ta varijabla ima više članova. Npr. deklariramo niz od 10 **int** varijabli:

int n[10]; - ovim redom smo deklarirali niz od 10 varijabli tipa **int**. Znači niz se deklarira tako da se pored oznake tipa podatka navede naziv niza, te u srednjim zagradama navedemo broj članova tog niza.

Članovima niza možemo pristupiti pomoću srednjih zagrada, npr. ako želimo u 3. član niza nešto upisati, tada bi to izgledalo ovako: **n[2] = 100;** Broj 2 unutar srednje zagrade nije greška, već svojstvo u C-u da svi nizovi počinju od indexa nula, tj index 0 označava 1. član, index 1 označava 2. član, index 2 označava 3. član itd... To znači da ako deklariramo niz kao **int n[10];**, onda govorimo compileru da za niz **n** predvidi 10 mjesta, pa će stoga maksimalni legalni index biti 9, tj **n[9]** je zadnji član niza. Zašto sam napisao "legalni", zato jer je moguće pristupiti i npr. 153. članu tog niza, ali onda compiler više ne garantuje da će te pristupati alociranoj memoriji, te će se u tom slučaju program možda srušiti. Compiler u ovom slučaju jedino garantuje za indexe od 0 do 9. QBASIC bi pri pristupnju 153 člana prijavio grešku, ali C ništa ne prijavljuje, već uredno obavlja svoj posao "pa kud puklo da puklo", zbog ovakvih stvari se programskom jeziku C povećava fleksibilnost, ali se smanjuje jednostavnost.

Primjer zadatka sa nizom **int**-ova, ovaj zadatak će unijeti 10 brojeva te će ih na kraju ispisati u obrnutom redoslijedu.

```
#include <stdio.h>
#include <conio.h>

int main(void)
{
    int i,b[10];

    for(i=0;i<10;i++)
    {
        printf("\nUnesi broj %d ",i);
        scanf("%d",&b[i]);
    }

    printf("\nIspis u obrnutom redoslijedu: \n");

    for(i=9;i>=0;i--)
    {
        printf("\nUnesi broj %d ",b[i]);
    }
    getch();

    return 0;
}
```

Objašnjenje programa:

Prvo pogledajmo deklaraciju varijabli: **int i,b[10];** -deklarišu se varijabla **i**, ovo je pomoćna varijabla za **for** petlju, te se deklariše niz od 10 **int**-ova pod nazivom **b**. U prvoj **for** petlji se unosi 10 brojeva i slažu se redom u **b[0], b[1], b[2]...b[9]**. U drugoj **for** petlji ispisuju se onda brojevi **b[9], b[8]....b[0]**. Uočite, u drugoj **for** petlji, način na koji je ta petlja inicijalizirana da bi brojala od 9 do 0.

Zadatak 3: Napravite program koji će unositi brojeve sa tastature sve dok se ne unese broj 0, tada sve unesene brojeve ispišite obrnutim redoslijedom. Predvidite maksimalno 100 unesenih brojeva. Uputstvo za rješenje: Između ostalog trebaće vam niz od 100 **int**-ova, **do...while** petlja, **for** petlja.

7. Pokazivači ili pointeri

Pokazivači su tipovi podataka koji pokazuju na neku varijablu, ono o čemu pokazivači fizički vode računa je memorijska adresa podatka na koji pokazuju, to znači da su svi pokazivači jednake dužine, u DOS-u su svi pokazivači 16 bitni, međutim razlikuju se po tome na koju vrstu podataka pokazuju. Na primjer, možete deklarirati pokazivač koji pokazuje na **int**-ove, možete deklarirati pokazivač koji pokazuje na **char**-ove itd...

Deklaracija pokazivača vrši se zvjezdicom ispred naziva pokazivača, npr;

```
int *pi;      /*varijabla pi je deklarirana kao pokazivač na int*/
float *mh     /*varijabla mh je deklarirana kao pokazivač na float*/
```

Svrha pokazivača je da bi se podaci između funkcija mogli prenositi preko pokazivača, tj. umjesto da se pošalju nekoj funkciji sve vrijednosti nekog niza, jednostavno se pošalje samo pokazivač na početni član toga niza.

Takođe možete pristupiti vrijednosti na koji pointer pokazuje, to se opet čini sa operatorom zvjezdice, npr: ako je deklariran pokazivač **int *k**; tada možemo na mjestu gdje taj pointer pokazuje upisati vrijednost 100 na sledeći način: ***k = 100**;

Operatorom **&** ispred neke varijable dolazimo do adrese gdje se ta varijabla u memoriji nalazi. Upotrebom ovog operatora možemo neki pokazivač usmjeriti da pokazuje na neku tačno određenu varijablu.

Primjer upotrebe pokazivača:

```
#include <stdio.h>
#include <conio.h>

int main(void)
{
    int x,*p;
    p = &x; /*preusmjeri p tako da pokazuje na x*/

    printf("\nUnesi broj ");
    scanf("%d",&x);

    printf("\nUneseni broj je: %d",*p);

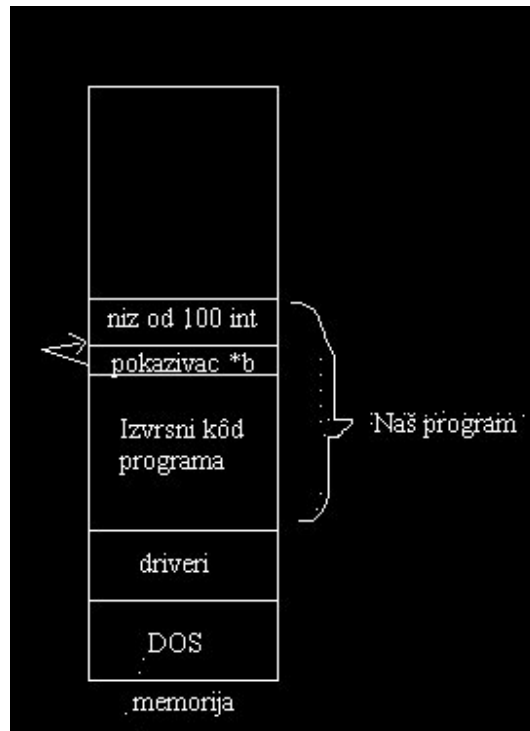
    getch();

    return 0;
}
```

Ono što je bitno uočiti kod ovog programa je da na kraju kada se ispisuje rezultat ne pristupa se unesenom broju preko varijable **x**, već preko pokazivača **p** koji je prethodno preusmjeren da pokazuje na varijablu **x**.

7.1 Pokazivači i nizovi

Pokazivači i nizovi su u C-u usko povezani, zato jer je svaki niz zapravo pokazivač na alociranu memoriju, alocirana memorija znači da je to dio memorije koji je operativni sistem dodijelio našem programu, te operativni sistem garantuje da tu memoriju neće dodijeliti niti jednom drugom programu. Znači, kod deklaracije niza **int b[100]**; OS alocira 100 mjesta za **int** varijablu, te pokazivač **b** preusmjeri da pokazuje na 0. član tog niza. Slikovito bi to izgledalo ovako:



Na datoj slici vidimo da pokazivač ***b** pokazuje na 0. član niza od 100 **int**-ova. Kod deklaracije **int b[100]**; nama nije moguće mijenjati mjesto na koje **b** pokazuje, to, se zove statički alocirana memorija koja se uvijek nalazi na istom mjestu i stalne je veličine. Međutim, postoji druga stvar koju možemo napraviti, a to je da neki drugi pokazivač preusmjerimo riječima na 20. član od niza **b**, to bi izgledalo ovako: **int b[100], *p; p = &b[20];** Onda će **p[0]** imati jednaku vrijednost kao **b[20]**. Prikažimo to programom:

```
#include <stdio.h>
#include <conio.h>

int main(void)
{
    int i, b[20], *p;
    p = &b[5];

    for(i=0; i<20; i++)
    {
        b[i] = i;
    }

    printf("\nIspis niza p i niza b");
```

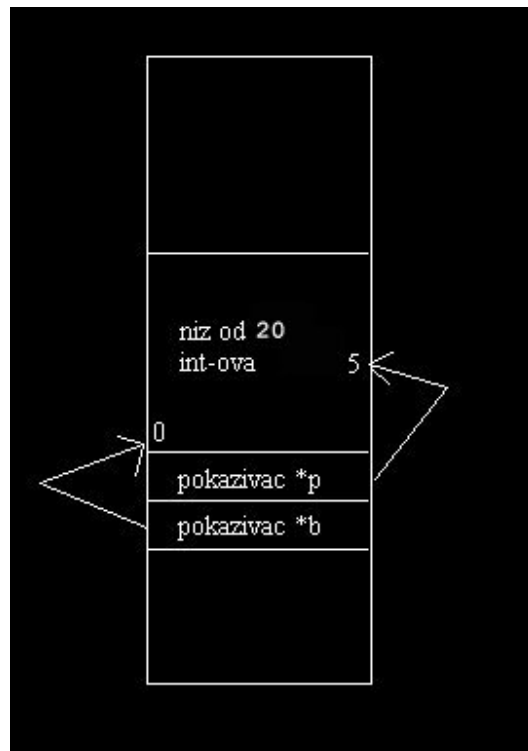
```

for(i=0;i<20;i++)
{
printf("\nb[%2d] = %2d, p[%2d] = %2d",i,b[i],i,p[i]);
}
getch();

return 0;
}

```

Svakako ovaj program kompajlirajte i pokrenite, te proučite izlaz na ekranu. Na početku se deklariraju varijable *i*, niz *b* od 20 *int*-ova, pokazivač na *int* **p*. U sledećem koraku *p* se preusmjeri da pokazuje na 6. član niza *b*. To se naravno postiže operatorom *&*. U prvoj *for* petlji inicijalizira se niz, tako da se u svaki član niza upiše vrijednost njegovog indexa, tj. *b*[0] će imati vrijednost 0, *b*[1] imaće vrijednost 1,... U drugoj *for* petlji ispisuju se oba niza od indeksa 0 do indeksa 20, pošto *p* pokazuje na 6, član od niza *b*, a 6. član niza *b* sadrži vrijednost 5, pa će se zbog toga za *p*[0] ispisati 5. Slikovito bi to otprilike ovako izledalo



Ako pokušamo prisutpiti članu *p*[20], a znamo da je *p* = *&b*[5], tada možemo zaključiti da je *p*[20] = *b*[25], a u deklaraciji smo predvidjeli samo 20 članova u nizu *b*, pa će onda sigurno *p*[20] pokazivati na nešto nealocirano, odnosno *p*[20] će sadržavati nepredvidivu vrijednost, a to se vidi ako se pokrene gornji program. Nemojte se plašiti da ćete tako srušiti DOS, naime, pri operaciji čitanja pod DOS-om nema straha, problemi bi mogli nastati kada bi počeli pisati po tuđoj memoriji.

Uočite da u *printf* funkciji kao specijalni znak nije upotrebljen *%d* već *%2d*, razilka je u tome što *%2d* predviđa uvijek 2 mjesta za ispis tog broja, *%3d*, predviđalo bi 3. mjesta itd...

7.2 Dinamičko alociranje memorije

Pojam dinamičkog alociranja memorije znači da program za neki niz alocira onoliko memorije koliko mu je potrebno, na primjer, ako želimo neku tekstualnu datoteku učitati u memoriju, tada ćemo prvo zatražiti od DOS-a informaciju o tome koliko je ta datoteka dugačka, te ćemo na osnovu toga alocirati potreban broj bita, i tek ćemo onda tu datoteku učitati u alociranu memoriju.

Funkcija koja služi za alociranje memorije: **malloc** - ova funkcija prima kao parametar broj bita koliko želimo alocirati, a kao rezultat vraća pokazivač na nulti bit alocirane memorije. Nakon što nam alocirana memorije više ne treba, možemo je osloboditi funkcijom **free**. Funkcija **free** kao parametar prima pokazivač na memoriju koju želimo osloboditi (deallocirati).

Napišimo program koji će prvo zatražiti broj brojeva koliko ih želimo unijeti, tada će program alocirati memoriju, u koju će onda unijeti zadani broj brojeva. Na ekranu, program mora ispisati brojeve tako da su svaka dva susjedna člana zamijenjena, npr. član 1 i član 2 su zamijenjeni itd...

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>

int main(void)
{
    int temp,i,*b,brmax;

    printf("\nUnesi maksimalan broj brojeva: ");
    scanf("%d",&brmax);

    b = (int *) malloc(brmax * sizeof(int));    /*alokacija*/

    if(b == NULL)                             /*da li je alokacija bila
    uspjesna*/
    {
        printf("\nNedovoljno memorije!");
        getch();
        return;                                /*izlaz iz programa*/
    }

    for(i = 0; i<brmax;i++)
    {
        printf("\nUnesi broj %d: ",i);
        scanf("%d",&b[i]);
    }

    for(i=1;i<brmax;i+=2)
    {
        temp = b[i];
        b[i] = b[i-1];
        b[i-1] = temp;
    }

    printf("\nRezultat:\n");
    for(i=0;i<brmax;i++)
    {
        printf("\n b[%2d]: %5d",i,b[i]);
    }

    free(b);                                  /*oslobodi memoriju*/
    getch();

    return 0;
}
```

Gornji program ne donosi ništa posebno novo jedino, što se ovdje dinamički alocira potrebna memorija za unešeni niz, pa ću jedino red **`b = (int *) malloc(brmax * sizeof(int));`** opširnije objasniti. Funkcija **`malloc`** kao parametar prima broj bitova koliko želimo alocirati, mi želimo alocirati **`brmax`** integera, da bi saznali koliko bita zauzima jedan integer, koristimo se operatorom **`sizeof`**. U njegove zagrade stavimo tip podatka od kojeg želimo saznati njegovu veličinu u bitima, odnosno u gornjem slučaju stavimo **`sizeof(int)`**. Sada veličinu **`sizeof(int)`** pomnožimo sa **`brmax`**, i onda smo dobili potreban broj bita za **`brmax`** integera. Funkcija **`malloc`** će vratiti kao rezultat pokazivač **`void *`**. Pokazivač **`void *`** ne pokazuje niti na jedan tip podatka, on je jednostavno pokazivač na neku memorijsku lokaciju. Da bi **`void *`** pokazivač pretvorili u **`int*`** pokazivač potrebno je staviti u zagrade poslije znaka dodjeljivanja tip podatka u koji želimo **`void *`** prevesti. Ako funkcija **`malloc`** nije uspjela (alocirati potreban broj bita), tada kao rezultat vraća **`NULL`** pointer. Uočite da u gornjem primjeru imamo jedan novi include: **`#include <stdlib.h>`**, to je zbog funkcija **`malloc`** i **`free`**.

Na kraju programa oslobodimo alociranu memoriju pomoću funkcije **`free`**.

Napomena: Nad pointerima možete vršiti isto aritmetičke operacije, tj možete uvećavati pokazivač da pokazuje npr. na sledeću memorijsku lokaciju. Tako npr. dobili bi isti rezultat kada bi pristupali **`p[4]`** ili **`*(p+4)`**. Zašto? Zato, jer **`p[4]`** pokazuje za četiri mjesta: udaljeniju lokaciju od **`p[0]`**, **`p == &p[0]`**, pa uvećavanjem **`p`** za 4 možemo doći opet do istog mjesta. U bitovim gledano, pointeri na **`int`** kod uvećanja za jedan u stvari se uveća za 2 bita, zato jer je **`int`** dužine dva bita.

<code>p[]</code>	 0 	1 	2 	3 	4 	5 	6
<code>p</code>	+0	+1	+2	+3	+4	+5	+6

8. Stringovi

Stringovi su u C-u nizovi **`char`**-ova gdje zadnji karakter u tom nizu ima vrijednost 0. Ako nebi imao vrijednosti 0, tada se nebi mogla odrediti dužina tog stringa. Znači zapamtite da svaki string završava sa vrijednošću 0 '\0'. Stringu se naravno pristupa preko pokazivača na **`char`**. Naravno, možemo definisati u kodu programa stringove. To bi deklarirali na sledeći način:

```
char *str = "Ovo je neki string";
```

Šta se zapravo događa? Kod ovakve deklaracije, compiler napravi pokazivač na **`char *str`**, zatim pogleda dužinu stringa **`"Ovo je neki string"`**, te alocira potrebnu dužinu znakova + 1. Dodatni znak služi za **`null`** znak, koji dolazi na kraj tog niza karaktera. Kada je compiler alocirao potreban broj bita, tada smjesti tekst stringa u taj alocirani dio memorije, i na kraju preusmjeri pokazivač **`*str`** da pokazuje na nulti član alociranog niza.

Unos stringova sa tastature.

Za unos stringa sa tastature postoje dva načina, **`gets()`** funkcija i **`scanf()`** funkcija. Nedostatak **`scanf()`** funkcije je da onemogućuje unos stringa koji sadrži razmake, te kod razmaka prekida unos stringa, **`gets()`** funkcija nema taj nedostatak. Znači upamtite, da kada želite unositi stringove koji će možda sadržavati razmake, tada obavezno morate koristiti funkciju **`gets()`**.

Sledeće na što morate obratiti pažnju je da kad koristite ove funkcije morate pretpostaviti maksimalnu dužinu unešenog stringa, a ako slučajno korisnik unese duži string od predviđenog, tada postoji mogućnost da se sruši OS. Predviđanje maksimalne dužine unesenog stringa, znači da alocirate određeni broj bita memorije (npr. 150), te uneseni string smjestite u tu alociranu memoriju. Ako bi korisnik slučajno unio više od 149 znakova, tada bi se moglo nešto nepredvidivo dogoditi. Primjer unosa stringa te njegovog ispisa:

```
#include <stdio.h>
#include <conio.h>
```

```
int main(void)
{
    char p[150];
```

```
printf("\n\nUnesi string: ");
gets(p);

printf("\nUneseni string je: %s",p);

getch();

return 0;
}
```

Prvo što program napravi je da statički alocira 150 char-ova (==150 bita), to je red **char p[150]**; Poslije alokacije ispiše se poruka na ekranu za unos stringa, te se izvršavanjem funkcije **gets** prelazi na unos stringa, funkcija **gets** prima kao parametar **char** pokazivač na mjesto u memoriji gdje želimo unijeti string. Poslije unosa stringa, taj isti string ponovno ispisujemo sa **printf** funkcijom i ključnim znakom **%s**. Naravno možemo i dinamički alocirati string sa funkcijom **malloc**, onda bi **char p[150]**; zamijenili sa **char *p;**
p = (char *) malloc(150);

Unos stringa sa funkcijom **scanf** izgleda ovako:
scanf("%s",p);

Uočite da sada kod **scanf** funkcije nismo koristili **&p**, već samo **p**, to je zato jer funkcija **scanf** prima pokazivače na mjesta u memorije gdje želimo pohraniti unešenu veličinu, a pošto je **p** pokazivač tada nebi imalo smisla stavljati **&p**. Međutim umjesto **p** mogli smo staviti **&p[0]**, znači pokazivač na nulti član niza **p**.

8.1 Određivanje dužine stringa

Za operacije nad stringovima postoji mnoštvo standardnih funkcija, pogledajte u helpu pod ključnom riječi string.**h**, međutim mi ćemo za vježbu napraviti funkciju koja određuje dužinu stringa, prisjetimo se prvo da svaki string završava sa vrijednošću 0, onda se prisjetimo da je **do...while** petlja ko stvorena za pretragu:

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>

int dulj_str(char *p);          /*deklaracija funkcije duz_str, prima kao
parametar pokazivač na char, a vraća int*/

int main(void)
{
    char *s;
    int duzina;

    s = (char *) malloc(1000);

    printf("\n Unesi string: ");
    gets(s);

    duzina = duz_str(s);        /*poziv funkcije duz_str*/
    printf("\nDuzina unesenog stringa je %d",duzina);

    getch();
}
```

```

return 0;
}

int duz_str(char *p)
{
int rez;
rez = 0;
do{
/*radi*/
rez ++;
/*povecaj duzinu*/
}while(p[rez] != 0);
/*da li smo dosli do null znaka, ako ne
nastavi*/

return rez;
}

```

Prvo razmislimo da li će ovaj program davati uvijek tačan rezultat, odgovor je negativan. Naime funkcija **duz_str** neće dati tačan rezultat ako je string dužine 0 (stručni naziv za string dužine 0 je '**null - string**'). Da bi to ispravili postoji jedan brzi i jednostavni način, možete u inicijalizaciji funkcije **duz_str** postaviti da **rez** bude -1. Drugi način bio bi upotreba **while()** petlje umjesto **do...while()**.

Zadatak 4: Napravite program koji će unijeti string, te u memoriji generisati novi string, koji je obrnut od unešenog. Na kraju će ispisati oba stringa, npr: Unošeno: *Zemlja*, izlaz: *Zemlja aljmeZ*.

Uputstvo za rješenje zadatka: Kao prvo, trebaće vam neka funkcija za saznavanje dužine stringa, možete napisati svoju vlastitu, a možete koristiti i **strlen()**, koja se nalazi u datoteci **string.h**. Nadalje ćete morati alocirati memoriju za novi string sa funkcijom **malloc**, te okretanje stringa ćete obaviti u **for** petlji.

9. Višedimenzionalni nizovi i višestruki pokazivači

Kada želimo neke podatke čuvati u obliku neke tabele, tada je potrebno neku varijablu deklarirati kao višedimenzionalni niz:

int s[50][20]; - deklaracija dvodimenzionalnog niza sa 50 * 20 integera
char j[50][20][100]; - deklaracija trodimenzionalnog niza sa 50 * 20 * 100 karaktera

Kao što vidite iz gornjih primjera, višedimenzionalni niz se deklarira tako da se dodaju srednje zagrade, gdje broj srednjih zagrada određuje dimenziju tog niza, a unutar zagrada treba upisati broj elemenata u toj dimenziji. Pristupanje nekom članu nekog elementa vrši se na sledeći način:

s[5][9] = 100; /*element 5,9 poprima vrijednost 100*/

Varijabla **s** iz gornjeg primjera je dvostruki pokazivač na **int**: **int **s**; a varijabla **j** je trostruki pokazivač na **char**: **char ***j**;

Niz **int *s[50]**; je niz pokazivača na **int**, a niz **char j **[50]**; je niz dvostrukih pokazivača na **char**. Kako što vidite ovdje smo unijeli novi pojam: dvostruki pokazivač ili pokazivač na pokazivač.

Pokazivač na pokazivač zapravo nije ništa drugo nego običan pokazivač na mjesto u memoriji, s razlikom da ne pokazuje na neku varijablu, nego pokazuje na pokazivač. Deklariše se s dvije zvjezdice ispred naziva pokazivača, npr: **char **pp**;. Pokazivač na pokazivač se koristi kod dvodimenzionalnih nizova, npr. ako želimo u memoriju na zgodan način čuvati neku tabelu brojeva sa 20 kolona i 50 redova, tada postoje dva načina da deklariramo datu tabelu: **statički** i **dinamički**. U slučaju kada imamo višedimenzionalne nizove tada se statička i dinamička alokacija razlikuju. Kod dinamičke alokacije tabela će zauzimati više memorije nego statički alocirana tabela, razlog je u tome da kod statičke alokacije compiler alocira jedan komad memorije koji on onda sam pretvara u dvije dimenzije. Statička alokacija se vrši tako da se veličina niza označi odmah pri deklaraciji datog niza: npr: **int s[50][20]**; Konkretno, za tabelu od 20 * 50 integera

compiler bi alocirao $20 * 50 * 2\text{bita} = 2000\text{bita}$. Tako bi prvih $50 * 2\text{bita}$ predstavljalo mjesta nulte kolone, drugih $50 * 2\text{bita}$ predstavljalo bi mjesta prve kolone itd. Zašto sve ovo množimo sa dva bita, zato jer je pod DOS-om veličina integer 2 bita (16 bita).

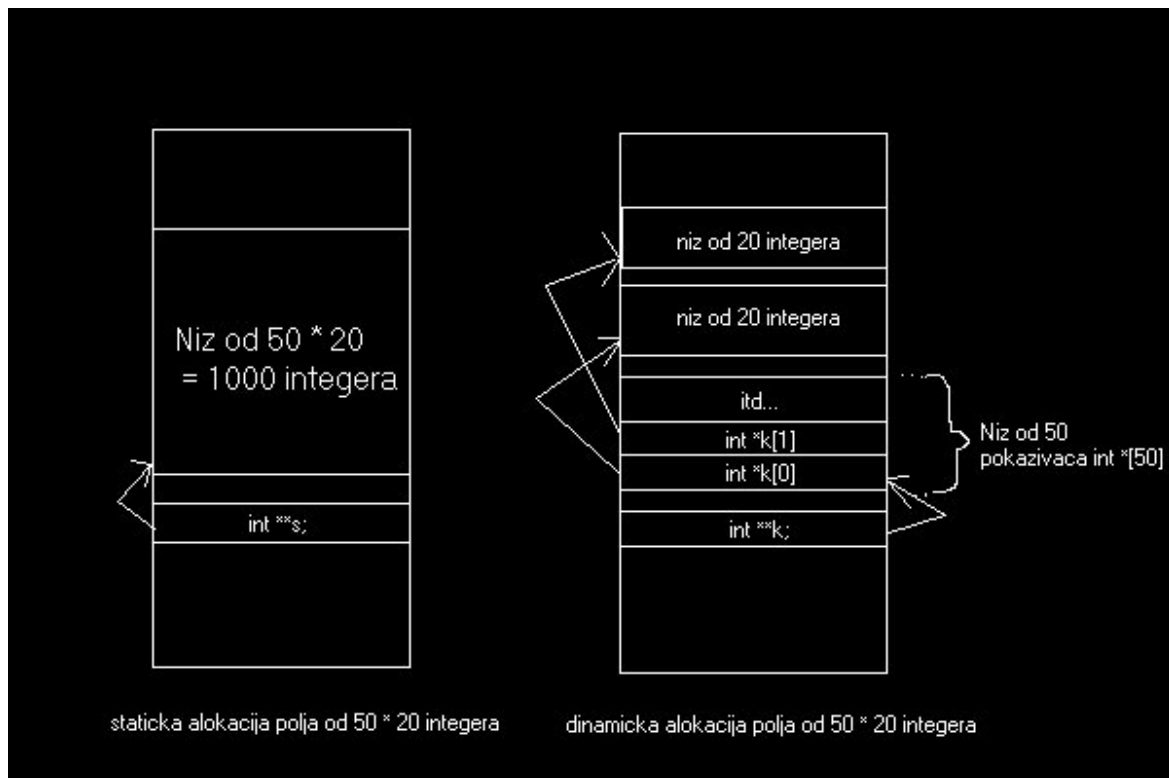
Prije nego što odredimo koliko bi dinamička alokacija ovakvog niza zauzimala memorije, potrebno je objasniti kako ovaj niz dinamički alocirati: Prvo moramo deklarirati neku varijablu kao dvostruki pokazivač na *int*: *int **k*; Sada moramo alocirati memoriju za 50 pokazivača na *int*, te ***k* preusmjeriti da pokazuje na tu memoriju:

*k = (**int) malloc(50*sizeof(int));* Unutar zagrada *malloc* funkcije nalazi se izraz koji izračunava potreban broj bita za 50 pokazivača na *int*. Sada moramo alocirati 50 puta po 20 *int* varijabli, te svaki taj alocirani blok preusmjeriti da na njega pokazuje jedan od pokazivača iz niza pokazivača **k[50]*; To ćemo napraviti *for* petljom, konačan izgled koda u kojem se dinamički alocira dvodimenzionalni niz integera glasi:

```
int i, **k;
k = (int **) malloc(50 * sizeof(int *));
for(i=0; i<50; i++)
{
    k[i] = (int *) malloc(20 * sizeof(int)); /*Unutar for petlje se za svaki
pokazivac alocira po 20 int-eva*/
}
```

Sada možemo izračunati koliko dinamička alokacija niza $50 * 20$ integera zauzima bita memorije. Prvo svaki pokazivač pod DOS-om je veličine 2bita, mi smo alocirali 50 pokazivača = 100bita, plus $50 * 20 * 2\text{bita} = 2100\text{bita}$.

Slikovito statička i dinamička alokacija izgledaju ovako:



Vjerodostojnost gornje slike možete provjeriti tako da ispisujete sa funkcijom *printf* uz pomoć specijalnog znaka *%p*, memorijsku lokaciju na koju pokazivač pokazuje. Kod statičke alokaciju pri ispisu *s* i *&s[0]*, dobićete jednaku vrijednost, dok ćete kod dinamičke alokacije pri ispisu *k* i *&k[0]* dobiti različite rezultate.

Napišimo program koji će statički alocirati mjesta za tabelu od 4*5 integera, a sa tastature će unijeti u tabelu prva 3*4 člana, te će onda na rubnim mjestima tabele upisati zbir sva prethodna 4 odn. 3 elementa. Vidi primjer tabelle:

3	6	16	2	27
8	100	3	9	120
16	3	19	2	40
27	109	38	13	

Primjer tabelle

Source programa:

```
#include <stdio.h>
#include <conio.h>

int main(void)
{
    int tabela[4][5],i,t;
    for(i=0;i<4;i++)
    {
        for(t=0;t<5;t++)
        {
            if(i==3 || t==4)
            {
                tabela[i][t] = 0; /*ako i,t pokazuju na rubno mjesto
                                   tada mjesto i,t postavi u nulu*/
                continue;        /* nastavi sa sledecim mjestom*/
            }
            printf("\nUnesi clan %d %d: ",i,t);
            scanf("%d",&tabela[i][t]);          /*Unos vrijednosti na mjesto i,t*/
        }
    }

    /*u sledecoj dvostrukoj for petlji racunaju se rubna mjesta */

    for(i=0;i<3;i++)
    {
        for(t=0;t<4;t++)
        {
            tabela[ 3][ t] += tabela[ i][ t]; /*u krajnjem redu u trenutnoj koloni dodaj
            trenutnu vrijednost */
            tabela[ i][ 4] += tabela[ i][ t]; /*u skroz desnoj koloni u aktivnom redu
            dodaj trenutnu vrijednost */
        }
    }
}
```

```

    }
}

/*u sledecoj dvostrukoj for petlji ispusuje se ukupna tabela */

for(i=0;i<4;i++)
{
    printf("\n");    /*ako je novi red pređi u novi red*/
    for(t=0;t<5;t++)
    {
        printf("%4d",tabela[i][t]);
    }
}

getch();

return 0;
}

```

U gornjem primjeru smo koristili statičku alokaciju. Svakako se gornjim programom poigrajte, te mu mijenjajte dimenzije itd., itd.

Sada ćemo napisati program koji će prvo unijeti broj stringova koje korisnik želi unijeti, tad će program unositi string po string, i za svaki string alocirati tačno onoliko mjesta koliko je za taj string potrebno. Znači alokacija će se izvesti dinamički. Na kraju program treba sve stringove obrnutim redoslijedom ispisati na ekran.:

```

#include <stdio.h>
#include <conio.h>
#include <string.h>
#include <stdlib.h>

int main(void)
{
    char **str, privstr[250]; /*deklaracija dvostrukog pokazivaca **str
                                i privremeng stringa za unos */
    int brStr,i,len;
    printf("\nUnesi broj stringova: ");
    scanf("%d",&brStr); /* u varijablu brStr pohranjuje se broj stringova*/

    str = (char **) malloc(brStr * sizeof(char *)); /* alokacija od brStr
                                                        pokazivaca na char*/

    for(i=0;i<brStr;i++)
    {
        printf("\nUnesi %d. string: ",i+1);
        scanf("%s",privstr);
        len = strlen(privstr); /*određivanje dužine unesenog stringa*/

        str[i] = (char *) malloc(len + 1); /*alokacija potrebnog broja
                                                bita za uneseni string */
        strcpy(str[i],privstr); /*kopiranje privremenog stringa u memoriju
                                    alociranu za uneseni string */
    }
}

```

```

for(i=brStr-1;i>=0;i--)
{
    printf("\n%s",str[i]); /*ispis*/
}

getch();

return 0;
}

```

Zadatak 5: Napravite program koji će prvo unijeti veličinu kvadratne tabele brojeva, tada će program dinamički alocirati, datu tabelu i unijeti brojeve u svaki član tabele, na ekranu će program ispisati sve članove tabele po dijagonalama. Npr:

```

veličina: 3
unesi 1 1: 1
unesi 1 2: 2
unesi 1 3: 3
unesi 2 1: 4
unesi 2 2: 5
unesi 2 3: 6
unesi 3 1: 7
unesi 3 2: 8
unesi 3 3: 9
ispis:
dijagonala1: 3
dijagonala2: 2 6
dijagonala3: 1 5 9
dijagonala4: 4 8
dijagonala5: 7

```

Uputstvo: Za rješenje ovog zadatka trebaće vam dvostruku pokazivač na **int**, moraćete dinamički alocirati niz integer, te ćete sa **for** petljama ispisivati dijagonale.

10. Strukture

Struktura je način da različite varijable nekako grupišemo, npr., ako želimo pamtit i koordinate od 100 tačaka, te svaka ta tačka ima svoj naziv u obliku stringa, tada bi morali imati 3 niza, niz za **y** koordinate **int y[100]**; niz za **x** koordinate **x[100]**;, te niz stringova **char *naziv[100]**; Da bi nekako mogli grupisati te različite podatke, postoje strukture, novodefinisanoj strukturi možemo dati naziv koji želimo, te je ta struktura onda postala korisnički definisani tip podatka:

```

typedef struct
{
    int x,y;
    char *naziv;
}TACKA;

```

Struktura TACKA je sada definisana kao novi tip podatka, da sumiramo, stvaranje novog tipa podatka se vrši tako da se upišu ključne riječi **typedef struct** te onda u vijugavim zagradama navedu varijable koje ta struktura treba sadržavati, poslije zatvorene vijugave zagrade onda upisujem naziv koji želimo dati datoj strukturi, i obavezno tačka zarez na kraju definicije. Postoji još jedan način definisanja korisničke strukture, kojim se sada nećemo baviti.

Deklarisanje niza od 100 varijabli tipa TACKA, vršimo analogno kao i kod osnovnih tipova podataka, dakle **TACKA t[100];** deklarisaće niz od 100 varijabli tipa TACKA, odnosno tačnije, deklarisaće pokazivač TACKA *t , koji će preusmjeriti da pokazuje na 100 * **sizeof(TACKA)** alociranih bita.

Pristupanje članovima niza TACKA zavisi od toga da li pristupamo preko pokazivača ili preko konkretne varijable, ako deklariramo varijablu **to** na sledeći način **TACKA to;** tada toj varijabli možemo vrijednost **x** postaviti na 100 ovako: **to.x = 100;**, naziv joj možemo preusmjeriti ovako: **to.naziv = "Naziv neke tacke";**. Znači zapamtite, članovima konkretne strukture se pristupa preko operatora tacke (.). Ako imamo nešto ovako:

```
TACKA *to;  
to = (TACKA *) malloc( sizeof(TACKA) );  
to -> x = 100;  
to -> y = 110;  
to -> naziv = "Ovo je drugi naziv";
```

Dakle, kada pristupamo preko pokazivača na neki član neke strukture, tada se koristimo operatorom **->**.

Napravimo program koji će generisati 10 različitih tačaka te ih u tekstualnom modu prikazati na ekranu. (Upozorenje: ovaj program koristi funkciju **gotoxy()** koja nije uključena u C standard, tako da ovaj program bez modifikacije možda neće raditi na nekim drugim kompajlerima osim na BC-u:

```
#include <stdio.h>  
#include <conio.h>  
#include <string.h>  
#include <stdlib.h>  
  
typedef struct /*deklaracija novog tipa podatka*/  
{  
int x,y; /*sadrzi dvije int varijable x i y*/  
char *naziv; /*i pokazivac na char*/  
}TACKA;  
  
void iscrtaj_tacku(TACKA *tp); /*deklaracija funkcije*/  
  
int main(void)  
{  
TACKA t[10]; /*staticka deklaracija niza od 10 varijablu tipa TACKA*/  
char priv_str[100]; /*privremeni string*/  
int i,len; /*pomocne varijable*/  
  
clrscr(); /*brisanje ekrana*/  
  
for(i =0;i<10;i++) /*petlja od 0 do 9*/  
{  
t[i].x = rand() % 80 +1; /*postavi slucajnu vrijednost u opsegu od 1 do 80 u x*/  
t[i].y = rand() % 20 +1;  
  
sprintf(priv_str,"%d,%d",t[i].x,t[i].y); /*pisi u string naziv tacke*/  
len = strlen(priv_str); /*odredi duzinu naziva tacke*/  
t[i].naziv = (char *) malloc(len+1); /*alociraj memoriju potrebnu za spremanje naziva */  
strcpy(t[i].naziv,priv_str); /*kopiraj naziv u alociranu memoriju*/
```

```

}

for(i=0;i<10;i++)
{
    iscrtaj_tacku(&t[i]); /*prosledjivanje funkciji iscrtaj_tacku pokazivac
                           na strukturu TACKA, koju zelimo iscrtati */
}

getch();

return 0;
}

void iscrtaj_tacku(TACKA *tp)
{

gotoxy(tp->x,tp->y); /*pristupanje preko pokazivaca vrijednostima x i y, te
                     pomocu funkcije gotoxy postaviti kursor na
                     zeljene koordinate*/
printf("%s",tp->naziv);/*ispis naziva tacke na trenutnu poziciju kursora*/

}

```

Uočimo u gornjem programu da smo prvo stvorili novi tip podatka i nazvali ga **TACKA**. Onda smo deklarirali funkciju **iscrtaj_tacku**, koja prima kao parametar, pokazivač na podatak tipa **TACKA**, u **main** funkciji onda statički alociramo niz od 10 varijabli tipa **TACKA**, te **x**, **y** članove svake te varijable u **for** petlji postavimo na neki slučajni broj unutar opsega koordinata ekrana. Funkcija koja je upotrebljena za dobijanje slučajnog broja zove se **rand()**; za svaku tačku onda još napravimo i naziv te tačke, koji se sastoji od koordinata te tačke. Za to smo koristili funkciju **sprintf**, ta funkcija ponaša se identično kao i **printf** funkcija, samo što ova ispisuje rezultat u string, a **printf** ispisuje rezultat na standardni izlaz (ekran), zbog toga **sprintf** ima jedan dodatni parametar: pokazivač na **char**, u kojeg će ta funkcija upisati željeni string. Valja još uočiti kako se u funkciji **iscrtaj_tacku** pristupa pojedinim članovima varijable tipa **TACKA**, preko operatora **->**. Dalje uočite da u funkciju **iscrtaj_tacku** nismo morali prosljeđivati tri varijable (**x**, **y**, **naziv**), već samo jednu (**TACKA**), dalje prosljeđivanje preko pokazivača na strukturu je brže od prosljeđivanja cijele strukture, te ima još jedna prednost prosljeđivanja pokazivača. Naime, vrijednosti izvorne strukture mogu se preko pokazivača mijenjati, dok kod prosljeđivanja po vrijednosti, morali bi još promjenjenu strukturu vratiti kao rezultat. To kod prosljeđivanja preko pokazivača nije potrebno.

Zadatak 6: Napravite program sa strukturom **UCENIK** koja će sadržavati ime, prezime, ocjene i prosjek. Ocjene trebaju biti niz integera i broj ocjena je za svakog učenika jednak. Prvo program treba unijeti koliko učenika treba unijeti, zatim program mora unijeti broj ocjena koliko ih se za svakog učenika potrebno unijeti, zatim redom program unosi za svakog učenika podatke i računa prosjek. Na kraju program treba ispisati sve učenike poredane po prosjeku.

Uputstvo: Prosjek neka bude **float**, ocjene neka bude pokazivač na **int**, ime, prezime neka budu pokazivač na **char**. Najjednostavniji algoritam za sortiranje po prosjeku koji možete upotrijebiti zove se metoda razmjene. Kod tog algoritma stavite dvije petlje i upoređujete svaki sa svakim, ako je prosjek iz spoljašnje petlje veći od onog iz unutrašnje, tada ih zamijeni.

11. #include i #define

U opštem slučaju: Pomoću **#include<dat.h>** govorite compileru da uzme izvornu datoteku **dat.h** te da je ubaci na mjesto gdje se nalazi **include**, i tek onda neka počne kompajlirati. Na primjer, ako datoteke **dat.h** i **mojprg.c** izgledaju ovako:

dat.h:

```
int mnozi(char a, char b)
{
    return a*b;
}
```

mojprg.c:

```
#include <dat.h>
int main(void)
{
    printf("\nProizvod: %d", mnozi(2, 3));

    return 0;
}
```

Compiler datoteku **mojprg.c** vidi kao:

```
int mnozi(char a, char b)
{
    return a*b;
}
int main(void)
{
    printf("\nProizvod: %d", mnozi(2, 3));

    return 0;
}
```

Kada se **include** datoteka navodi sa znakovima **<>**, tada će compiler zadanu datoteku pokušati pronaći u standardnom **include** direktoriju, međutim kada radimo svoje vlastite **include** datoteke, tada ih možemo pozivati i sa navodnicima, npr:

```
#include "c:\bc\mojinc.h"
```

Pomoću naredbe **#define** možemo nekoj vrijednosti dati neko simboličko ime, te kasnije možemo to simboličko ime upotrebljavati u kodu, a compiler će umjesto imena stavljati na to mjesto vrijednost tog simboličkog imena. Npr:

```
#define MAX_X 640
```

definisana je konstanta **MAX_X**, te joj se pridružuje vrijednost 640, kasnije u kodu možete onda imati nešto tipa:

if(x > MAX_X) return 0; Umjesto brojčanih vrijednosti možete konstantama pridruživati i stringove, čak i funkcije, itd... Prednost upotrebe konstanti je u tome da ih lako možete mijenjati, te ne morate prelaziti cijeli kod svaki put kada neku specifičnu vrijednost mijenjate.

12. Stvari koje sam zbog jednostavnosti prešutio

Kada BORLANDC radi sa ekstenzijom **.CPP**, tada on program tretira kao **C++** program, dok ekstenzija **.C** odgovara C programu.

U **C/C++** tip podatka **int** nije uvijek 16bitne dužine, već njegova dužina ovisi od kompilera do kompilera, pod 16bitnim DOS-ovim compilerima integer je dužine 16, međutim pod 32bitnim DOS-ovim compilerima kao npr. **DJGPP int** je dužine 32 bita, u MS VISUAL C-u **int** je takođe 32 bita.

U C-u nije potrebno **void *** pointer pretvarati u željeni oblik, kao što smo to radili tokom tutorijala, dok je u **C++** to itekako potrebno. Međutim, C-u ne smeta, tako da sam za svaki slučaj uvijek postavio i tu pretvorenu.

Gdje god sam koristio izraz da compiler nešto alocira, ili slično, to je bila laž, naime compiler je onaj koji stvara potreban kod koji će alocirati.

Pointer u DOS-u ne mora uvijek biti dužine 2 bita. Postoje **far** pointeri koji su uvijek 4bita veličine, a kada radite u **LARGE** memorijskom modelu tada se svi pointeri **far** pointeri.

KRAJ

Došli smo do kraja ovog kratkog putovanja kroz programski jezik C. Vjerujem da ste nešto naučili, ali isto tako sam siguran da su vam se tek sada u glavi pojavila mnoga pitanja na koja ovdje ne možete naći odgovor. U tu svrhu preporučujem vam da sebi nabavite neku knjigu, da uzmete vremena i krenete na daljnja istraživanja.

Zamolio bih vas samo jednu stvar, da mi pošaljete mail sa svojim komentarom na miroslav@prijedor.com.

C Tutorijal - rješenja zadataka

Zadatak 1:

```
#include <stdio.h>
#include <conio.h>

int main(void)
{
    int suma,i,x;

    suma = 0;

    for(i=1;i<=10;i++)
    {
        printf("\nUnesi broj %d: ",i);
        scanf("%d",&x);
        suma += x;
    }

    printf("\nProsjek: %d",suma / 10);          /* neispravniji nacin */
    printf("\nProsjek: %f",(float)suma / (float)10); /* ispravniji nacin */

    getch();

    return 0;
}
```

Zadatak 2:

```
#include <stdio.h>
#include <conio.h>
```

```

int main(void)
{
char c;
int b;

b = 0;
printf("\nPritisici tipke, q za kraj\n");

do{

c = getche();
b++;

}while(c != 'q');

printf("\nBroj unesenih tipki: %d",b);

return 0;

}

```

Zadatak 3:

```

#include <stdio.h>
#include <conio.h>

int main(void)
{
int i,max,b[100];

max = 0;
do{

printf("\nUnesi broj: ");
scanf("%d",&b[max]);
max++;

if(max == 100) break; /*prekidanje petlje ako je uneseno 100 brojeva */

}while(b[max-1] != 0);

max-=2; /*max se mora smanjiti za dva zato jer je za dva prevelik
        probajete maknuti taj red */

printf("\nSada obrnuto:");

for(i=max;i>=0;i--)
{
printf("\nBroj: %d: %d",i,b[i]);
}

getch();

```

```
return 0;
}
```

Zadatak 4:

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
#include <stdlib.h>
```

```
int main(void)
{
    int duzina,i;
    char str1[200],*str2; /*pretpostavljamo da uneseni
    string nece biti duzi od 200 znakova*/

    printf("\nUnesi string: ");
    gets(str1);

    duzina = strlen(str1);
    str2 = (char *) malloc(duzina + 1); /*alociranje duzina + 1 znak, dodatni
    znak ce poslužiti za null znak*/

    for(i=0;i<duzina ;i++)
    {
        str2[i] = str1[duzina - i-1];
    }
    str2[duzina] = 0;

    printf("\nUneseni: %s\nObrnuti: %s",str1,str2);

    return 0;
}
```

Zadatak 5:

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
#include <stdlib.h>
```

```
int main(void)
{
    int vel, **tabela,i,t,brD=0;
    printf("\nUnesi velicinu kvadratne matrice: ");
    scanf("%d",&vel);
    tabela = (int **) malloc(vel * sizeof(int*));
```

```

for(i=0;i<vel;i++)
{
    tabela[i] = (int *) malloc(vel * sizeof(int));
    for(t=0;t<vel;t++)
    {
        printf("\nUnesi clan %d %d: ",i,t);
        scanf("%d",&tabela[i][t]);
    }
}

printf("\nRezultat: \n");

for(i=vel-1;i>=0;i--)    /*vrti petlju gdje je i pocetna kolona dijagonale */
{
    printf("\nDijagonala%d: ",++brD); /*ispisi redni broj dijagonale */
    for(t=0;t<vel;t++)
    {
        if(i+t >= vel)continue; /*ako je i+t vece od vel tada nastavi sa sledecim */
        printf(" %d",tabela[t][i+t]);
    }
}

for(i=1;i<vel;i++)    /*vrti petlju gdje je i pocetni red dijagonale */
{
    printf("\nDijagonala%d: ",++brD);
    for(t=0;t<vel;t++)
    {
        if(i+t >= vel)continue;
        printf(" %d",tabela[i+t][t]);
    }
}

getch();

return 0;
}

```

Zadatak 6:

```

#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <string.h>

/*stvaranje strukture:*/
typedef struct
{
    char *ime;
    char *prezime;
    int *ocjene;
    float prosjek;
}UCENIK;

```

```

/*globalne varijable*/
int brUc,brOc;

/*deklaracija funkcija*/
char *unesi_str(void);
void izracunaj_prosjek(UCENIK *uc);

int main(void)
{
int i,t,priv,*redoslijed; /*redoslijed služi za slaganje po prosjeku*/
UCENIK *ucenik;

printf("\nUnesi broj ucenika: ");
scanf("%d",&brUc);
printf("\nUnesi broj ocjena po uceniku: ");
scanf("%d",&brOc);

ucenik = (UCENIK *) malloc(brUc * sizeof(UCENIK)); /*alokacija*/
if(!ucenik) { printf("\nOut of memory!"); return; } /*provjera*/
redoslijed = (int *) malloc(brUc * sizeof(int));
if(!redoslijed) { printf("\nOut of memory!"); return; }

for(i = 0;i<brUc;i++)
{
redoslijed[i] = i; /*pretpostavi poredak*/
ucenik[i].ocjene = (int *) malloc(brOc * sizeof(int));
if(!ucenik[i].ocjene){ printf("\nOut of memory!"); return; }

printf("\nUnesi ime za ucenika %d: ",i+1);
ucenik[i].ime = unesi_str();
printf("\nUnesi prezime za ucenika %d: ",i+1);
printf("\nUnesi prezime za ucenika %d: ",i+1);
ucenik[i].prezime = unesi_str();

for(t=0;t<brOc;t++)
{
do{
printf("\nUnesi ocjenu %d za ucenika %d: ",t+1,i+1);
scanf("%d",&priv);
}while(priv < 1 || priv > 5);/*dali je unesena ocjena iz skupa brojeva 1-5*/
ucenik[i].ocjene[t] = priv;
}

izracunaj_prosjek(&ucenik[i]); /*pozovi funkciju za racunanje prosjeka*/
}

/*sortiranje metodom razmjene*/
for(i=0;i<brUc;i++)
{
for(t=0;t<brUc;t++)
{
if(ucenik[redoslijed[i]].prosjek > ucenik[redoslijed[t]].prosjek)

```

```

    {
        /*ako je iz prosjek iz spoljasnje petlje veci, tada zamjeni*/
        priv = redoslijed[i];
        redoslijed[i] = redoslijed[t];
        redoslijed[t] = priv;
    }
}

/*ispis*/
for(i=0;i<brUc;i++)
{
    printf("\nUcenik rednog broj: %d:",redoslijed[i]+1);
    printf("\nIme      :%s",ucenik[redoslijed[i]].ime);
    printf("\nPrezime   :%s",ucenik[redoslijed[i]].prezime);
    printf("\nProsjek    :%f\n",ucenik[redoslijed[i]].prosjek);

}

return 0;

}

/*funkcija za unos stringa*/
char *unesi_str(void)
{
    char priv_str[150],*s; /*buffer od 150 znakova*/
    int len;

    scanf("%s",priv_str);
    len = strlen(priv_str);
    s = (char *)malloc(len + 1);
    if(!s){ printf("\nOut of memory!"); exit(1); }
    strcpy(s,priv_str);
    return s;
}

void izracunaj_prosjek(UCENIK *uc)
{
    int i,zbir=0;

    for(i = 0; i<brOc;i++)
    {
        zbir += uc->ocjene[i];
    }
    uc->prosjek = (float)zbir / (float)brOc;
}

```

